



山东大学



计算思维

程序与算法基础（8学时）

千载文脉凝风骨，根植山大盛芳华

主讲人：李琨

时间：2025年秋季学期



第3章 问题求解与算法

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

知识要点

问题,
问题求解,
问题归约,
与或图,
基元问题,
问题求解系统,
问题求解策略,
算法式,
启发式,
问题解决过程,
抽象, 数学模型, 数学建模, 哥尼斯堡七桥问题
计算机求解问题概念模型。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

人类问题求解的思维过程

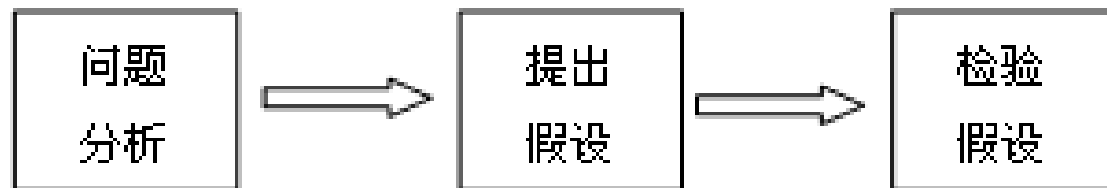
• 问题及其来源

- 问题是指需要解决还没有解决的事。
- 来源
 - 社会实践、科学研究中发现和提出问题
 - 他人的任务

• 问题求解

- 问题求解是人们为寻求问题答案，对问题及其所处的环境，根据知识和经验、条件、约束而进行的一系列思维活动。
- 问题千差万别，问题求解策略不同，手段和方法各异

• 问题求解思维过程



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

问题分析

- 对问题给出的条件、目标和任务进行研究，明确问题的基本含义。
- 问题分析通常借助于数学、逻辑学等方法，例如：问题归约、问题抽象、数学建模等，其结果是问题的形式化，或建立一个抽象的问题模型，使问题的描述更加严谨。
- 例：
 - 科学家们在观察自然现象时发现，某些植物在夜晚会关闭叶片边缘的小孔（气孔），而在白天则打开。他们想了解这一现象背后的原因。
 - 一家零售公司注意到其在线销售额增长缓慢，而竞争对手的销售却在快速增长。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

提出假设

- 当问题分析清楚后，接下来是要寻找问题的答案，而答案和求解方法都是未知的。
- 提出假设就是在问题和答案之间寻找解决问题的方法和途径。
- 例：
 - 一个可能的假设是植物关闭气孔是为了减少水分的蒸发损失（蒸腾作用），因为夜晚没有光合作用，不需要大量的水分来制造食物。
 - 可能是该公司网站的用户体验不如竞争对手的好，导致顾客更倾向于在其他地方购物。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



检验假设

- 对提出的假设，进行验证，来检查其是否正确。
- 实践检验
 - 对问题给定相应的数据，检查假设的结果，看是否正确，从而来判断假设的真伪。
 - 许多物理、化学、生物实验就是这种模式。
- 理论验证
 - 假设是不可实验的
- 例：
 - 科学家们可以在实验室中设置实验，对比不同光照条件下植物的气孔活动情况，并测量其水分流失的速度。如果结果支持假设，那么科学家们就可以得出结论，植物关闭气孔是为了减少水分损失。
 - 公司可以进行网站改版，并邀请一部分用户试用新版网站，并收集反馈。同时，也可以进行A/B测试，即让一部分用户继续使用旧版网站，另一部分用户使用新版网站，然后比较两个版本之间的转化率差异。



领域问题及形式化描述

- 问题的要素

- 现实，已经具备的条件基础
- 目标，对预期结果的描述或一种要求

- 问题形式化

问题= {现实，目标}

题解=目标 - 现实= {A1, A2, ..., An}

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

问题归约表示

• 问题归约

- 对于复杂的问题，直接进行问题求解往往是困难的
- 问题归约就是对问题进行归纳和简化，从而把一个复杂问题转换为相对简单的问题。

• 三要素

- 目标：即问题的初始描述。
- 算子集：用来将给定问题变换为若干子问题的操作。
- 基元问题集：已有解或其解十分明显可以直接描述的问题。

• 例：

- 目标：计算复杂多项式 $P(x) = x^5 + 3x^4 - 2x^3 + x - 7$ 的值
- 算子集：
 - 计算 x^n ，其中 n 是非负整数。
 - 将两个多项式相乘。
 - 将两个多项式相加。
- 基元问题集：
 - 计算 x 的幂次方（ $x^5, 3x^4, -2x^3, x$ ），这是基础数学运算，可以直接执行。
 - 加法和乘法也是基础的数学运算，可以直接执行。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

与或图

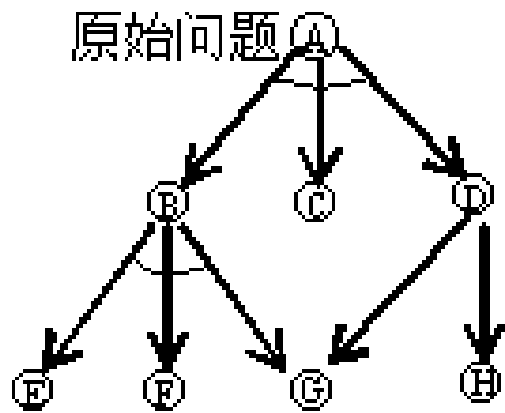
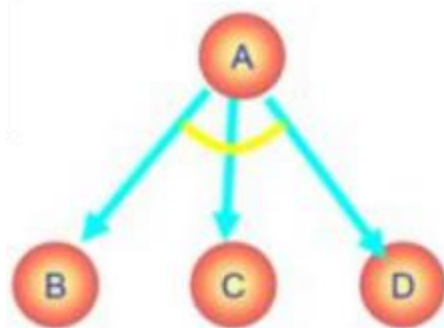
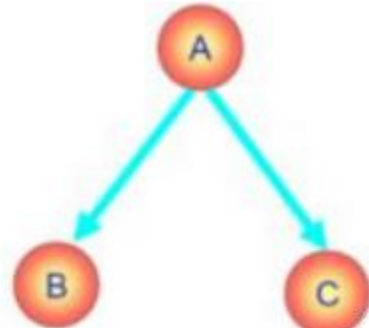


图 3-1 问题的归约表示

- “或”节点，原来问题被变换为若干子问题，而只需要解决其中之一便可解决原问题，代表这些子问题的节点称为相对于原问题的“或”节点。
- “与”节点，如果原问题被变换为缺一不可（均需解决）的若干子问题，那么代表这些子问题的节点称为相对于原问题节点的“与”节点，



与图



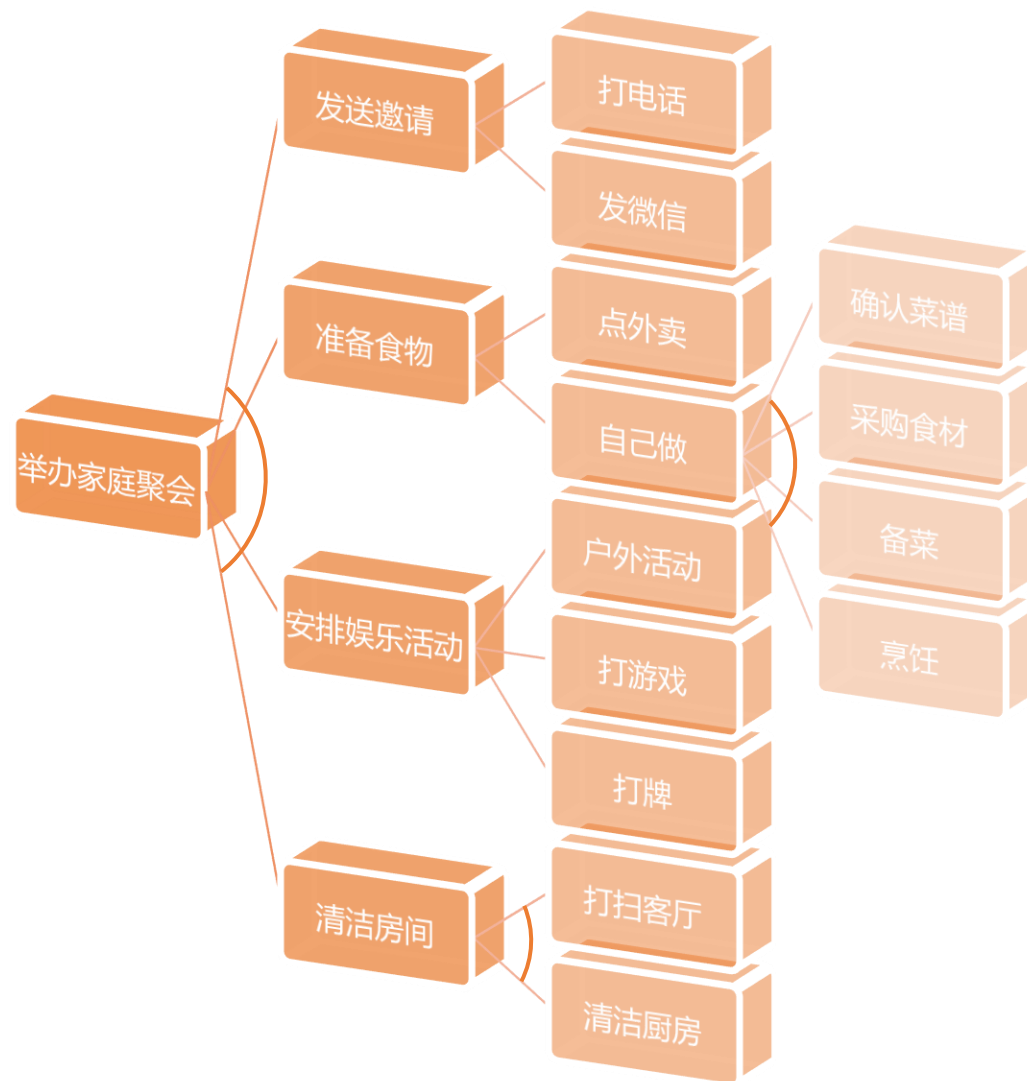
或图

与或图

• 问题：举办家庭聚会

• 子问题分解：

- 发送邀请[OR]
 - 打电话[基元问题]
 - 发微信[基元问题]
- 准备食物[OR]
 - 点外卖[基元问题]
 - 自己做[AND]
 - 确定菜谱[基元问题]
 - 采购食材[基元问题]
 - 备菜[基元问题]
 - 烹饪[基元问题]
- 安排娱乐活动[OR]
 - 户外活动[基元问题]
 - 打游戏[基元问题]
 - 打牌[基元问题]
- 清洁房间[AND]
 - 打扫客厅[基元问题]
 - 清洁厨房[基元问题]



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



问题求解策略--算法式

- 算法式是指按照逻辑来求解问题的策略。如果问题有解，算法式一定可以得到问题的答案或解。
- 例如
 - 解一个6个字母的字谜，只要将这6个字母进行全排列，一定可以找到这个字。为了找到这个字，最坏情况下需要尝试720中不同的排列。
- 常用的算法策略：枚举方法
- 缺点：费时

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

Fibonacci sequence

•斐波那契序列: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, ...

•通项公式:
$$a_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]$$

•递推公式:

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

•特性: 斐波那契数列虽然是由自然数组成的, 但是其通项公式却涉及无理数。

•极限比值: 当 n 趋向于无穷大时, 斐波那契数列相邻两项的比值趋近于黄金分割比, 即: $\frac{\sqrt{5}-1}{2}$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

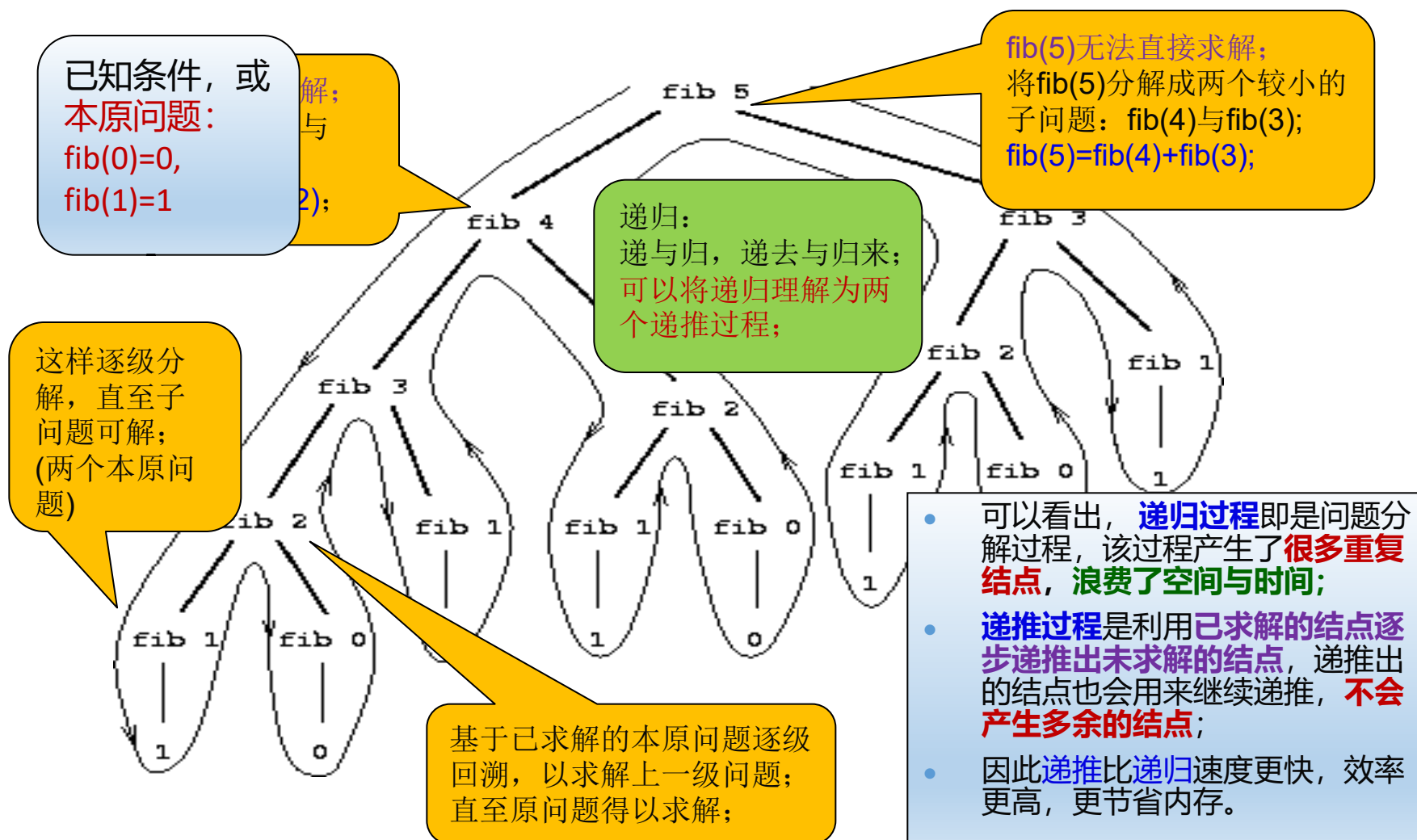
3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题





问题求解策略--启发式

- 根据以往解决问题的经验形成一些经验规则。启发式不能保证一定得到答案。
- 例如
 - 计算机突然不能上网。可能是网线没接好，也可能是网络协议问题，或者是病毒造成的。
- 常用的启发式策略
 - 手段目的分析，是指不断地将当前状态和目标状态进行比较，然后采取措施尽可能地缩小这两个状态之间的差异。
 - 顺向工作，也称顺向推理，是指从问题的已知条件出发，通过逐步扩展已有的信息直到问题解决的一种策略。
 - 逆向工作，又称逆向推理，是指从问题的目标状态出发，将问题解决的目标分解成若干子目标，直至使子目标按逆推途径与给定的条件建立直接联系或等同起来。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

问题求解系统

- 求解问题就是要求解一个问题的结果，或找出一种从现实到目标的行动序列，并予以执行。

- 问题求解状态图

- 状态
- 活动

- 问题的解

活动序列A2-A4-A6

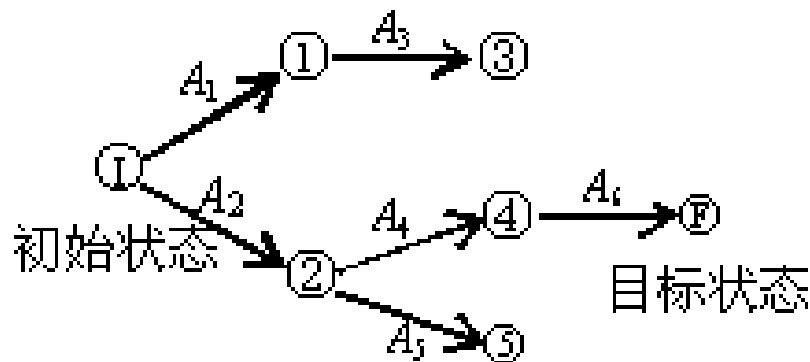


图 3-2 问题求解概念图

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

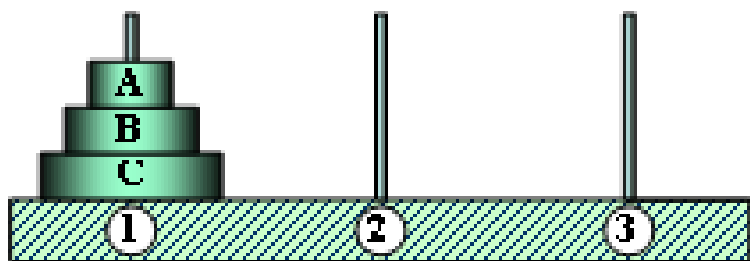
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

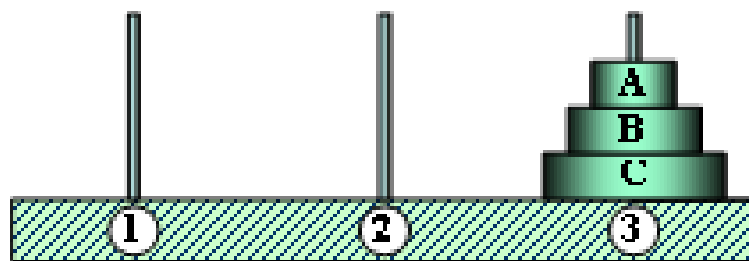
3.6 网络搜索问题

例3-1汉诺塔问题 (Tower of Hanoi Problem)

- 在印度，有一个古老传说。在世界中心贝拿勒斯的圣庙里，安放着一块黄铜板，板上插着三根宝石针。梵天（印度教的主神勃拉玛）在创造世界的时候，在其中一根针上从下往上放下了由大到小的64个金片。这就是所谓梵塔，又称汉诺塔。
- 不论白天黑夜，都有一个值班的僧侣按照梵天不渝的法则，把这些金片在三根针上移来移去：一次只能移一片，并且要求不管在哪根针上，小片永远在大片的上面。当所有的64片都从梵天创造世界时所放的那根针上移到另外一根针上时，世界就将在一声霹雳中消灭，梵塔、庙宇和众生都将同归于尽。



(a) 初始状态

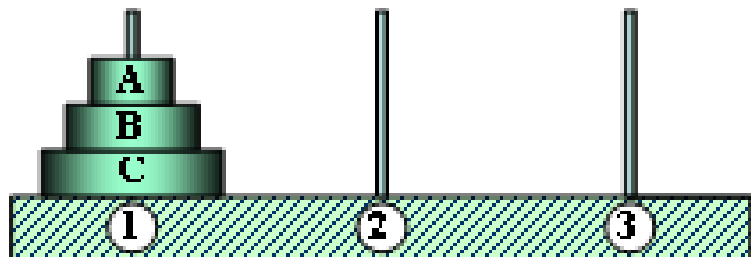


(b) 目标状态

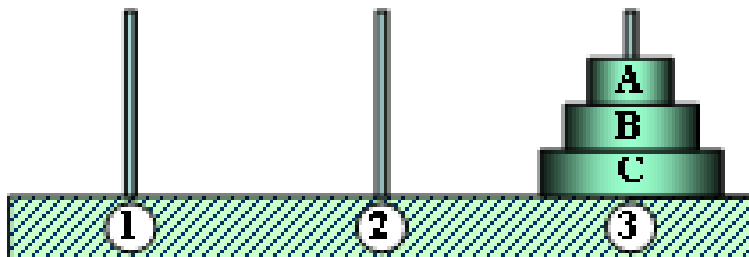
图 3-3 Hanoi 塔问题

汉诺塔问题求解策略

- 汉诺塔问题 (Tower of Hanoi Problem)



(a) 初始状态



(b) 目标状态

图 3-3 Hanoi 塔问题

- 启发式求解策略
 - 先把上面的 $N-1$ 个盘子从第1根柱子上搬移到第2根柱子上,
 - 然后把第1根柱子上最下面的盘子搬移到第3根柱子上
 - 然后把第2根柱子上的 $N-1$ 个盘子搬到第3根柱子上

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

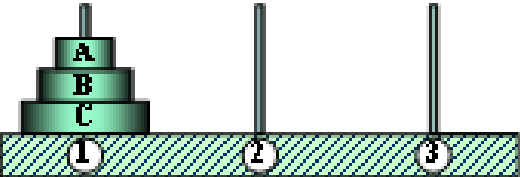
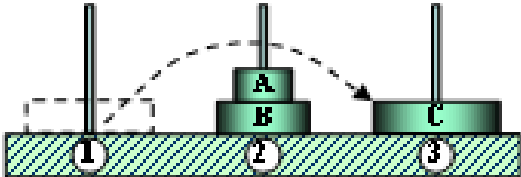
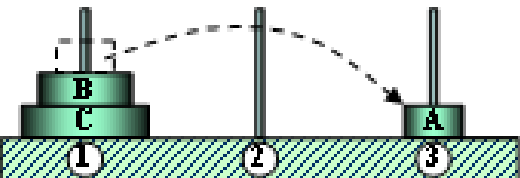
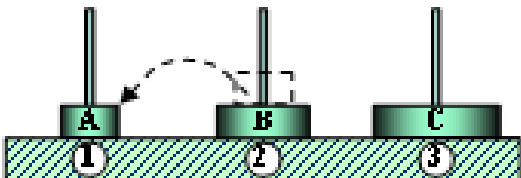
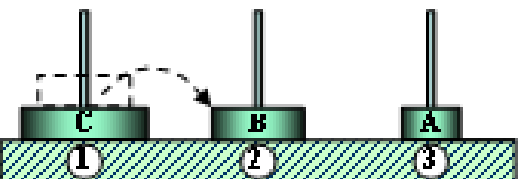
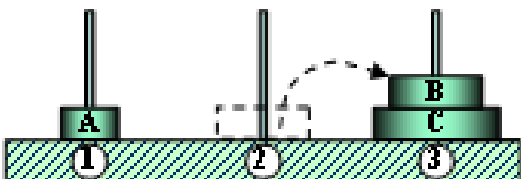
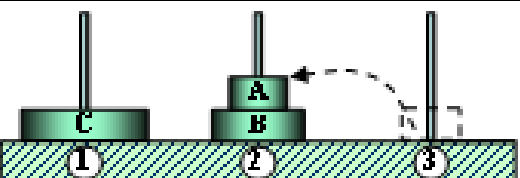
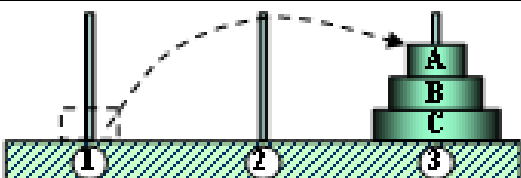
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

汉诺塔问题解

表 3-1 Hanoi 塔的搬移过程

序号	搬移	状态	序号	搬移	状态
0			4	1->3	
1	1->3		5	2->1	
2	1->2		6	2->3	
3	3->2		7	1->3	

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



提出问题

• 发现问题，提出问题

当 $n=64$ 时，需要搬移的次数是 2^{64} ，有： $2^{64} = (2^{10})^{6.4} > (10^3)^{6.4} = 10^{19.2}$ ，

设每秒钟计算搬移 1 亿次，有：1 年 $= 365 \times 24 \times 3600$ 秒 $= 31536000 \approx 3.15 \times 10^7$ 秒，

要计算出 64 个盘子的搬移顺序，仍需要约 10^4 年！

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



问题分析方法之一——抽象

• 抽象的概念

- 抽象 (Abstraction) , 就是从众多的事物中抽取出共同的、本质性的特征, 而舍弃其非本质的特征。
- 抽象是一种重要的思维方法。
- 共同特征是指那些能把一类事物与他类事物区分开来的特征, 这些具有区分作用的特征又称**本质特征**。
- 例如, 对苹果、香蕉、葡萄等比较, 它们共同的特性就是水果, 从而抽象出水果这一**概念**。

• 抽象的层次性

- 例如, 对于在校大学生这一群体, 从低到高, 我们可以抽象为: 大学生 (具有高等学校学历)、学生 (学习状态)、年轻人 (按照年龄属性)、人 (社会属性)、动物 (生物学特征)、生物 (生物学特征)、物质 (按哲学观点) 等。
- 从不同的层次去认识和观察事物, 是人们分析问题和思维的一部分。



事物抽象的方法

- 分离，就是暂不考虑研究对象与其它对象的联系，把要研究的对象单独的抽取出来进行考察，研究其属性。
- 提纯，就在研究对象的错综复杂的属性和联系中，去掉那些与研究目标关系不大的内容，更好的揭示其本质特征。
- 简略，提纯本身就是一种简化，简略是抽象的最后环节，它是对研究对象抽象结果的一种表达。目的都是要表达研究对象的本质性特性，从而产生一类新的概念。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



数学模型与数学建模

- 数学模型，对实际问题的数学抽象，用数学符号、数学式子、程序、框图等对实际问题本质属性的抽象而又简洁的刻画，用以描述客观事物的特征、内在联系及发展和运动规律。
- 通过数学模型，可以解释某些客观现象，预测未来发展规律，或能为控制某一现象的发展提供某种意义下的优化策略。
- 数学模型是现实问题的抽象，数学模型用数学语言描述了现实问题的特征及其内部联系或与外界的联系。
- 应用知识从实际问题中进行抽象、提炼出数学模型的过程称为数学建模（Mathematical Modeling）

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

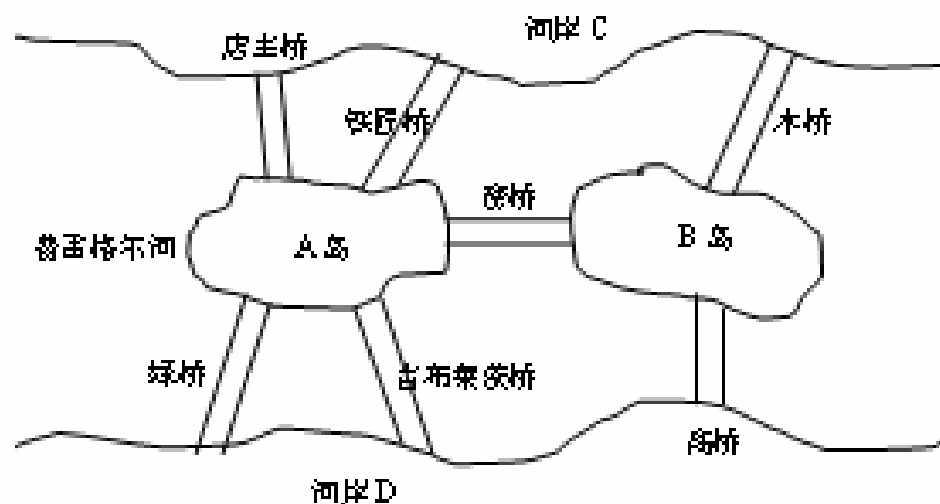
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

哥尼斯堡七桥问题

- 18世纪初，在普鲁士的哥尼斯堡（德国时期旧称哥尼斯堡 Koenigsberg，现为俄罗斯的加里宁格勒，1946年由德国划入前苏联）的一个公园里，普雷格尔河从中穿过，河中有两个小岛，有七座桥把两个岛与河岸联系起来
- 有人提出一个问题：一个步行者怎样才能不重复、不遗漏地一次走完七座桥。



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

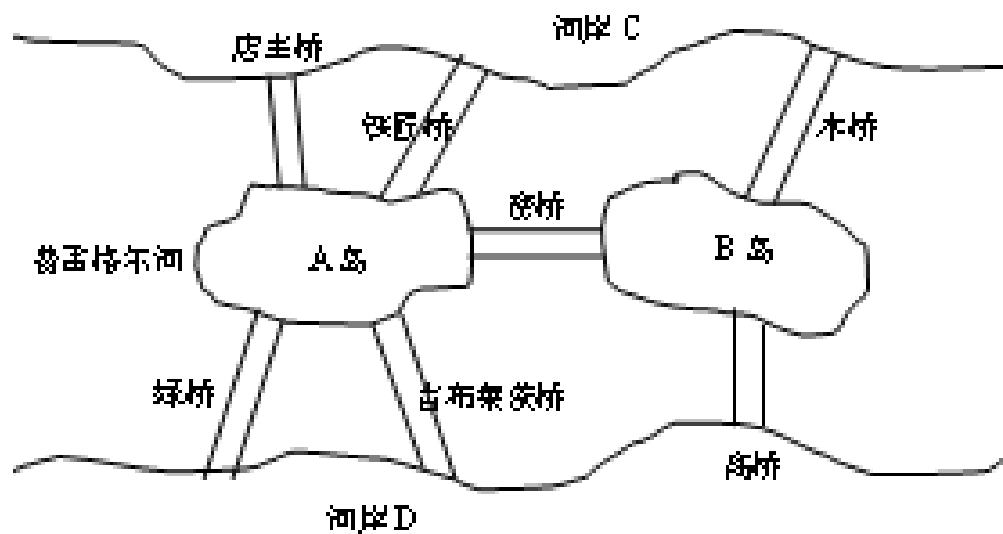
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

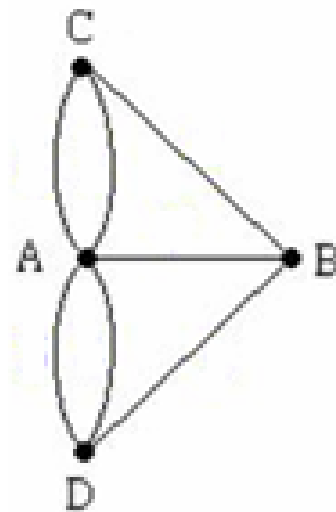
3.6 网络搜索问题

七桥问题的数学抽象

- 1736年，29岁的欧拉 向圣彼得堡科学院递交了关于哥尼斯堡七桥问题的论文—“**与位置几何相关的一个问题的解**”。在论文中，欧拉将七桥问题抽象出来，把每一块陆地考虑成一个点，连接两块陆地的桥以线表示。并由此得到了如图一样的几何图形



(a) 七桥问题



(b) 七桥问题的图形抽象

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

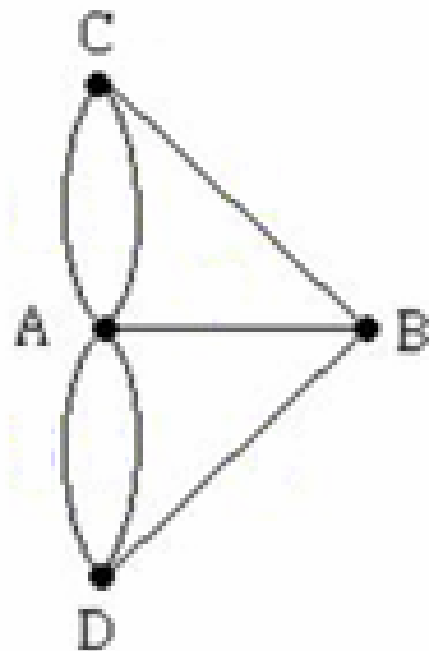
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

图论



- 欧拉把问题归结为如图的“一笔画”问题，证明上述走法是不可能的，即从任意一点出发不重复的走遍每一座桥，最后再回到原点是不可能的。
- 在解答问题的同时，欧拉开创了数学的一个新的分支—图论（Graph Theory）。图论的创立，为人们问题求解提供了一种新的数学理论和工具。



模型的分类--按特性分 (1)

- 静态模型和动态模型
 - 静态模型是指要描述的系统各量之间的关系是不随时间的变化而变化的，即与时间变量无关，一般都用代数方程来表达。
 - 动态模型是指描述系统各量之间随时间变化而变化的规律的数学表达式，一般用微分方程或差分方程来表示。
- 连续时间模型和离散时间模型
 - 时间变量是在一定区间内变化的模型称为连续时间模型，用微分方程描述
 - 将时间变量离散化，所获得的模型称为离散时间模型，用差分方程描述



模型的分类--按特性分 (2)

- 参数模型与非参数化模型
 - 参数模型是指可以采用有限个参数进行描述的数学模型，并给定模型参数的值域。
 - 非参数模型是指系统的数学模型中非显式地包含可估参数。
- 集中参数模型和分布参数模型
 - 集中参数模型是指描述系统的特征、动态不随空间坐标变化的模型，通常把系统看作一个整体，只研究输入与输出之间的拟合关系，而对系统内部的过程和机理不予考虑，即把所有要素都看作属于空间一个点。
 - 分布参数模型则是描述系统特征、动态随空间坐标变化的模型。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



模型的分类--按特性分(3)

- 随机性模型和确定性模型
 - 确定性模型中变量间的关系是确定的
 - 随机性模型中变量之间的关系是以统计值或概率分布的形式给出的
- 线性模型和非线性模型
 - 线性模型中各量之间的关系是线性的，可以应用叠加原理，即几个不同的输入量同时作用于系统的响应，等于几个输入量单独作用的响应之和。
 - 非线性模型中各量之间的关系不是线性的，不满足叠加原理。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



模型的分类--按数学方法

- 几何模型
- 图论模型
- 微分方程模型
- 线形规划
- 非线性规划模型
- 概率论模型
- 等等

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



模型的分类--按应用领域

- 经济学数学模型
- 管理学数学模型
- 社会学数学模型
- 地质学数学模型
- 气象学数学模型
- 天文学数学模型
- 工程学数学模型
- 生物学数学模型
- 医学数学模型 等等

此外，还有许多特殊的问题模型，如：交通运输问题模型、经济决策模型等。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

数学建模的基本步骤

- 模型准备
- 模型假设
- 模型建立
- 模型求解
- 模型分析
- 模型检验
- 模型应用



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



计算机问题求解

- 数学模型可以对问题进行严谨的形式化描述
- 计算机系统可以实现问题求解的自动化
 - 计算机的海量存储、高速度，高精度、自动运行和不知疲倦
 - 设备智能化
 - 计算机在各种各样的问题求解中正在发挥越来越重要的作用

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

计算机问题求解概念模型

• 计算机问题求解概念模型

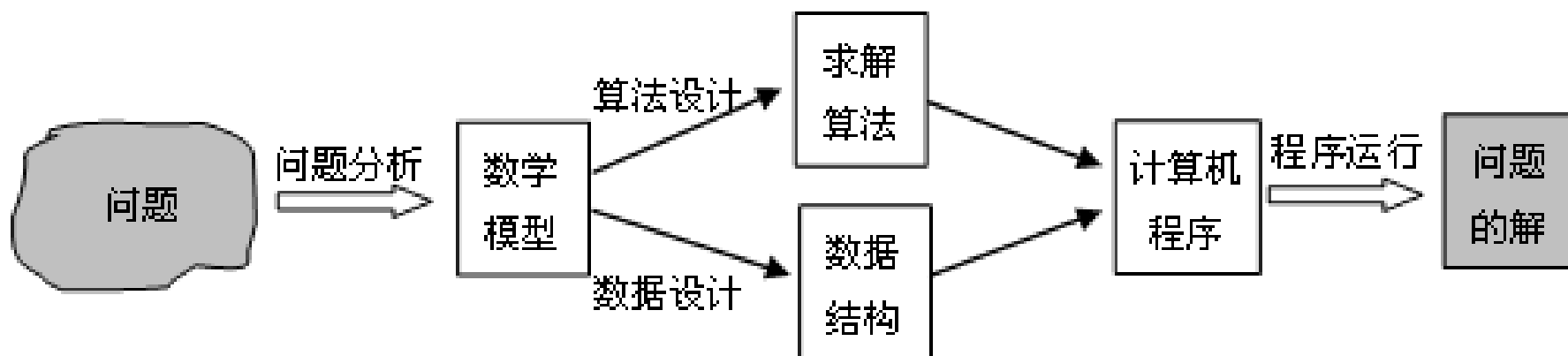


图 3-6 计算机求解问题概念模型

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



举例1：阿基米德群牛问题

- 太阳神有一牛群，由白、黑、花、棕四种颜色的公、母牛组成。在公牛中，白牛数多于棕牛数，多出之数相当于黑牛数的 $\frac{1}{2} + \frac{1}{3}$ ；黑牛数多于棕牛数，多出之数相当于花牛数的 $\frac{1}{4} + \frac{1}{5}$ ；花牛数多于棕牛数，多出之数相当于白牛数的 $\frac{1}{6} + \frac{1}{7}$ 。在母牛中，白牛数是全体黑牛数的 $\frac{1}{3} + \frac{1}{4}$ ；黑牛数是全体花牛数 $\frac{1}{4} + \frac{1}{5}$ ；花牛数是全体棕牛数的 $\frac{1}{5} + \frac{1}{6}$ ；棕牛数是全体白牛数的 $\frac{1}{6} + \frac{1}{7}$ 。
- 问这牛群是怎样组成的？

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

问题分析

- 这是一个看起来很复杂的问题，问题包含了许多约束条件，求解的量也很多。
- 为了更好的理解问题的含义，引入数学变量，设：白、黑、花、棕四种颜色的公牛、母牛数量分别为 x_1, x_2, x_3, x_4 和 y_1, y_2, y_3, y_4 ，这样可以将要求的问题解表示成一个表格

表 3-1 阿基米德分牛问题数据表

	白色	黑色	花色	棕色
公牛	x_1	x_2	x_3	x_4
母牛	y_1	y_2	y_3	y_4

数学建模

表 3-1 阿基米德分牛问题数据表

	白色	黑色	花色	棕色
公牛	x1	x2	x3	x4
母牛	y1	y2	y3	y4

- 在公牛中
 - 白牛数多于棕牛数，多出之数相当于黑牛数的 $\frac{1}{2} + \frac{1}{3}$;
 - 黑牛数多于棕牛数，多出之数相当于花牛数的 $\frac{1}{4} + \frac{1}{5}$;
 - 花牛数多于棕牛数，多出之数相当于白牛数的 $\frac{1}{6} + \frac{1}{7}$ 。
- 在母牛中
 - 白牛数是全体黑牛数的 $\frac{1}{3} + \frac{1}{4}$;
 - 黑牛数是全体花牛数 $\frac{1}{4} + \frac{1}{5}$;
 - 花牛数是全体棕牛数的 $\frac{1}{5} + \frac{1}{6}$;
 - 棕牛数是全体白牛数的 $\frac{1}{6} + \frac{1}{7}$ 。
- 根据问题的约束条件，可以建立下列约束关系

$$\left\{ \begin{array}{l} x1-x4 = x2 \left(\frac{1}{2}+\frac{1}{3} \right) \\ x2-x4 = x3 \left(\frac{1}{4}+\frac{1}{5} \right) \\ x3-x4 = x1 \left(\frac{1}{6}+\frac{1}{7} \right) \\ y1 = (x2+y2) \left(\frac{1}{3}+\frac{1}{4} \right) \\ y2 = (x3+y3) \left(\frac{1}{4}+\frac{1}{5} \right) \\ y3 = (x4+y4) \left(\frac{1}{5}+\frac{1}{6} \right) \\ y4 = (x1+y1) \left(\frac{1}{6}+\frac{1}{7} \right) \end{array} \right.$$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



3.2 算法与算法分析

- 算法及其描述
 - 算法的特征
 - 算法描述
 - 算法的正确性
- 算法复杂性分析
 - 时间复杂性分析
 - P问题与NP问题
 - 空间复杂性

知识要点

算法，
算法描述，
流程图，
N-S流程图，
伪代码，
算法分析，
算法复杂性，
P问题，
NP问题，
NP完全问题，
NP难度问题，
时间复杂性。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法及其特征

• 算法的概念

- 算法 (Algorithm) 一词最早出现在数学中，原意是关于数字的运算法则。
- 算法指问题求解的方法及求解过程的描述，是一个精心设计的计算序列，用以解决一类特定的问题。

• 算法的特征

- 确定性：算法中的每一条指令必须有确定的含义，不能产生二义性。
- 可行性：算法描述的步骤在计算机上是可行的。
- 有穷性：一个算法必须在执行有穷步后结束，每一步必须在有穷的时间内完成。
- 输入：一个算法可以有零个或多个输入。
- 输出：算法执行过程中或结束后要有输出结果，或者产生相应的动作指令。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



算法基本结构

• 顺序结构

- 按照从上到下依次执行算法步骤的结构就是顺序结构，它是最简单、最基本的算法控制结构。

• 分支结构（选择结构）

- 能根据是否满足**给定条件**而**执行不同步骤**的算法控制结构
- 分支结构的条件判断结果一般只有“是”或“否”两种情况，并且一定会选择“是”或“否”中的一个分支中的步骤执行，不能两个都执行，也不能两个都不执行。

• 循环结构

- 算法在一定条件下重复执行某些步骤的控制结构。这里的“一定条件”就是判断循环是否要进行下去的**循环条件**，“重复执行的某些步骤”称为**循环体**。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



算法描述——自然语言描述

- 用自然语言将问题的求解步骤清晰的表述出来即可。在步骤描述上，要求语言简练，层次清晰。
- 为表述方便，每一步可以加上标号，例如Step1, Step2...等
- 对于复杂的步骤，可以进行进一步展开，如Step1.1,Step1.2等
- 采用自然语言描述算法可以较好的描述问题的**大的求解思路**，对于比较精细的求解步骤自然语言在描述上比较困难。
 - 例如，对于判断、分支、重复，特别是这些逻辑的嵌套，自然语言描述时显得不够清晰直观。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

自然语言描述举例

【例 3-1】对于正整数 n ，求和式 $\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n$ 的值。

对于上述问题，采用自然语言描述求解过程，算法描述如下：

算法 3-1 求正整数 n 的和式算法

Step1: 输入正整数 n ;

Step2: 设和式用 Sum 表示，且为 Sum 赋初值为 0;

Step3: 自然数 i 赋初值为 1;

Step4: 将 i 的值累加到 Sum

4.1 若 $i \leq n$ 则

$Sum \leftarrow Sum + i$;

$i \leftarrow i + 1$;

转 Step4;

4.2 若 $i > n$ 转 Step5

Step5: 输出 Sum ;

Step6: 结束。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法描述——流程图描述

• 流程图（Flow Diagram）

- 由图框和流程线组成的图形，**图框**表示各种类型的操作，图框中的文字和符号表示操作的内容，**流程线**表示操作的先后次序。
- 流程图可以很好的表达顺序、分支和重复逻辑，可以较好的描述数据处理、算法描述及系统功能描述等

• 流程图常用图形符号

含义	符号	解释说明
起止框		代表算法的开始 / 结束
输入、输出框		代表算法的输入 / 输出
处理框		代表算法执行的操作
判断框		标志算法的“判断”操作
流程线		标志算法的执行顺序

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

流程图举例

【例 3-1】对于正整数 n ，求和式 $\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n$ 的值。

• 优点

- 简单直观，可很好的表达顺序、分支和重复逻辑结构，可以很自然的将算法流程图转化成计算机程序。

• 不足

- 对于复杂的算法，流程图手工绘制和修改都比较麻烦

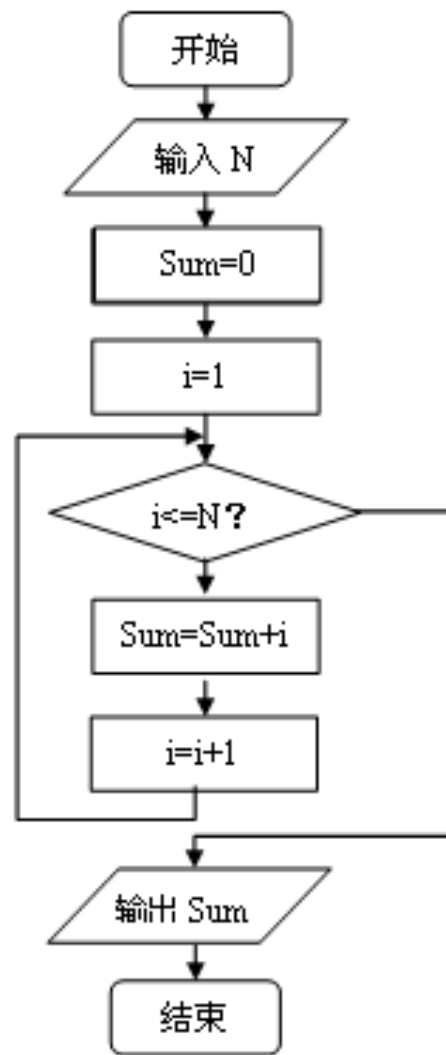


图 3-7 算法流程图

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

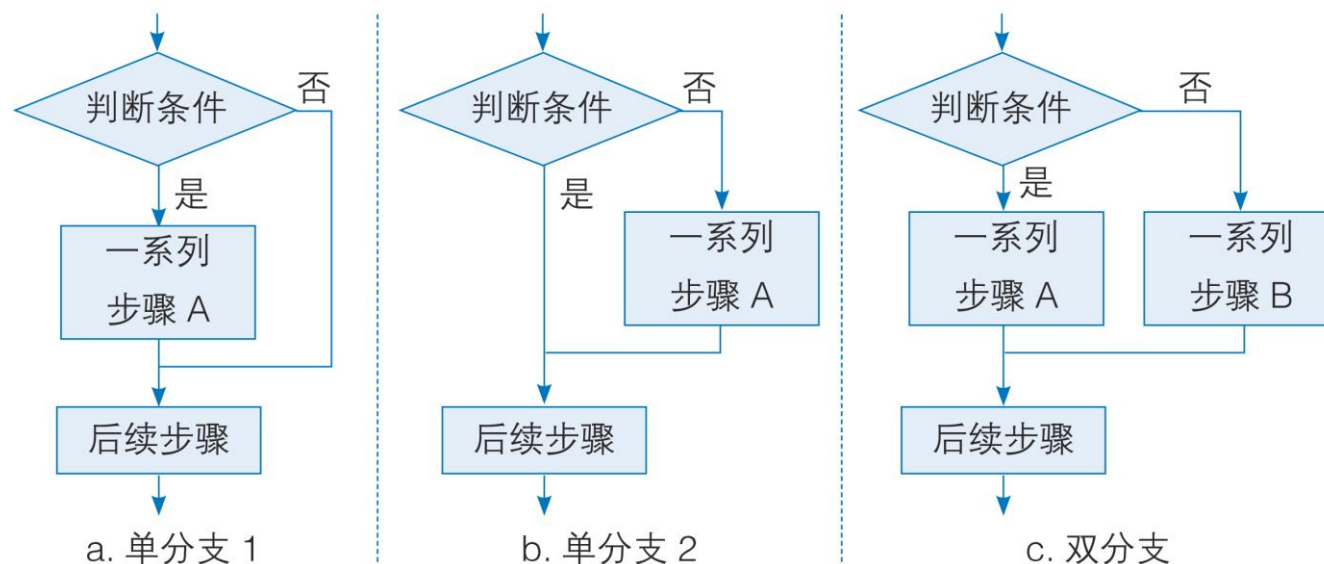
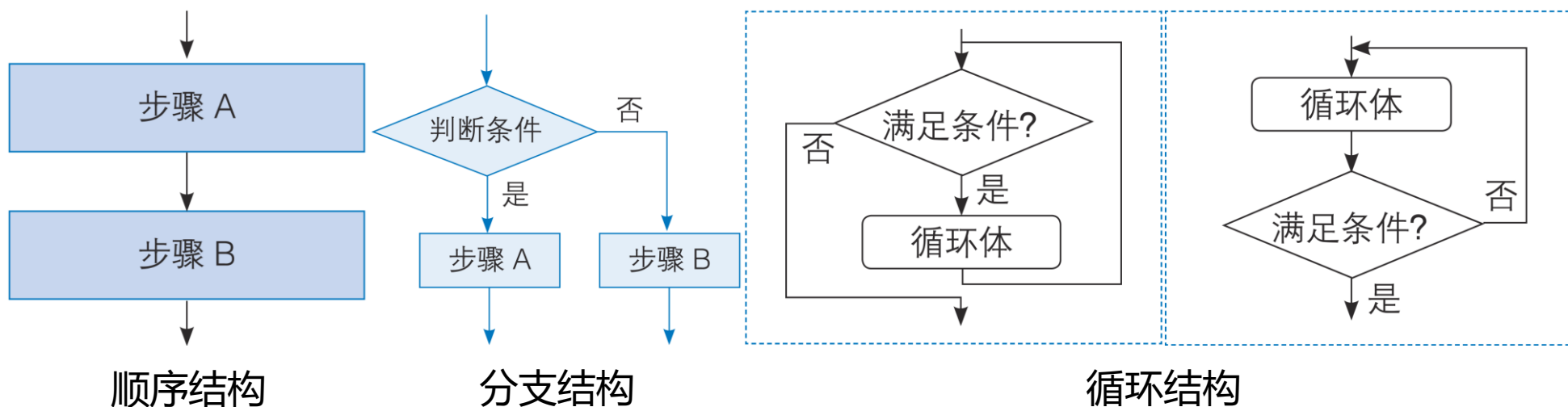
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法描述——流程图描述



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

N-S流程图

- 1973年，美国学者I.Nassi和B.Shneiderman提出了一种新的流程图形式，这种流程图完全去掉了流程线，算法的每一步都用一个矩形框来描述，在框内还可以包含其他框的流程图形式。即由一些基本的框组成一个大的框，这种流程图又称为N-S结构流程图，又称“盒图”
- NS图有三种基本结构，分别表示：顺序、选择和循环

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

N-S流程图结构

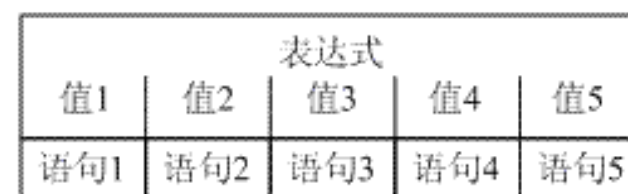
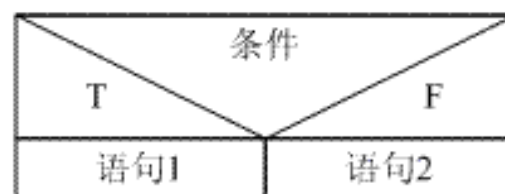
- NS图有三种基本结构，分别表示：顺序、选择和循环



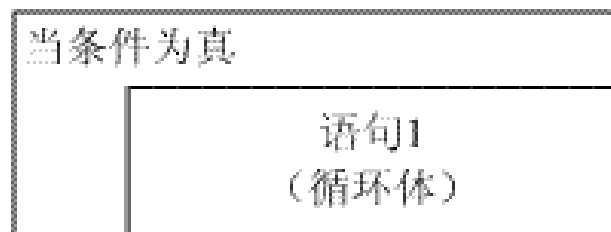
顺序结构N-S图



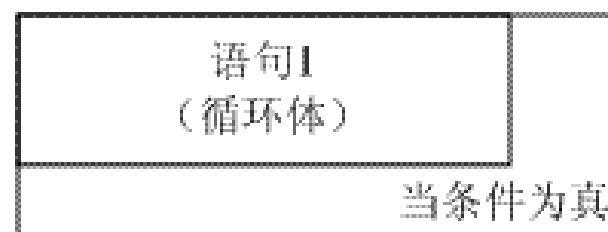
选择（分支）结构N-S图



switch结构N-S图



当型（while）循环结构N-S图



直到型（do-while）循环结构N-S图

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

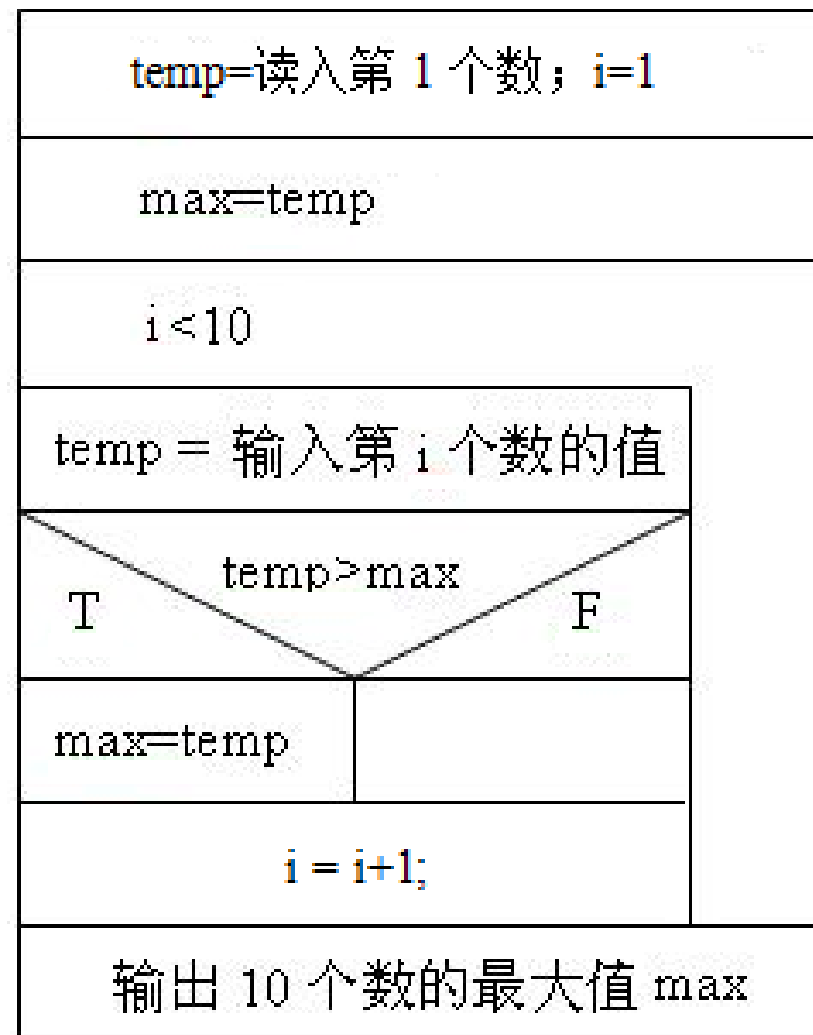
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例

- 输入10个数，求最大值，并输出
- NS图把整个程序写在一个大框图内，这个大框图由若干个小的基本框图构成。虽然去掉了流程线，但可读性较差。
- 为结构化编程设计，不便于表达面向对象的编程思想（受时代局限）





算法描述--伪代码描述

- 伪代码 (Pseudocode)
 - 是一种介于自然语言和计算机程序设计语言 (Pascal、C语言等) 之间的文字和符号来描述算法的工具，并以编程语言的书写形式来描述算法。
 - 伪代码没有严格的语法规则，只是遵循简单的书写约定
- 伪代码不是用户和分析师的工具，而是设计师和程序员的工具

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



伪代码一般的书写形式

- 每一条指令占一行，指令后不跟任何符号（某些编程语言中语句也会以分号结尾）；
- 书写上的“缩进”表示程序中的分支程序结构，用缩进取代传统Pascal中的begin和end语句和C中的“{”和“}”来表示程序的块结构可以大大提高代码的清晰性；同一模块的语句有相同的缩进量，次一级模块的语句相对与其父级模块的语句缩进；
- 变量名和保留字不区分大小写
- 赋值语句用符号 $\leftarrow$ 表示， $x \leftarrow \text{exp}$ 表示将exp的值赋给x
- 使用与程序设计语言类似的关键字表达分支和循环结构
 - if, if eles, switch
 - while, do while, for
- 数组元素的存取有数组名后跟 “[下标]”表示，结构变量或对象采用变量名后加点和域名方式访问

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



举例--输入三个数，输出其最大值

• 问题求解算法的伪代码描述

算法 3-2: 求三个数的最大值算法

Begin

输入 a, b, c

if $a > b$ 则 $\max \leftarrow a$

else $\max \leftarrow b$

if $c > \max$ 则 $\max \leftarrow c$

输出 $\max$

End

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



算法描述方法的比较

- 自然语言
 - 适合描述大的求解思路
 - 细节描述上比较麻烦
- 流程图
 - 精确描述，适合描述实现细节
 - 流程图绘制和修改麻烦
- 伪代码
 - 折中了自然语言和流程图的优点，在大的求解思路 and 具体细节上都可以使用

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

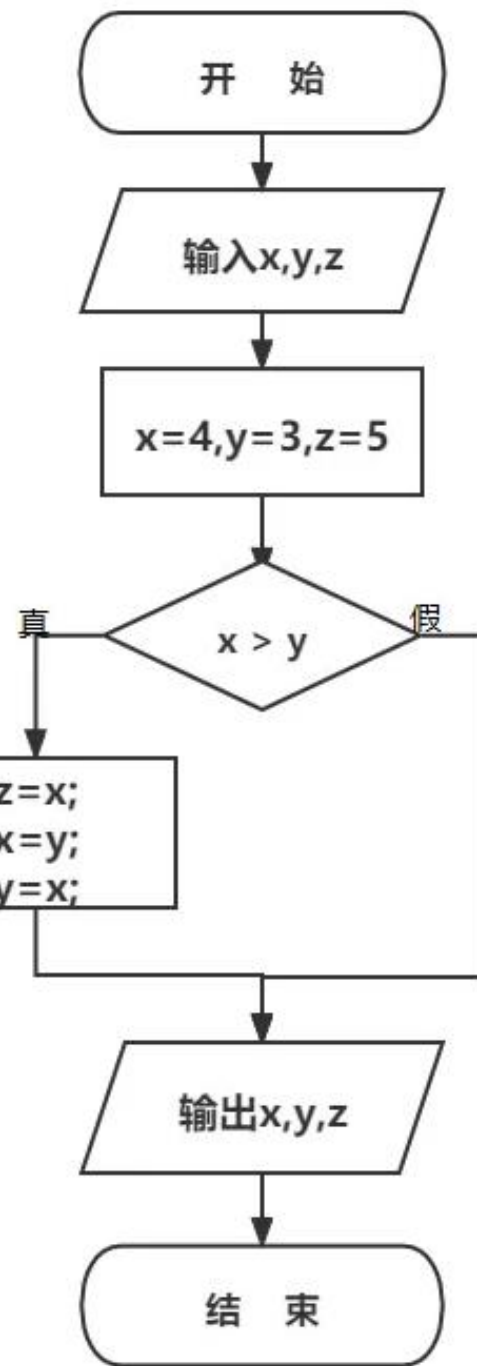
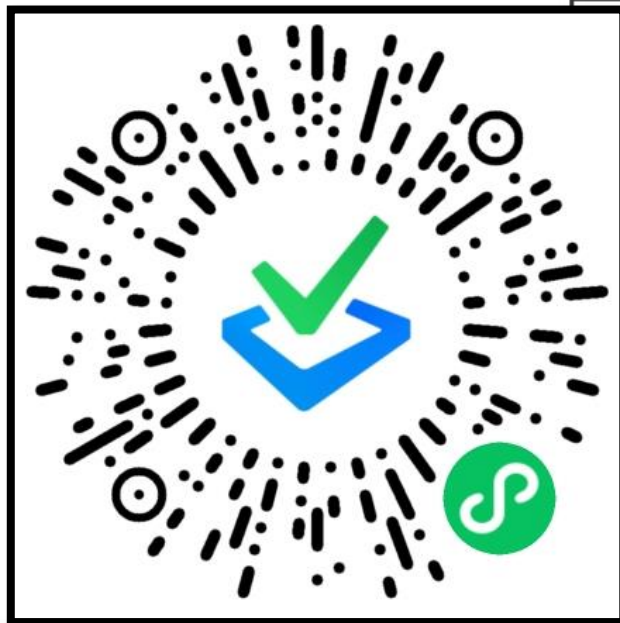
算法——练习题

- 1. 阅读如图所示的流程图，回答其输出结果
- 2. 下述伪代码的功能是

 $s \leftarrow 0$ $\text{for } i = 1; i < 10; i += 2$ $s = s + (i + 1)$

输出s

- A. 自然数1~9的累加和
- B. 自然数1~10的累加和
- C. 自然数1~9中的奇数之和
- D. 自然数1~10中的偶数之和





算法分析

- 传统意义下，就是对算法进行正确性、时间复杂性和空间复杂性的分析，从而来评价算法的优劣，或估计算法实现后的运行效果。
- 其他评价指标
 - 在网络化环境下，一个图形图像压缩算法，压缩比就是一种重要的算法性能指标，保证回复质量一定的情况下，越小，传输越快。
- 算法分析通常是对同一类问题的不同求解算法之间的性能比较

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法正确性分析

• 计算三角形面积算法

```
// 输入三角形的三条边长 a, b, c
// 使用海伦公式来计算面积
// 先计算半周长 s
s = (a + b + c) / 2
// 计算面积
area = sqrt(s * (s - a) * (s - b) * (s - c))

// 输出结果
输出 area
```

海伦公式: [1]

假设在平面内, 有一个三角形, 边长分别为a、b、c, 三角形的面积S可由以下公式求得:

$$S = \sqrt{p(p-a)(p-b)(p-c)}$$

而公式里的p为半周长 (周长的一半):

$$p = \frac{a+b+c}{2}$$

• 算法的正确性层次

- 对于一组数据能够得出正确的结果。
- 对于精心挑选的、苛刻的测试用例算法可以得到正确的结果 (如不满足三角形构成条件的输入)。
- 对于一切合法的输入数据, 算法得到的结果都是正确的。 (最难达到, 软件测试的重要性->测试用例优先级排序)

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

时间复杂性

- 算法的时间复杂性 (Time Complexity) 是指根据该算法编写的程序在运行过程中, 从开始到结束所需要的时间。
- 从算法中选取一种对于所研究的问题来说是基本运算的操作作为**元操作**, 以该元操作重复执行的次数作为算法的时间度量。例如, 排序类算法, 用数据的比较和移动作为元操作。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

时间复杂性分析

- 定义：设 $f(n)$ 和 $g(n)$ 是从自然数集到非负实数集的两个函数，如果存在一个自然数 n_0 和一个常数 $c>0$ ，使得 $n\geq n_0$ ， $f(n)\leq cg(n)$ ，则记为 $f(n)=O(g(n))$ ，称 $g(n)$ 是 $f(n)$ 的**上界**；如果是 $n\geq n_0$ ， $f(n)<cg(n)$ ，称 $g(n)$ 是 $f(n)$ 的**严格上界**，记为 $f(n)=o(g(n))$ 。
- 一个算法中元操作重复执行的次数与求解问题的规模（问题长度） n 呈某个函数关系 $T(n)$
 - 设 $T(n)$ 是算法的执行时间， n 是问题规模， $f(n)$ 为 n 的函数，若 $T(n)=O(f(n))$ ，则称 $f(n)$ 为算法的时间复杂性上界。时间复杂性通常用 $O(f(n))$ 来表示
- 算法的执行时间与问题长度无关，则该算法的时间复杂度记为 $O(1)$ ，表示常数时间复杂度

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

时间复杂性分析

• 常见的时间复杂度

- 一般情况下，只要算法中都是顺序结构或分支结构，不存在循环语句、递归语句等，即使有成千上万行的代码，其时间复杂度也是 $O(1)$ 。

```
function accessElement(arr, index):  
    return arr[index]
```

- 当算法的执行时间以输入规模的对数增长时，称为对数时间复杂度 $O(\log n)$

```
i = n * n;  
while(i != 1)  
    i = i/2;
```

- 如果算法的执行时间与输入规模成正比，即算法的每一步处理输入数据中的一个元素，则称该算法具有线性时间复杂度 $O(n)$

```
for (i = 1; i++; i < n):  
    a = a + i;
```

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

时间复杂性分析

- 常见的时间复杂度

- 每处理一个元素都需要进行对数时间操作的情况下会出现线性对数时间复杂度 $O(n \log n)$ 。

```
for i = 1 to n:  
    j = n  
    while j > i:  
        print("i =", i, "j =", j)  
        j = j / 2
```

- 当算法的执行时间与输入规模的幂次方成正比时，称其具有多项式时间复杂度，例如 $O(n^2)$, $O(n^3)$

```
for (i = 1; i++; i < n):  
    for (j = 1; j++; j < n):  
        a = a + i + j;
```

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

时间复杂性分析

• 常见的时间复杂度

- 当算法的执行时间呈指数增长时，意味着随着输入规模的增加，执行时间将以指数速度增加，此时为指数时间复杂度 $O(2^n)$ 或 $O(a^n)$, $a > 1$ 。
- 随着输入规模的增长，执行时间按阶乘增长的算法时间复杂度为 $O(n!)$ 。
- 时间复杂度的表达式括号内不一定是 n ，也可以是其他字母，例如 $O(m*n)$

```
a = 0;
for (i = 1; i++; i < n):
    for (j = 1; j++; j < m):
        a = a + i + j;
```

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例

冒泡分类算法伪代码描述如下：

```
void BubbleSort(int A[],int n)
{
    int i,j,temp;
    (1) for (i=0;i<n-1;i++)
    {
        (2) for (j=n-1;j>=i+1;j--)
        {
            (3) if (a[j]<a[j-1])
            {
                (4) temp=A[j-1];
                (5) A[j-1]=A[j];
                (6) A[j]=temp;
            }
        }
    }
}
```

- ① 语句 (3) ~ (6) 的运行时间为常数时间，均为 $O(1)$ ，则总时间为 $O(1)$
- ② 语句 (2) 为循环控制语句，执行次数为 $n-i-1$ ，每次循环执行时间为 (3) ~ (6) 的时间，因此，总的时间为 $(n-i-1) \cdot O(1)$ 。
- ③ 语句 (1) 为外重循环，执行次数为 $n-1$ 次，第 i 次时间消耗为 $(n-i-1) \cdot O(1)$
- 因此有：

$$T(n) = \sum_{i=0}^{n-2} (n-i-1) \leq O(n(n-1)/2) = O(n^2)$$

算法时间复杂度与求解问题长度的关系

- 假设计算机执行一次基本操作需要的时间为1ms
- 常见算法时间复杂度与求解问题长度的关系

表 3-2 算法时间复杂度与求解问题长度的关系

算法	时间复杂度	可解问题最大长度 (n)		
		1 秒	1 分钟	1 小时
A_1	$O(n)$	1000	60000	3.6×10^4
A_2	$O(n \log_2 n)$	140	4893	2.0×10^5
A_3	$O(n^2)$	31	240	1987
A_4	$O(n^3)$	10	33	153
A_5	$O(2^n)$	9	15	21

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

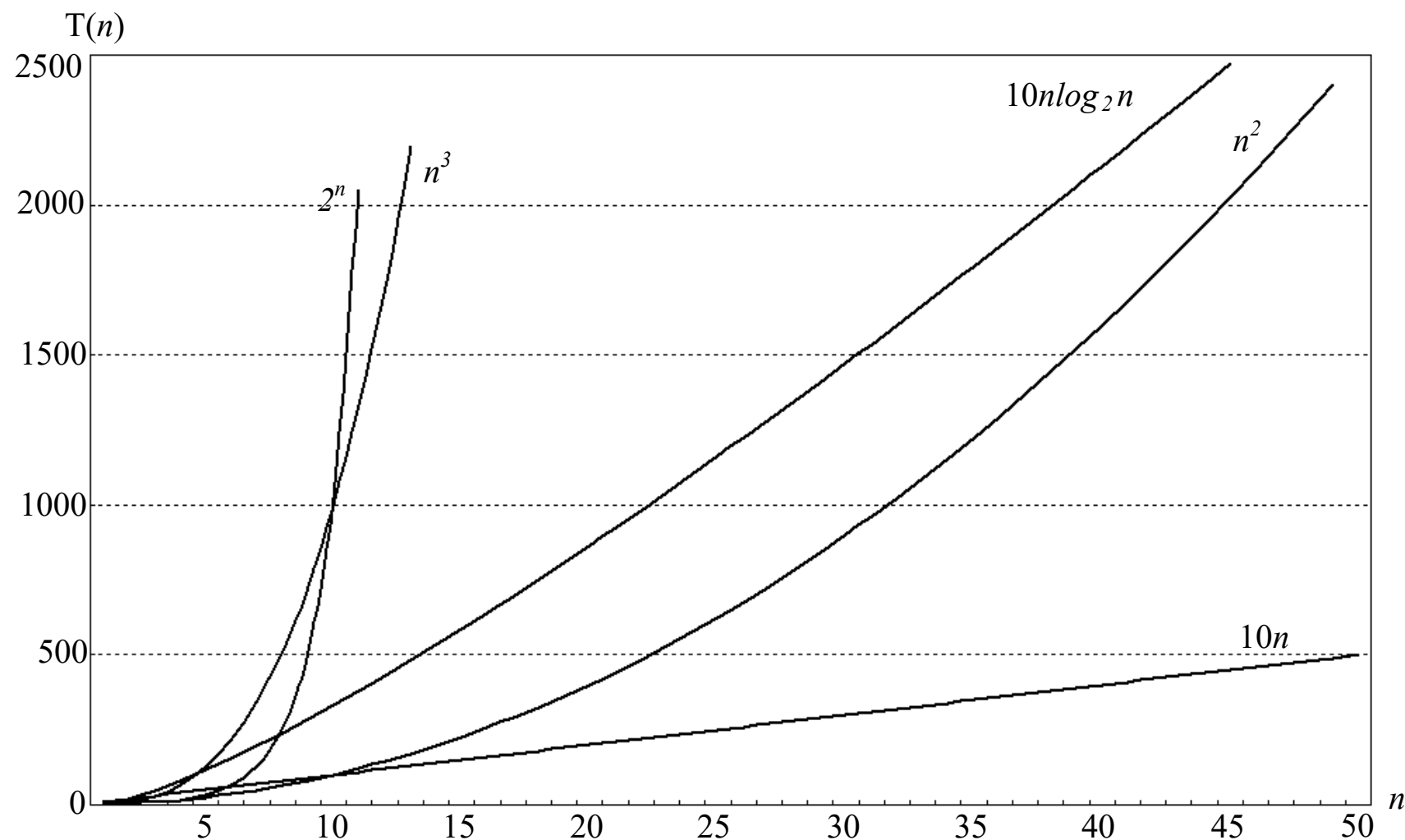
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

常见时间复杂度函数



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



计算机速度对求解问题长度的影响

- 计算机速度对求解问题长度的影响

表 3-3 计算机速度对求解问题长度的影响

算法	时间复杂度	可解问题最大长度(n)	
		计算机速度提高前	计算机速度提高 10 倍
A_1	$O(n)$	s_1	$10s_1$
A_2	$O(n\log_2n)$	s_2	$10s_2$
A_3	$O(n^2)$	s_3	$3.16s_3$
A_4	$O(n^3)$	s_4	$2.15s_4$
A_5	$O(2^n)$	s_5	$s_5+3.3$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例——TSP (Traveling Salesman Problem) 问题

- 19世纪初，爱尔兰数学家、物理学家、天文学家哈密顿提出：有 n 个城市，城市之间的距离都是已知的，现有一旅行商从某个城市出发到每个城市旅行，途中只能经过每个城市一次，最终回到出发点，求最短的一条路径。
- 问题分析
 - 1. 从某城市出发，最后回到该城市
 - 2. 每个城市只能访问一次
 - 3. 所走路程最近
- 解决上述问题，最简单的方法就是列出所有的路径，计算每条路径的长度，然后取一个最小值。

TSP问题模型

- TSP问题要从图G的所有路线中求取最小成本的一条路线，而从初始点出发的路线数量共有 $(n-1)!$ 条，即等于除初始结点外的 $n-1$ 个结点的排列数，因此TSP问题是一个排列问题
- 通过枚举 $(n-1)!$ 条路线，从中找出一条具有最小成本的路线的算法，其计算时间显然为 $O(n!)$ 。
- 例如：
 - 当 $n=4$ 时，有 6 条路径，
 - $n=6$ 时，有 120 条路径。
 - 当 n 增大时，其路径的排列组合数会急剧增长，
 - 当 $n=20$ 时，其路径数量 $(n-1)! = 19! \approx 1.21645 \times 10^{17}$ 。
 - 如果是现在的计算机，每秒可以计算 10^8 条路径，也需要 38 年的时间！

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

小蜜蜂采蜜的启示

- 2010年10月25日，英国一项最新研究说，在花丛中飞来飞去的小蜜蜂显示出了轻易破解“旅行商问题”的能力，如能理解蜜蜂的解决方式，将有助于人们改善交通规划和物流等领域的工作。
- 英国伦敦大学皇家霍洛韦学院等机构的研究人员对蜜蜂在花丛间采蜜的行为进行了研究：
 - 蜜蜂每天都要在蜂巢和花朵间飞来飞去，为了采蜜而在不同花朵间飞行是一件很耗精力的事情，因此实际上蜜蜂每天都在解决“旅行商问题”。
 - 他们利用人工控制的假花进行了实验，结果显示，不管怎样改变花的位置，蜜蜂在稍加探索后，很快就可以找到在不同花朵间飞行的最短路径。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

TSP问题的应用

- 如何规划最合理高效的道路交通，以降低成本，减少拥堵。
- 如何更好地规划物流，以减少运营成本。即：一个物流配送公司，将 n 个客户的订货沿最短路线全部送到，如何确定最短路线。
- 在互联网环境中如何更好地设置交换（Switch）和路由（Router）节点，以更高效地让信息流动。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

P问题与NP问题

- P问题

- 问题求解算法的时间复杂度是该问题实例规模 n 的多项式函数，则这种可以在多项式时间内解决的问题属于P（Polynomial，多项式）类问题，通俗地称所有复杂度为多项式时间的问题为易解问题

- 非确定性（Non-Deterministic）问题

- 有些计算问题是确定性的，只要按照公式推导，按部就班的一步步来，就可以得到结果（阶乘等）。
- 无法按部就班的计算得到。比如，找某个范围内最大质数的问题。有没有一个公式，可以一步步推算出下一个质数应该是多少呢？这样的公式是没有的。

- NP问题，非确定性多项式问题

- 非确定问题通常存在这样的算法，它不能直接告诉答案是什么，但可以计算某个可能的结果是否正确。这个可以验证“猜算”的答案正确与否的算法，假如可以在多项式（polynomial）时间内计算出来，就叫做非确定性多项式问题，即NP问题。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



NP问题的分类

- NP完全 (NP-complete) 问题
 - 如果这个问题的所有可能答案，都是可以在多项式时间内进行正确与否的验算的话，就叫非确定多项式完全问题，即NP-Complete问题。
- NP难度 (NP-hard) 问题
 - 非确定性多项式完全问题可以用穷举法得到答案，一个个检验下去，最终便能得到结果。
 - 但是这样算法的复杂程度，是指数关系，因此计算的时间随问题的复杂程度成指数的增长，很快便变得不可计算了，这就是NP-hard问题。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例--NP-hard问题

- 美国的数据加密标准DES (Data Encryption Standard)。DES采用长度为64位密钥 (实际密钥56位, 8位用于奇偶校验), 对64位二进制数据加密, 得到64位密文数据。

对 56 位密钥, 其穷举的数量是 2^{56} 。

假如我们在一台运算速度为一亿次每秒的计算机上用枚举法求解 DES 密码, 每秒钟判断一亿个密码是否正确, 则需要花费的时间是:

$$2^{56}/10^8 \text{ 秒} = 2^{56}/10^8/60/60/24/365 \text{ 年} \approx 22.85 \text{ 年}$$

如果是 128 位密钥呢, 则可能的排列组合数目是:

$$2^{128} = (2^{10})^{12.8} > (10^3)^{12.8} = 10^{38.4}$$

如果每秒钟可验证百亿种猜测, 可以计算需要多少时间, 结果是约需要花费**一万亿亿年**!

用暴力法求解 TSP 问题, 计算所有可能的路径花费的时间呈指数增长, n 个城市可能的路径有 $(n-1)!$, 如果 $n=26$, 则 $25! = 15511210043330985984000000 \approx 1.55 \times 10^{25}$ 。

若每秒可计算 10^6 条路径, 则一年可计算 3.15×10^{13} 路径, 计算全部路径需要 **5000 亿年**!

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



3.3 算法设计及算法分类

- 穷举法
- 递推法
- 递归法
- 回溯法
- 迭代法
- 分治法
- 贪心法
- 动态规划法

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

穷举法

- 穷举法，又称暴力算法，顾名思义就是对于要求解的问题，列举出问题解空间的所有可能的情况，并逐个测试，找出符合问题条件的解，从而得到问题的解。
- 通常情况是，穷举法为一个NP-hard问题，即非确定性多项式难解问题
- 例：**百钱买百鸡问题**
 - 问题：公元5世纪末，我国古代数学家张丘建在他的《算经》中提出了著名的“百钱买百鸡问题”：鸡翁一，值钱五，鸡母一，值钱三，鸡雏三，值钱一，百钱买百鸡，问翁、母、雏各几何？

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

问题分析与建模

- 问题：公元5世纪末，我国古代数学家张丘建在他的《算经》中提出了著名的“百钱买百鸡问题”：鸡翁一，值钱五，鸡母一，值钱三，鸡雏三，值钱一，百钱买百鸡，问翁、母、雏各几何？

分析：设鸡翁、鸡母、鸡雏的个数分别为 x, y, z ,

则百钱买百鸡问题可以用如下方程表示：

$$\begin{cases} 5x + 3y + z/3 = 100 \\ x + y + z = 100 \\ 1 \leq x < 20, 1 \leq y < 33, 3 \leq z < 100, z \bmod 3 = 0 \end{cases}$$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

• 算法描述

算法 3-3 百钱买百鸡问题求解算法

//百钱买百鸡问题穷举算法

//设鸡翁、鸡母、鸡雏的个数分别为 x, y, z

for ($x=1; x<20; x++$)

 for ($y=1; y<33; y++$)

 for ($z=3; z<100; z++$)

 if ($(x+y+z==100)$ and $(5x+3y+z/3==100)$ and $(z \bmod 3==0)$)

 writeln(x, y, z)

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法分析

```
for (x=1;x<20;x++)  
    for (y=1;y<33;y++)  
        for (z=3;z<100;z++)  
            if (x+y+z==100) and (5x+3y+z/3==100) and (z mod 3==0)  
                writeln(x,y,z)
```

- 上述算法是一个三重循环，最内层的条件判断最大需要执行 $19*32*97$ 次，即 58976 次。
- 有什么改进方案？

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



算法二

- 在条件判断中，利用了整数的求模运算，如果将鸡雏的个数设为 $3z$ ，可以避免该项判断，且可减少内重循环次数。
- 将鸡雏的个数设为 $3z$ ，可以避免该项判断，且可减少内重循环次数。

```
for (z=1; z<34; z++)  
    if (x+y+3z==100) and (5x+3y+z==100)  
        writeln(x, y, 3z)
```

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



递推法

- 递推算法是一种根据递推关系进行问题求解的方法。
 - 基本思想是把一个复杂的庞大的计算过程转化为简单过程的多次重复，该类算法利用了计算机速度快和自动化的特点。
 - 通过已知条件，利用特定的递推关系可以得出中间推论，直至得到问题的最终结果。
- 递推关系，递推关系可以抽象为一个简单的数学模型
 - 给定一个数的序列 $a_0, a_1, \dots, a_n, \dots$ 若存在整数 n_0 ，使当 $n > n_0$ 时，可以用等号（或大于号、小于号）将 a_n 与其前面的某些项 a_i ($0 < i < n$)联系起来，这样的式子称为递推公式，又称递推关系。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



顺推法与逆推法

- 顺推法，从已知条件出发，按照递推关系一步步地递推，直至求出问题的最终结果。
- 逆推法，在不知道初始条件的情况下，从问题的结果出发，经递推关系逐步推算出问题的解，这个问题的解也是问题的初始条件。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：斐波纳契数列 (Fibonacci Sequence)

- 斐波纳契数列 (Fibonacci Sequence) , 又称黄金分割数列: 1、1、2、3、5、8、13、21、.....
- 斐波那契数列是意大利数学家列昂纳多·斐波那契 (Leonardo Fibonacci) (1170年~1240年) 首先研究的, 斐波那契数列是一个奇妙的数列, 随着数列项数的增加, 前一项与后一项之比越来越逼近黄金分割的数值0.6180339887...

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



问题分析

- 递推公式

- $F_0=1, F_1=1, F_n=F(n-1)+F(n-2) \ (n \geq 2, n \in \mathbb{N}^*)$

- 问题分析

- 从斐波那契数列的定义可知，斐波那契数列的第1项为1，第2项也为2，递推关系是当前项等于前2项之和。因此，通过顺推可以得到第n项的值。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



求斐波拉契数列的顺推算法

算法3-5 求斐波拉契数列的顺推算法

// 求斐波那契数列第10项的值并输出

f[0]=1

f[1]=2

n=2

while (n<10)

{

 f[n]=f[n-1]+f[n-2]

 n=n+1

}

write(f[9])

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例:卡车穿越沙漠问题

- 一辆重型卡车要穿过800公里的沙漠，已知卡车每公里耗油1升，卡车总载油能力为400升，显然卡车装满一次油是无法穿越沙漠的，因此货运公司需要在沿途提前建立几个储油点，卡车司机从起点出发，往返多趟将汽油逐渐携带运送至各储油点，才能使卡车顺利穿越沙漠。
- 要让卡车以消耗最少的汽油穿越沙漠，试问沿途最少需要建立几个储油点？每个储油点需要存储多少汽油？
- 注意：需要考虑卡车司机在储油点间往返所消耗的汽油。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

问题分析

- 设沿途设置 n 个储油点，用倒推法来解决这个问题，从终点向起点倒推，可以逐一求出每个储油点的位置及储存量。

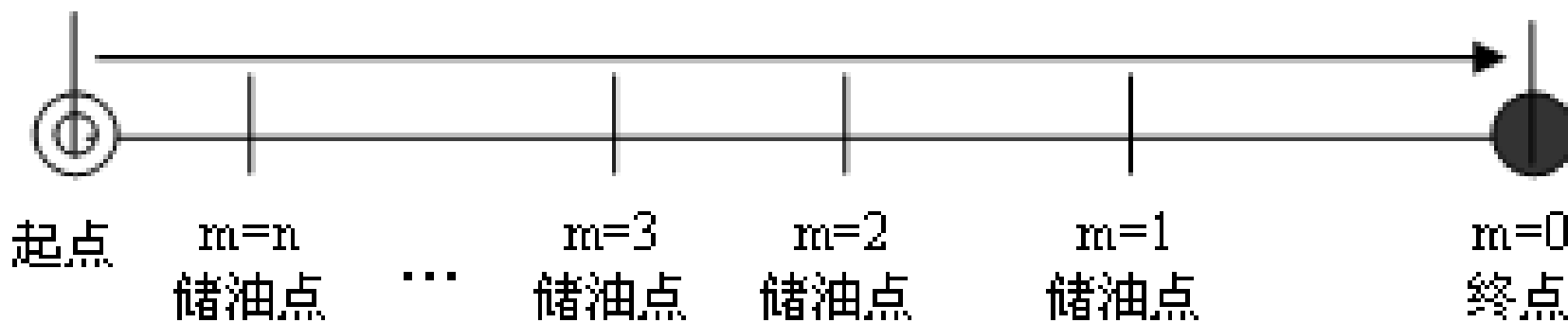


图 3-11 储油点建设示意图

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

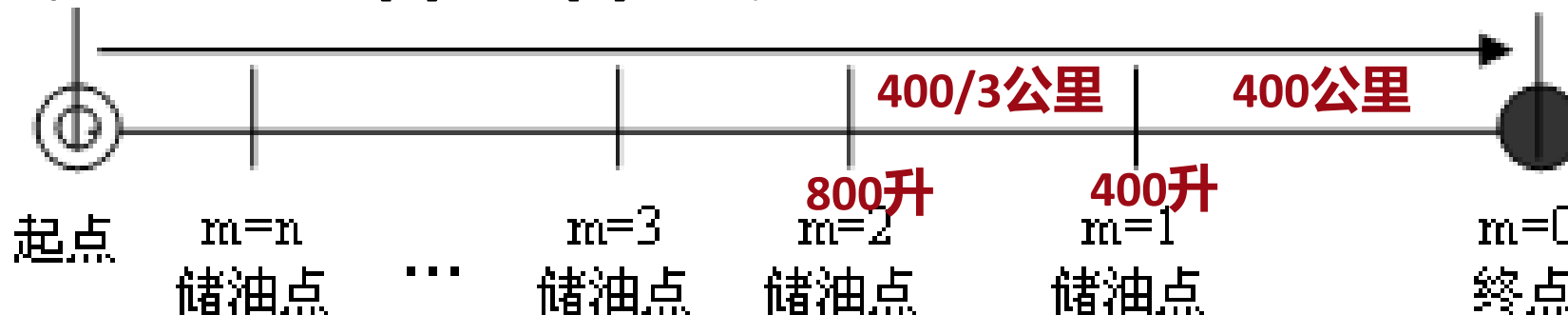
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

分析

- (1) 为了消耗最少的汽油，最后一个储油点应该离终点400公里，且此处储油400升，即储油点 $m=1$ 处离终点 $\text{dist}[1]=400$ 公里，储油量 $\text{oil}[1]=400$ 升。
- (2) 为了在 $m=1$ 处储存400升汽油，卡车最少从储油点 $m=2$ 处开两趟载满油的车到储油点 $m=1$ 处，则储油点 $m=2$ 处储油量 $\text{oil}[2]=400*2=800$ 升，其中，400升用于储存在 $m=1$ 处，400升用于从 $m=2$ 到 $m=1$ 处的油料运输。要将400升油从储油点 $m=2$ 处运送到储油点 $m=1$ 处，则从 $m=2$ 到 $m=1$ 处至少需开两趟，需要开三次路程，因此有 $\text{dist}[2]=\text{dist}[1]+400/3$ 公里。



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

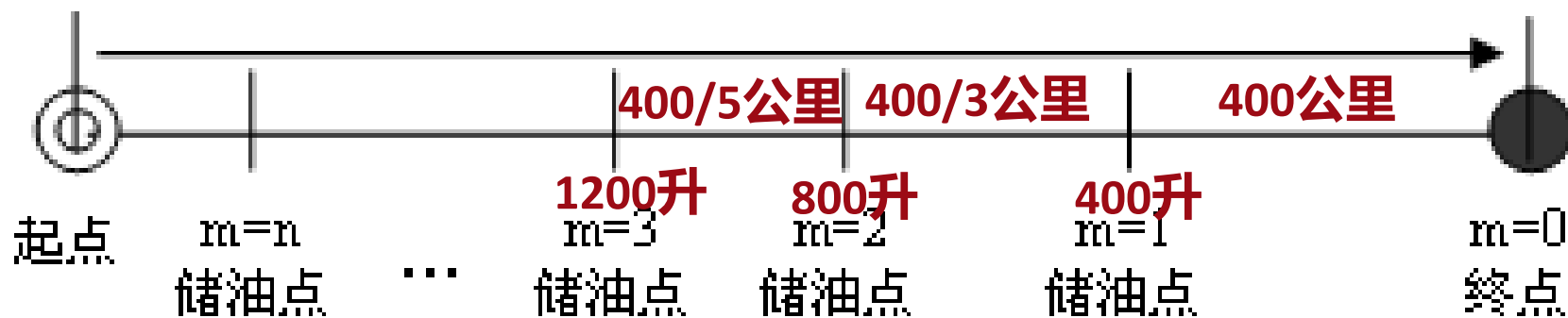
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

分析 (续1)

- (3) 为了在储油点 $m=2$ 处储存800升汽油，卡车最少从储油点 $m=3$ 处开三趟载满油的车到储油点 $m=2$ 处，则储油点 $m=3$ 处储油 $oil[3]=400*3=1200$ 升，其中，800升用于储存在 $m=2$ 处，400升用于从 $m=3$ 到 $m=2$ 处的油料运输。储油点 $m=3$ 处到储油点 $m=2$ 处至少开三趟需要开五次路程，因此， $dist[3]=dist[2]+400/5$ 公里。



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

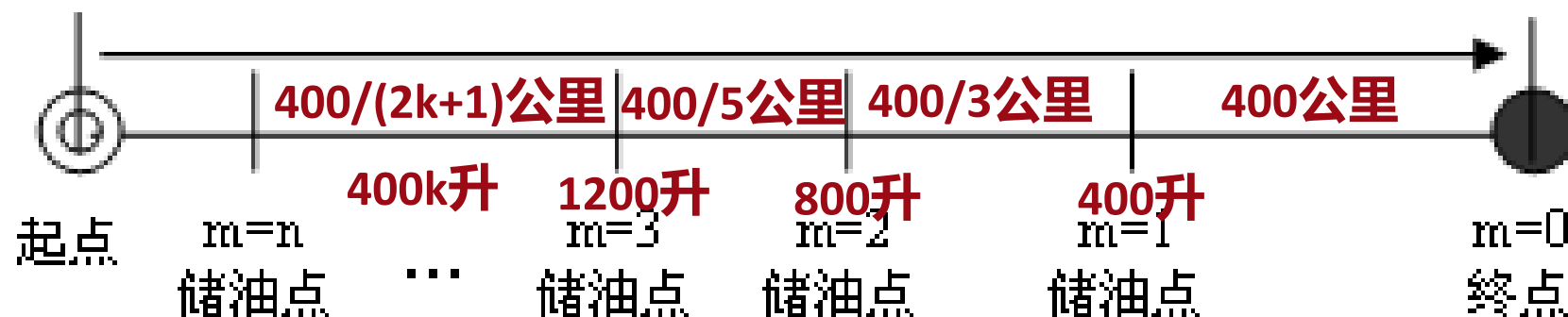
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

分析 (续2)

- (4) 依次类推, 为了在 $m=k$ 处储存 $oil[k]=k*400$ 升汽油, 则储油点 $m=k+1$ 处储油 $oil[k+1]=(k+1)*400$, 其中400升用于往返运送的消耗。要在 $m=k$ 处储存 $k*400$ 升油, 从 $m=k+1$ 处至 $m=k$ 处至少需要 $k+1$ 趟, 最后一次无需返回 $m=k+1$ 处, 因此两地往返最小需 $2(k+1)-1=2k+1$ 次路程, 则储油点离终点 $dist[k+1]=dist[k]+400/(2*k+1)$ 。



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

分析 (续3)

- (5) 最后一个储油点为 $m=n$ ，它至起点的距离为 $800-\text{dist}[n]$ ，储油为 $\text{oil}[n]=n*400$ 升，为了在 $m=n$ 处储 $n*400$ 升汽油，卡车最少从起点开 $n+1$ 趟满载车至 $m=n$ 处，需要开 $2(n+1)-1=2n+1$ 次路程，总耗油量为 $(800-\text{dist}[n])*(2n+1)$ ，即起点储油量为 $\text{oil}[n]+(800-\text{dist}[n])*(2n+1)=n*400+(800-\text{dist}[n])*(2n+1)$ 。



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

储油点部署问题逆推算法

算法3-6 储油点部署问题逆推算法

```
// 问题求解逆推算法
k=1;
dist[1]=400; //从m=1处开始倒推
oil[1]=400;
do {
    k=k+1;
    ①oil[k]赋值为当前储油点所需要存储的油量;
    ②dist[k]赋值为当前储油点到终点的距离;
} while (③当前储油点到终点的距离小于总路程)
//从起始点开始, 依次输出储油点序号, 距离起始点的距离, 及储油数量
n=k;
for k=1 to n
{
    print(k, 800-dist[n-k+1], oil[n-k+1])
}
```

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

递推法随堂练习

• 请正确填写算法中标号①②③的三部分对应的伪代码

①

- A. $oil[k] = (k-1) * 400$
- B. $oil[k] = (k+1) * 400$
- C. $oil[k] = k * 400$
- D. $oil[k] = 400$

②

- A. $dist[k] = dist[k-1] + 400 / (2 * k - 1);$
- B. $dist[k] = dist[k] + 400 / (2 * k - 1);$
- C. $dist[k] = dist[k+1] + 400 / (2 * k + 1);$
- D. $dist[k] = dist[k-1] + 400 / (2 * k + 1);$

③

- A. $dist[k-1] < 800$
- B. $dist[k] < 800$
- C. $dist[k-1] \leq 800$
- D. $dist[k] \leq 800$

算法3-6 储油点部署问题逆推算法

// 问题求解逆推算法

k=1;

dist[1]=400; //从m=1处开始倒推

oil[1]=400;

do {

 k=k+1;

 ①oil[k]赋值为当前储油点所需要存储的油量;

 ②dist[k]赋值为当前储油点到终点的距离;

} while (③当前储油点到终点的距离小于总路程)

// 从起始点开始, 依次输出储油点序号, 距离起始点的距离, 及储油数量

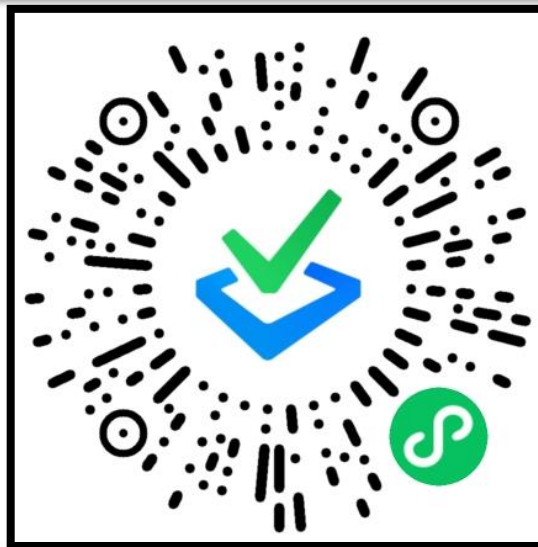
n=k;

for k=1 to n

{

 print(k, 800-dist[n-k+1], oil[n-k+1])

}



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



递归法

- 在问题求解思想上，递推是从已知条件出发，一步步的递推出未知项，直到问题的解。
- 从思想上讲，递归也是递推的一种（即：**逆推法**），只不过它是对待解问题的递推，直到把一个复杂的问题递推为简单的易解问题。
- 然后再一步步的返回去，从而得到原问题的解。
- 严格来说，递归并不是一种算法，而是一种编程方法或技术（编程范式）

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

求N!递归法描述

• 求n!的递归定义

$$n! = \begin{cases} 1 & \text{当 } n=0 \text{ 时;} \\ n(n-1)! & \text{当 } n>0 \text{ 时;} \end{cases}$$

• 递推求解

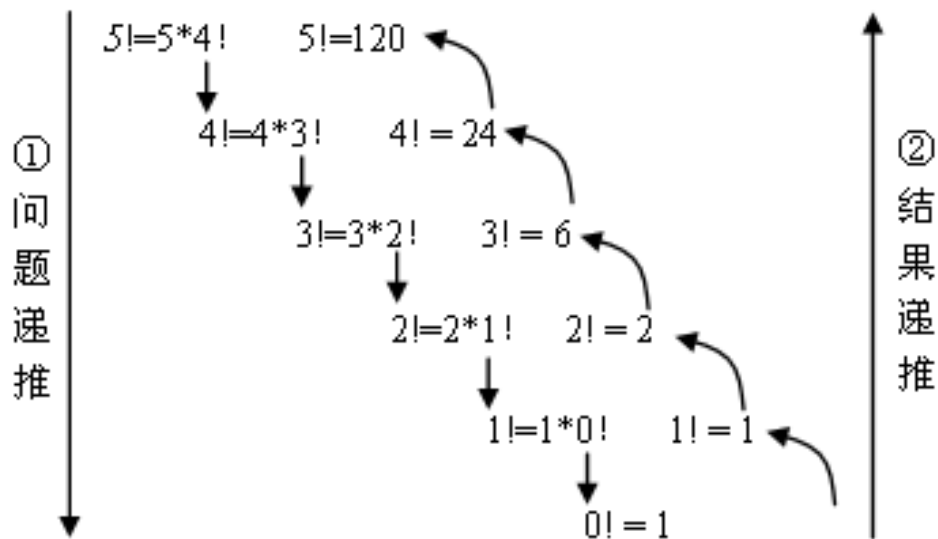


图 3-12 递归问题的求解过程

例3-8：求斐波那契数列值的递归算法

算法3-8 求斐波那契数列递归算法

//函数fib返回第 n ($n>1$)个斐波那契数列的值

```
int fib(int n)
```

```
{
```

```
    if (n == 0)
```

```
        return(1)
```

```
    else
```

```
        if (n == 1)
```

```
            return(2)
```

```
        else return (fib(n-1)+fib(n-2))
```

```
}
```

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-9: Hanoi塔问题递归算法

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

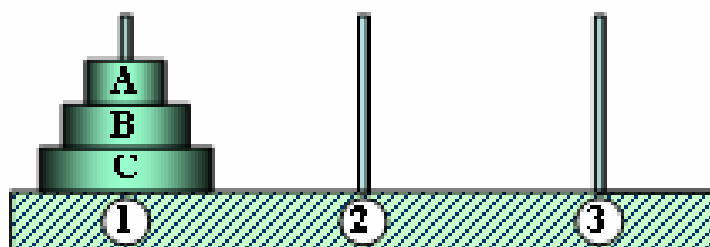
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

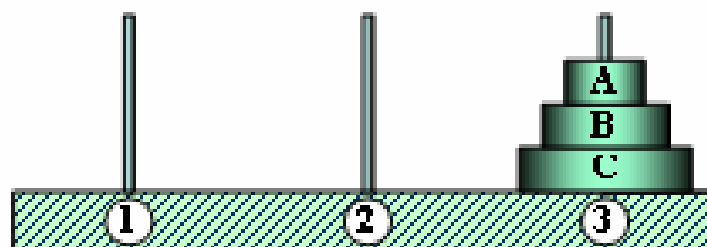
3.6 网络搜索问题

汉诺塔游戏起源于一个古老的传说

例3-9: Hanoi塔问题递归算法



(a) 初始状态



(b) 目标状态

// N为盘子数目, 三根柱子分别为from, to和temp, 分别表示起始柱子, 目标柱子和临时柱子

```
void hanoitower(n,from,to,temp)
```

```
{
```

```
    if (n>0) {
```

```
        // 把from柱子上的n-1格盘子搬移到temp柱子上, 用to柱做临时柱子
```

```
        hanoitower(n-1,from,temp,to);
```

```
        movedisk(from,to);
```

```
        // 把temp柱子上的n-1格盘子搬移到to柱子上, 用from柱做临时柱子
```

```
        hanoitower(n-1,temp,to,from);
```

```
    }
```

```
}
```

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

递归法

- 递归最恰当的比喻，就是查词典。我们使用的词典，本身就是递归，为了解释一个词，需要使用更多的词。当你查一个词，发现这个词的解释中某个词仍然不懂，于是你开始查这第二个词，可惜，第二个词里仍然有不懂的词，于是查第三个词，这样查下去，直到有一个词的解释是你完全能看懂的，那么递归走到了尽头，然后你开始后退，逐个明白之前查过的每一个词，最终，你明白了最开始那个词的意思。

【8分钟搞懂“编程的鬼打墙”——递归】

https://www.bilibili.com/video/BV14q5fzAEpD/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

回溯法

- 在现实世界中，许多问题不是通过确定的计算公式得到的，只能通过试探、回溯的方法来求解。
- 回溯法试图在问题的所有解空间树中，从根节点开始，按照深度优先的方法对树进行搜索。当搜索到某一个节点时，判断该节点是否包含在问题的解集中，如果在，继续深度搜索。如果不在，则回溯到该节点的父节点开始下一个试探。如果确定了某个节点不在解集中，逐层向父节点回溯。
- 回溯法
 - “试探 - 失败返回 - 再试探”的问题求解方法，例如：某迷宫问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

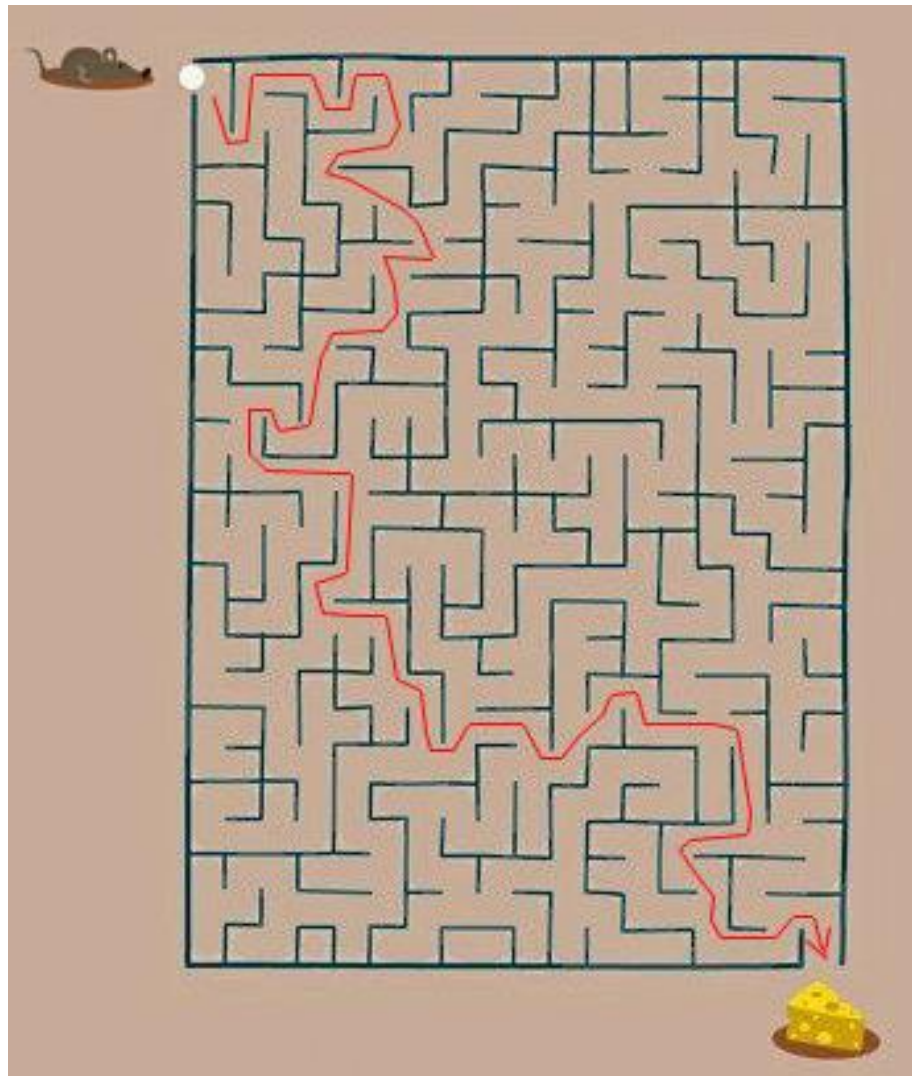
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

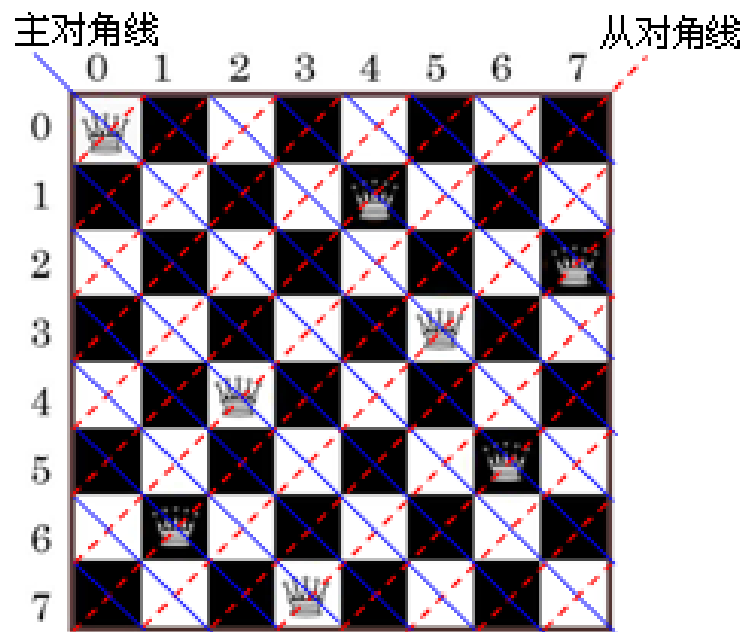
老鼠走迷宫

- 一只实验室的小老鼠被用来做迷宫智力实验。科学家在迷宫的一角放上一块奶酪，小老鼠要在最快时间内找到奶酪。

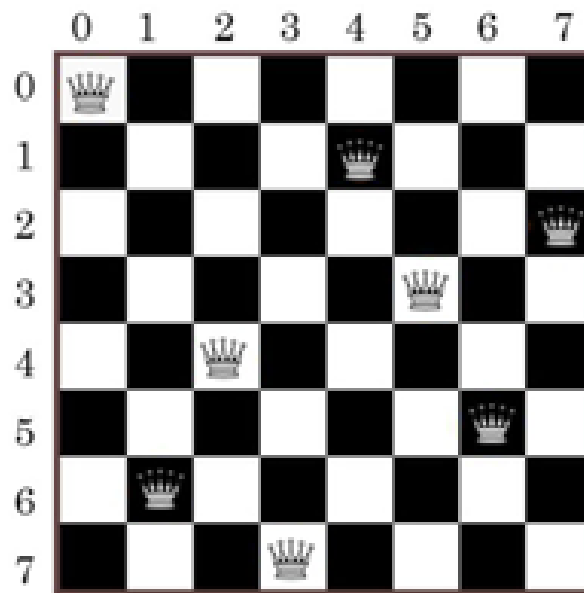


例3-10：八皇后问题

1850年，数学家高斯（Gauss）提出了这一问题，即：在 8×8 格的国际象棋上摆放八个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上，问有多少种摆法。



(a) 棋盘对角线



(b) 摆放示例

【回溯法求解八皇后问题，理解八皇后问题的每个细节】

https://www.bilibili.com/video/BV1SM4m1Q7pH/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

回溯法 pk 穷举法

- 从本质上讲，回溯法也是一种穷举法，也是通过“枚举 - 判断”来寻找问题的解，不同的是，穷举法每次枚举的都是问题的一个完整解，而回溯法每次测试的是解的一部分。
- 例如，在走迷宫问题中，当遇到岔路时，枚举其中的一条路，来验证是否满足约束条件（即查看该路是否可以通行），如果满足，该路就加入到已有的部分解集中，从而得到更大的部分解集。如果一条路不能满足约束条件，则选择下一条路，如果没有可选的路，则回溯到该岔路的上一个路口。持续该过程，直到部分解集扩展为原问题的一个解，即找到一条到出口的通路。利用递归编程，可以找出所有的问题解。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

迭代法

- 迭代法又称辗转法，是一种不断用变量的旧值递推新值的过程，跟迭代法相对应的是直接法（或者称为一次解法），即一次性解决问题。
- 严格的讲，和递归法一样，迭代法是一种编程技术。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-11：使用辗转相除法求两数的最大公约数

• 辗转相除法，又称欧几里得算法

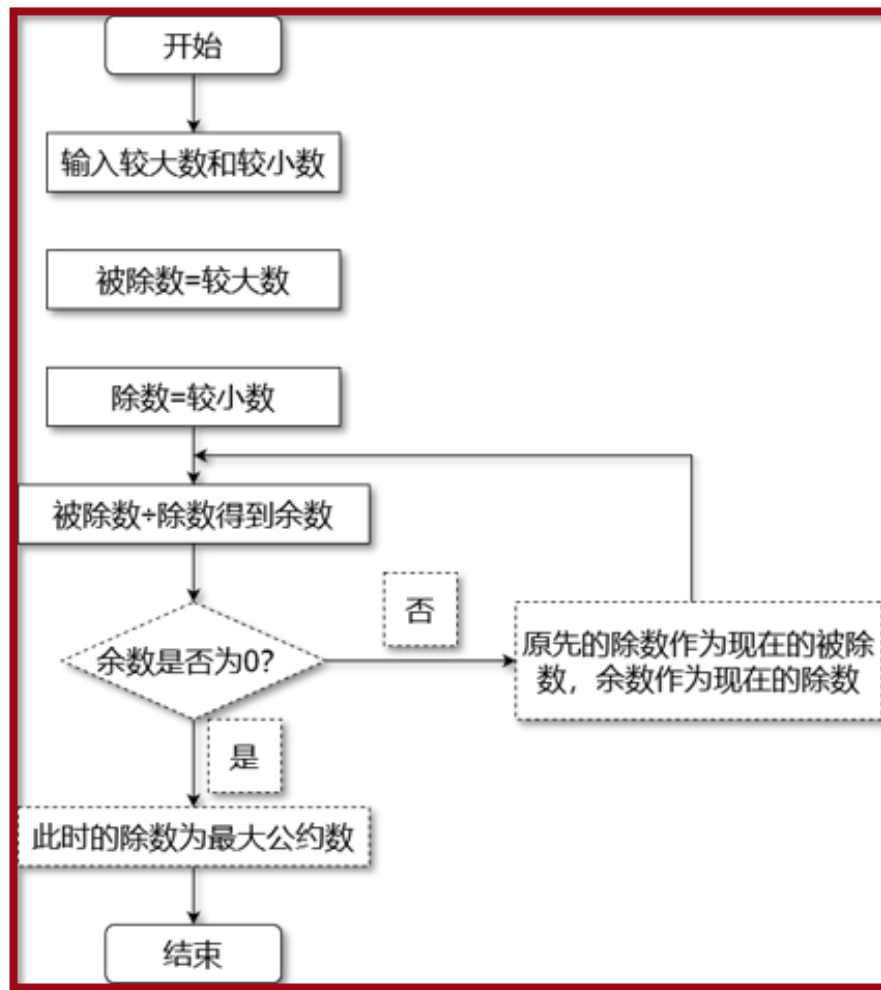
- 用较大数（被除数）除以较小数（除数），再用得到的余数（第一余数）去除除数，再用得到的余数（第二余数）去除第一余数，如此反复，直到最后余数是0为止。最后的除数就是这两个数的最大公约数。

- 例如，计算12和8的最大公约数过程如下：

- $18/12=1\cdots 6$

- $12/6=2\cdots 0$

- 因此6就是12和18的最大公约数



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



分治法

- 各个击破，分而治之 (Divide and Conquer)
- 特征
 - 问题可以分解为若干个规模较小的相同问题
 - 问题分解出的各个子问题是相互独立的，即子问题之间不包含公共子问题。
 - 问题规模缩小到一定程度时可以容易地解决
 - 问题分解出的子问题的解可以合并为该问题的解

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-13:

• 问题

- 给定一个长度为 n 的整数数组，编写一个求出其最大值和最小值的分治算法

• 分析

- 当数组中只有1个数时，可以直接给出结果，如果有2个数时，则一次比较即可得出结果。
- 当 $n > 2$ 时，我们可以将数组分成两个元素个数较小的数组，分别求解他们的最大和最小值，最后比较两个数组的解来得到原问题的解

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



分治法排序（归并排序）

• 问题

- 房间内随意散落着宽度相同、长度不同的8张纸，纸的长度都标注在纸面上，分别是8cm、4cm、5cm、7cm、1cm、3cm、6cm和2cm。现在需要收拾整理这些纸，将它们按长度顺序摞起来，最长的在下面，最短的在上面。假设每次比较两张纸的长度大小都需要花费1秒钟，对比以下两种方法的优劣。
- 方法1：逐一比较
- 方法2：分治法

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

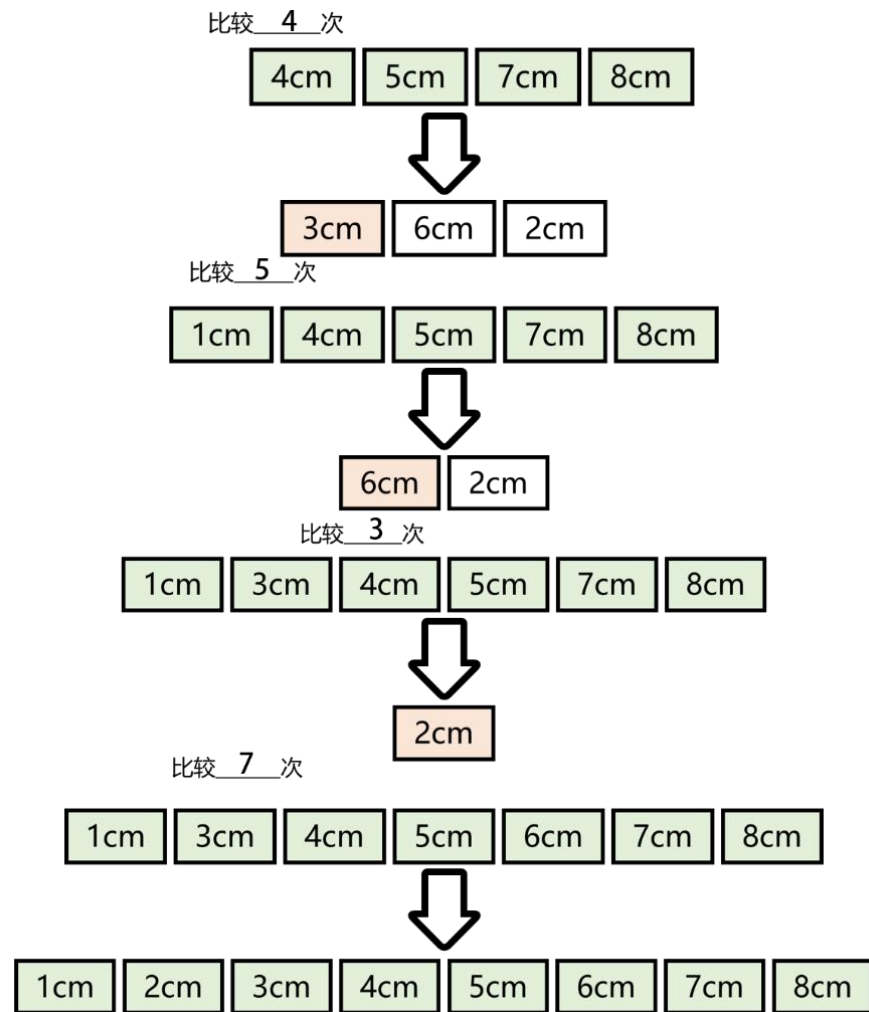
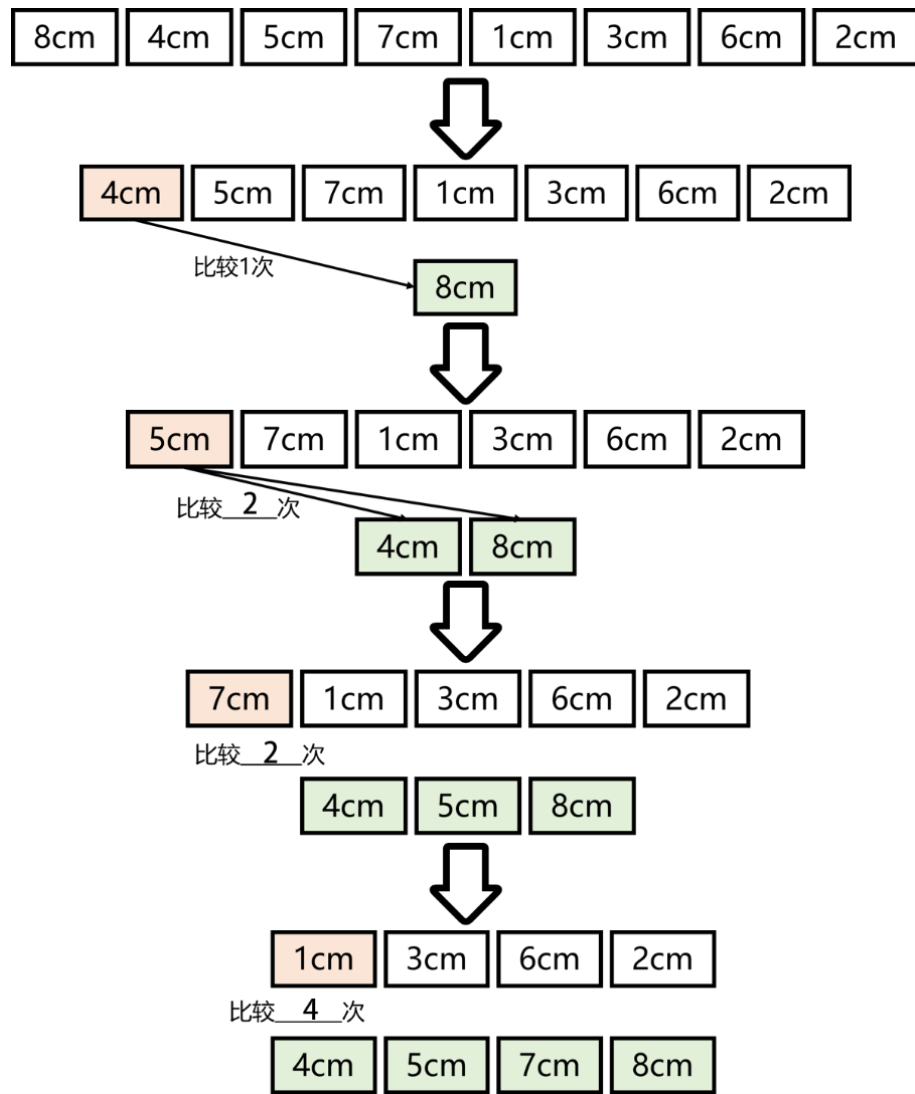
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

分治法排序 (归并排序)

方法1: 逐一比较



最终完成排序, 合计比较 24 次。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

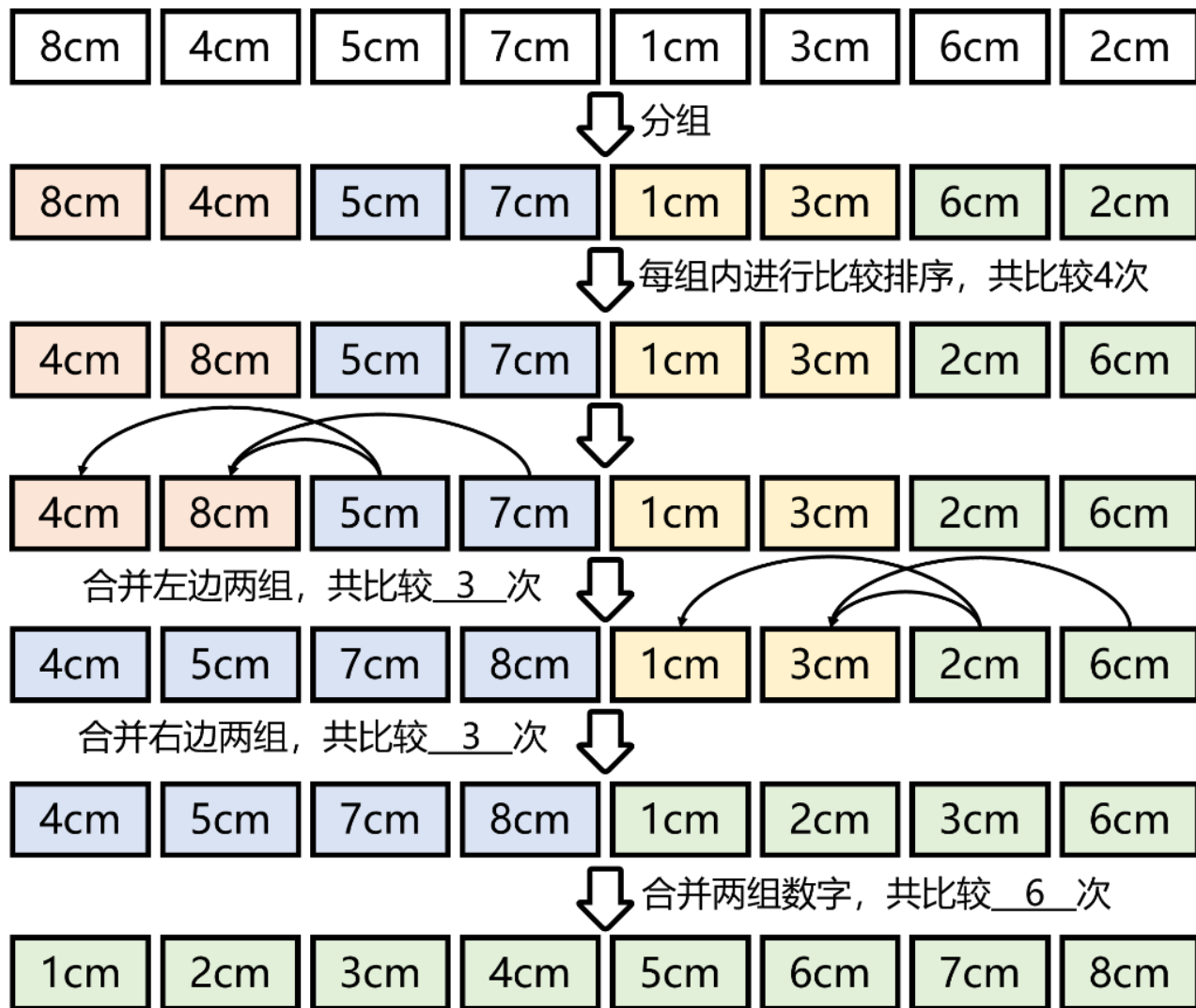
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

分治法排序（归并排序）

方法2：分治法



最终完成排序，合
计比较 16 次。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

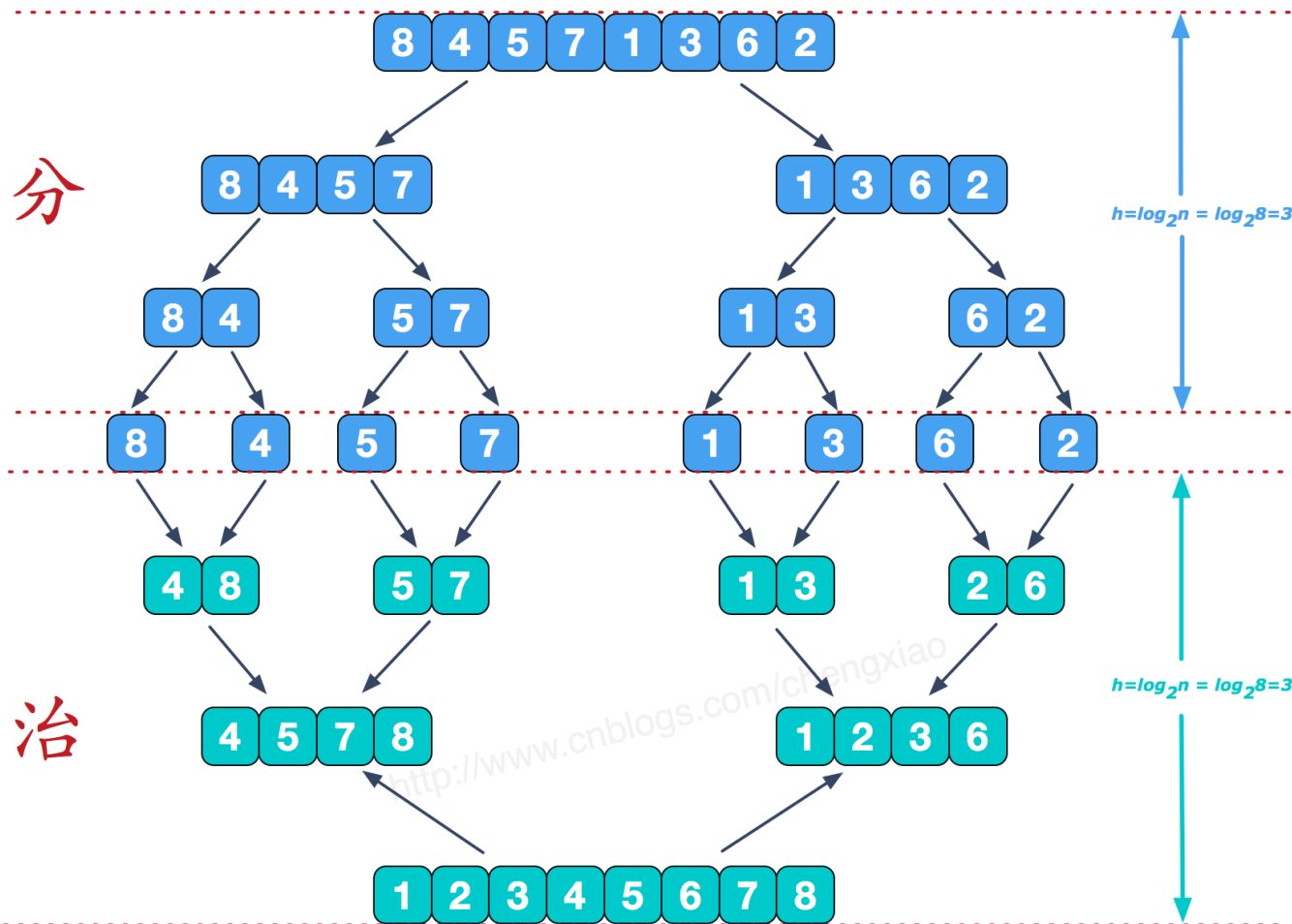
3.6 网络搜索问题

分治法排序（归并排序）

• 问题

• 给定初始序列 {8,4,5,7,1,3,6,2}，请设计算法将其从小到大排列。

• 分析



贪心法

- 贪婪法的基本思想是：从当前情况出发根据某个优化目标做最优选择，而不考虑各种可能的整体情况，从而避免了为找最优解要穷尽所有可能的问题求解方法。
- 在贪心算法中，选取一个量度标准作贪婪处理所得到该量度意义下的最优解并不一定是问题的最优解，可能是次优解。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

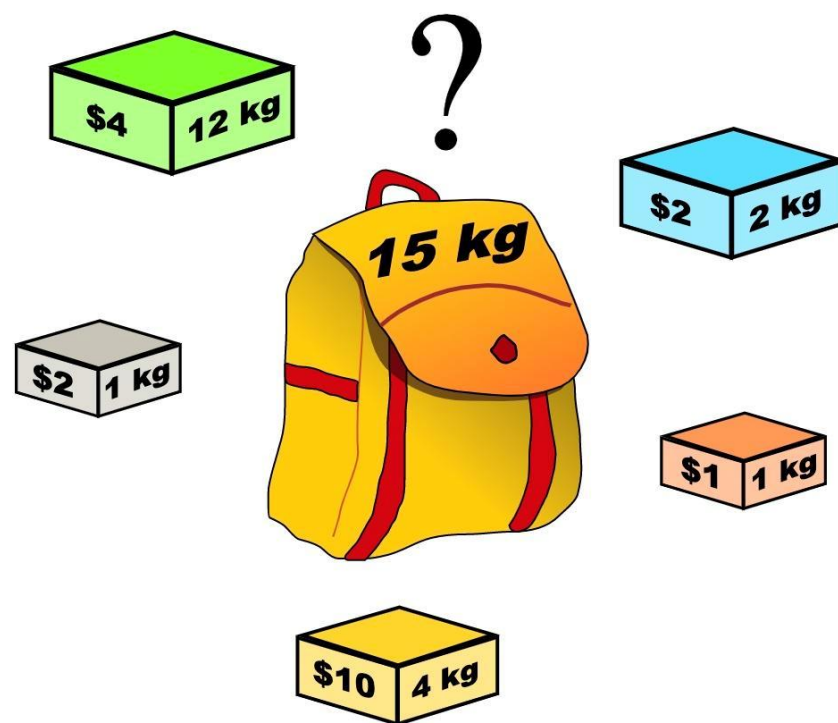
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-14：用贪心法求解0-1背包问题

- 问题描述：有N件物品和一个最多能被重量为W的背包。第i件物品的重量是 $weight[i]$ ，得到的价值是 $value[i]$ 。每件物品只能用一次，求解将哪些物品装入背包里物品价值总和最大。



求解思路：

- 穷举法逐一尝试所有组合？
可以找到最优解，但时间复杂度为 $O(2^n)$ ，是一个NP难解问题。
- 按价值从高到低选择？
- 按重量从小到大选择？

例3-14：用贪心法求解0-1背包问题

- 0-1背包问题：给定 n 种物品和一个背包，物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 W_m 。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？
- 求解0-1背包问题，采用穷举法可以找到最优解，但时间复杂度为 $O(2^n)$ ，是一个NP难解问题。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-14：用贪心法求解0-1背包问题

- 一个例子：背包最大承重 $W=5$ ，3个物品重量和价值如表所示

➤穷举法逐一尝试所有组合？

组合	总重量	总价值
{}	0	0
{A}	2	3
{B}	3	4
{C}	4	5
{A, B}	5	7
{A, C}	6	8
{B, C}	7	9
{A, B, C}	9	12

← 最优

	重量	价值
A	2	3
B	3	4
C	4	5

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-14：用贪心法求解0-1背包问题

- 一个例子：背包最大承重 $W=5$ ，3个物品重量和价值如表所示

➤按价值从高到低选择？

1. 选择物品C，价值5，重量4；
2. 由于此时物品A和B加上后都超重，因此无法再装更多物品。
3. {C}，价值5 **← 不能找到最优解**

	重量	价值
A	2	3
B	3	4
C	4	5

➤按重量从小到大选择？

1. 选择物品A，价值3，重量2；
2. 选择物品B，价值7，重量5；
3. 由于此时物品C加上后超重，因此无法继续添加。
4. {A, B}，价值7 **← 能找到最优解**

**按重量从小到大
选择一定能找到
最优解吗？**

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-14：用贪心法求解0-1背包问题

- 另一个例子：背包最大承重 $W=10$ ，4个物品重量和价值如表所示

	重量	价值
A	2	3
B	3	4
C	4	5
D	5	7

最优➡

组合	总重量	总价值
{}	0	0
{A}	2	3
{B}	3	4
{C}	4	5
{D}	5	7
{A, B}	5	7
{A, C}	6	8
{A, D}	7	10
{B, C}	7	9
{B, D}	8	11
{C, D}	9	12
{A, B, C}	9	12
{A, B, D}	10	14
{A, C, D}	11	15
{B, C, D}	12	16
{A, B, C, D}	14	19

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-14：用贪心法求解0-1背包问题

- 另一个例子：背包最大承重 $W=10$ ，4个物品重量和价值如表所示

➤按价值从高到低选择？

1. 选择物品D，价值7，重量5；
2. 选择物品C，价值12，重量9；
3. 由于此时物品A和B加上后都超重，
因此无法再装更多物品。

4. {C,D}，价值12。 ← **不能找到最优解**

	重量	价值
A	2	3
B	3	4
C	4	5
D	5	7

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-14：用贪心法求解0-1背包问题

- 另一个例子：背包最大承重 $W=10$ ，4个物品重量和价值如表所示

➤按重量从低到高选择？

1. 选择物品A，价值3，重量2；
2. 选择物品B，价值7，重量5；
3. 选择物品C，价值12，重量9；
4. 选择物品D，超重；

	重量	价值
A	2	3
B	3	4
C	4	5
D	5	7

5. $\{A, B, C\}$ ，价值12。← 不能找到最优解

有没有更合理的选择方案？

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-14：用贪心法求解0-1背包问题

- 另一个例子：背包最大承重 $W=10$ ，4个物品重量和价值如表所示

➤按价值密度从高到低选

1. 选择物品A，价值3，重量2；
2. 选择物品D，价值10，重量7；
3. 选择物品B，价值14，重量10；
4. 选择物品C，超重；
5. $\{A, B, D\}$ ，价值14。 ← 能找到最优解

	重量	价值	价值密度
A	2	3	$3/2=1.5$
B	3	4	$4/3=1.33$
C	4	5	$5/4=1.25$
D	5	7	$7/5=1.4$

- 贪心算法的核心思路：为了使包内物品的总价值最大，先计算出所有物品的单位价值，
- 然后从单位价值由高到低的顺序依次选择物品放入背包，在不超过背包限重的情况下尽可能放更多的物品。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3-14：用贪心法求解0-1背包问题

- 贪心算法追求局部最优，不一定能找到全局最优

物品 [↕]	价值 [↕]	重量 [↕]	单位价值 [↕]
1 [↕]	10 [↕]	35 [↕]	0.29 [↕]
2 [↕]	40 [↕]	30 [↕]	1.33 [↕]
3 [↕]	30 [↕]	60 [↕]	0.5 [↕]
4 [↕]	50 [↕]	50 [↕]	1 [↕]
5 [↕]	35 [↕]	40 [↕]	0.88 [↕]
6 [↕]	40 [↕]	10 [↕]	4 [↕]
7 [↕]	30 [↕]	25 [↕]	1.2 [↕]

• $W=100$

- 按贪心方法，各物品单位价值从高到低分别6、2、7、4、5、3、1，选取物品6、物品2、物品7、物品1，总重量恰好100，总价值为120。
- 事实上这并不是最优解，最优解是物品6、物品2、物品4，总重量虽然为90，但总价值是130。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

其他背包问题

- 完全背包问题：有 N 种物品和一个容量为 V 的背包，每种物品都有无限件可用。第 i 种物品的体积是 c ，价值是 w 。将哪些物品装入背包可使这些物品的体积总和不超过背包容量，且价值总和最大。
- 多重背包问题：有 N 种物品和一个容量为 V 的背包。第 i 种物品最多有 n 件可用，每件体积是 c ，价值是 w 。求解将哪些物品装入背包可使这些物品的体积总和不超过背包容量，且价值总和最大。
- 分组背包问题：有 N 件物品和一个容量为 V 的背包。第 i 件物品的容量是 c ，价值是 w 。这些物品被划分为若干组，每组中的物品互相冲突，最多选一件。求解将哪些物品装入背包可使这些物品的容量总和不超过背包容量，且价值总和最大。

【运筹科普系列小故事之背包问题】

https://www.bilibili.com/video/BV1tgSiY2Ei8/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



动态规划

- 动态规划(Dynamic Programming, 简称DP)是运筹学的一个分支, 它是解决多阶段决策过程最优化的一种数学方法。把多阶段问题变换为一系列相互联系的单阶段问题, 然后逐个加以解决。
- 动态规划算法通常用于求解具有某种最优性质的问题。在这类问题中, 可能会有许多可行解。每一个解都对应于一个值, 我们希望找到具有最优值的解。
- 可以直观地理解为带记录的递归。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

动态规划

- 动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题，先求解子问题，然后从这些子问题的解得到原问题的解。
- 与分治法不同的是，适合于用动态规划求解的问题，经分解得到子问题往往不是互相独立的。若用分治法来解这类问题，则分解得到的子问题数目太多，有些子问题被重复计算了很多次。如果能够保存已解决的子问题的答案，而在需要时再找出已求得的答案，这样就可以避免大量的重复计算，节省时间。
- 动态规划算法没有统一的处理方法，必须根据问题的各种性质并结合一定的技巧来处理。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：整数三角形问题

- 有一个群岛，共分为若干层，第1层有一个岛屿，第2层有2个岛屿，……，第 n 层有 n 个岛屿。每个岛上都有一块宝，其价值是一个正整数。寻宝者只允许从第一层的岛屿进入，从第 n 层的岛屿退出，不能后退，他能收集他所经过的所有岛屿上的宝贝。但是，从第 i 层的岛屿进入第 $i + 1$ 层的岛屿时，有且仅有2条路径。
- 你的任务是：对于给定的群岛和岛上宝贝的价值，计算一个寻宝者行走一趟所能收集宝贝的最大价值。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

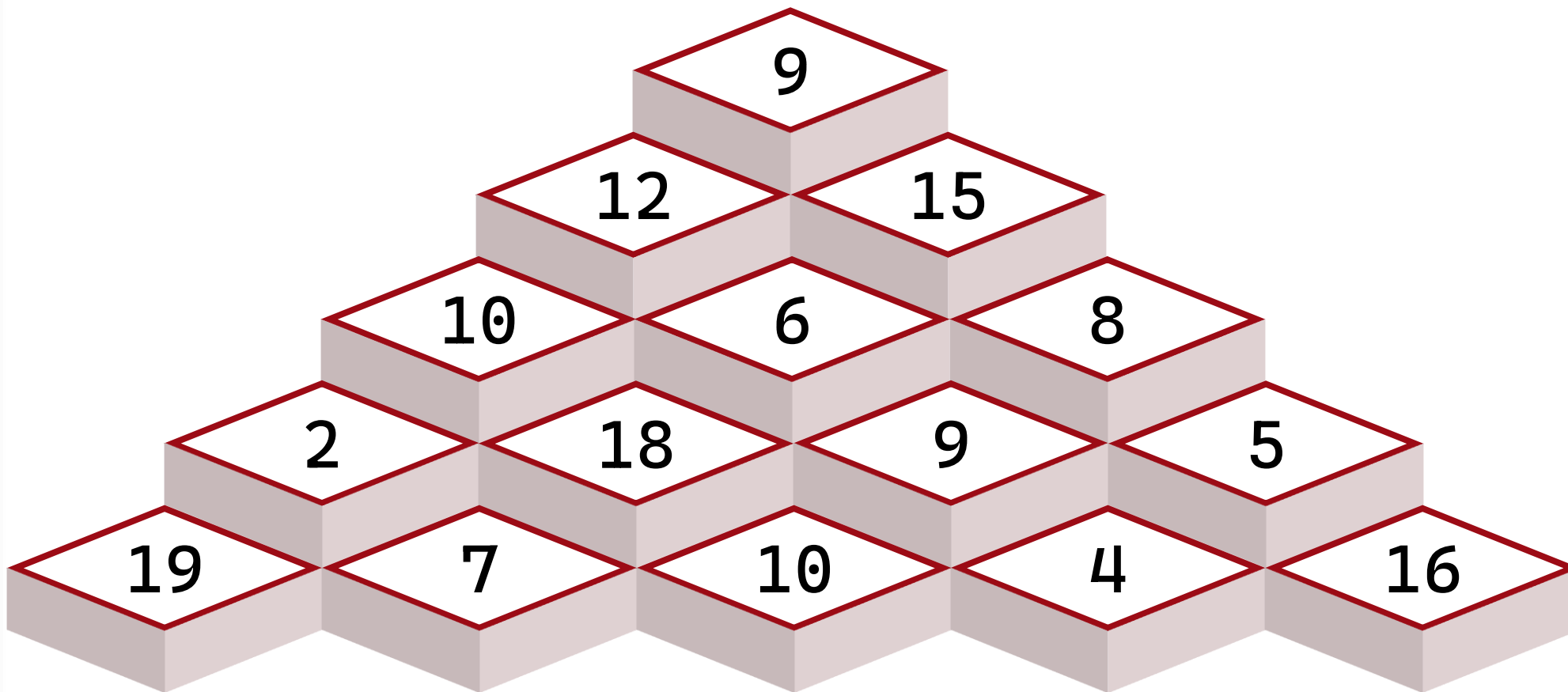
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：整数三角形问题



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

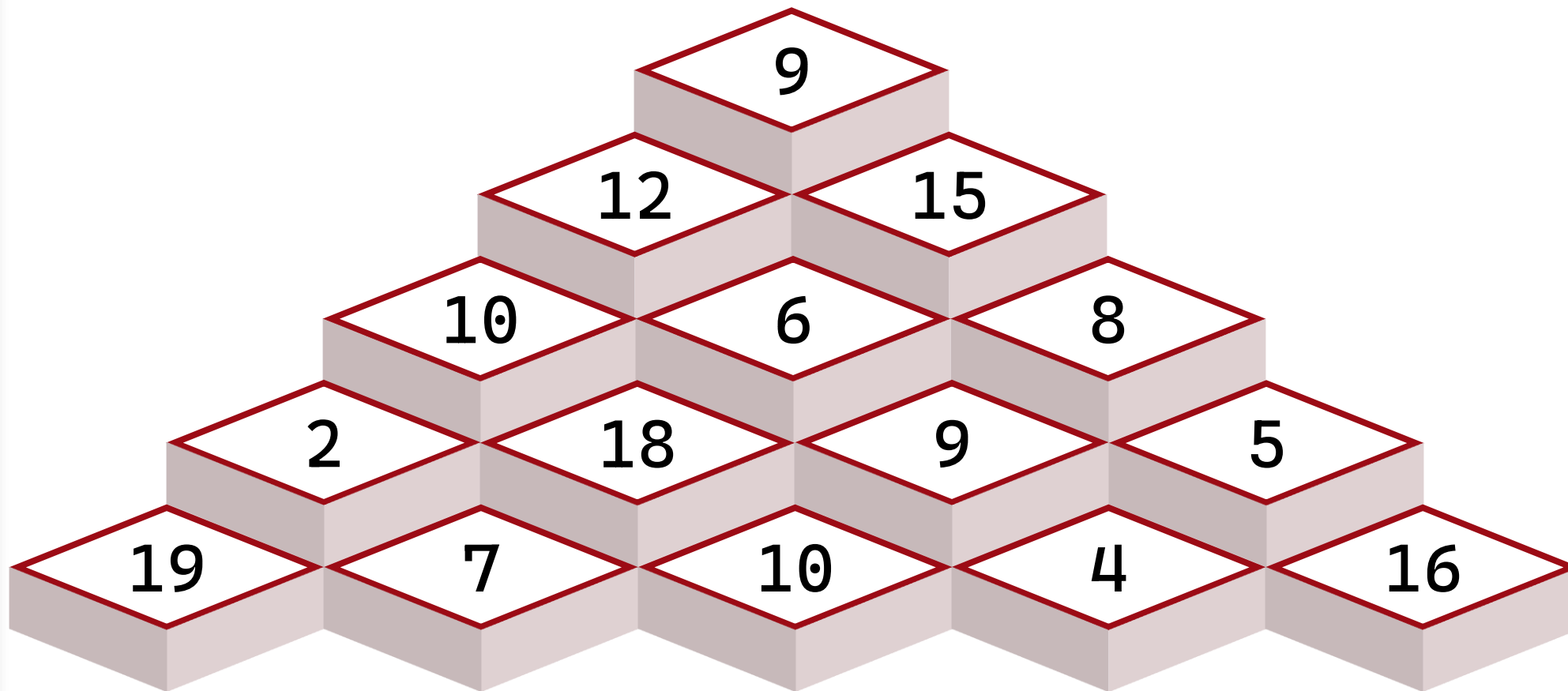
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：整数三角形问题



贪心法：9→15→8→9→10，总价值51

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

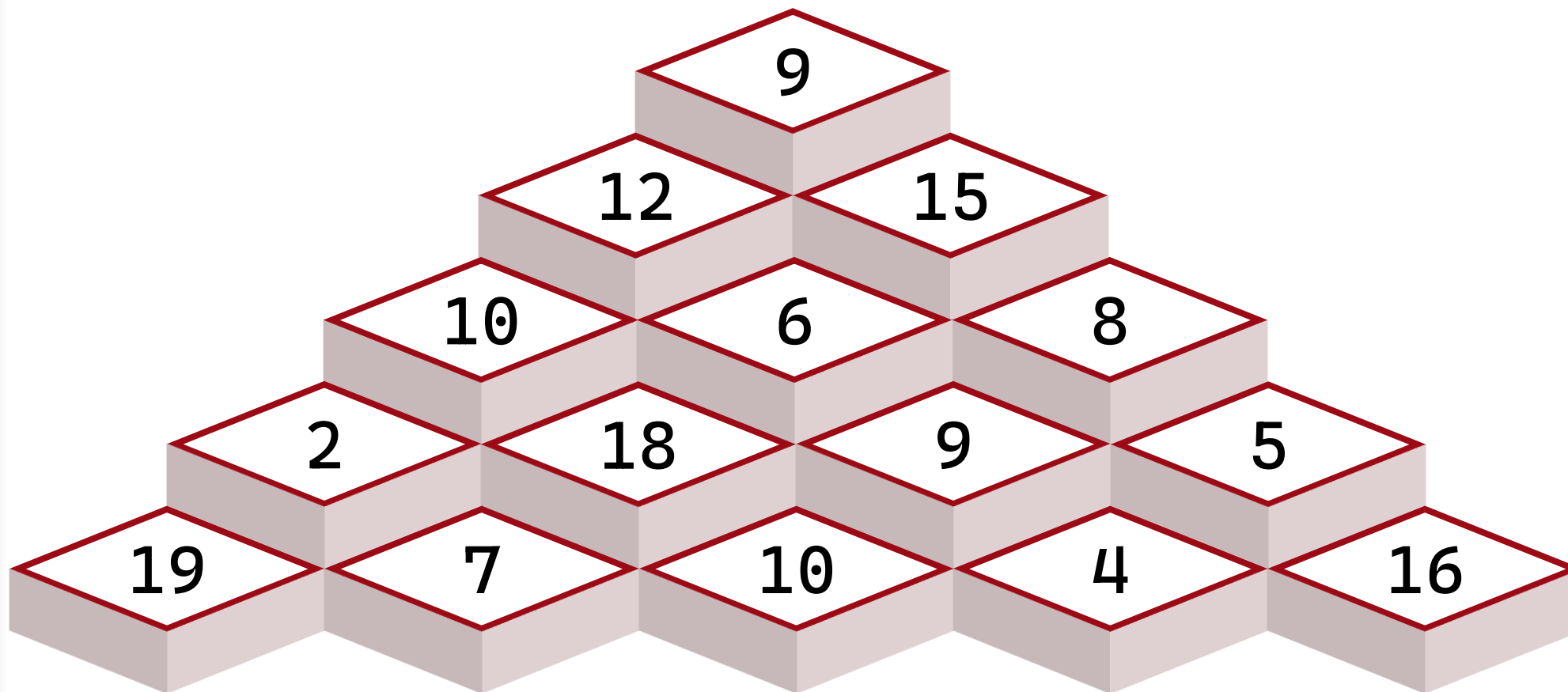
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：整数三角形问题



贪心法：9→15→8→9→10，总价值51

穷举法：9→12→10→18→10，总价值59

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

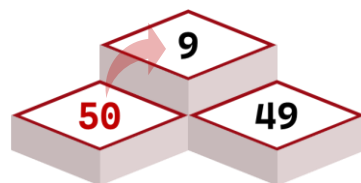
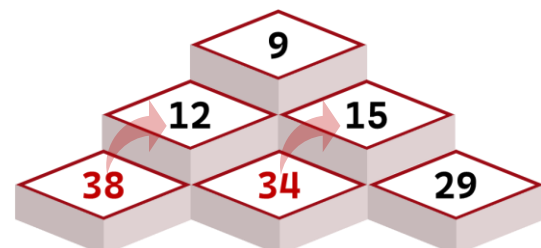
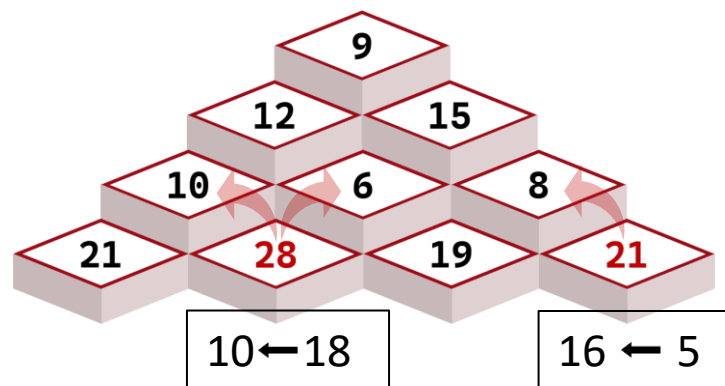
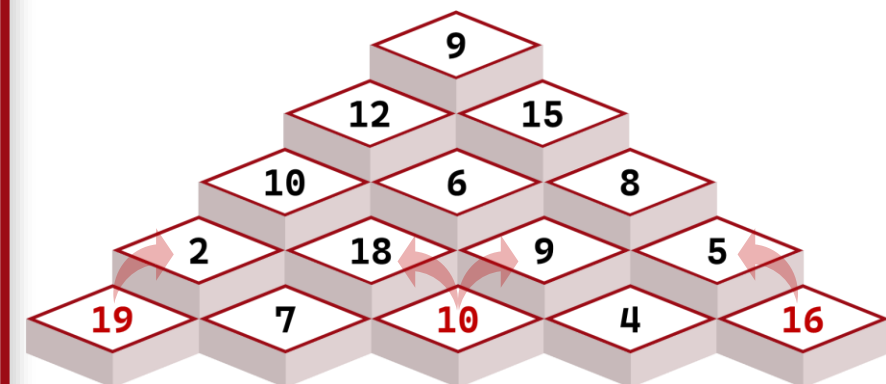
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：整数三角形问题

动态规划：从终点出发倒推，并记录最优路径和计算结果



10 ← 18 ← 10

10 ← 18 ← 6

10 ← 18 ← 10 ← 12

10 ← 18 ← 10 ← 12 ← 9

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

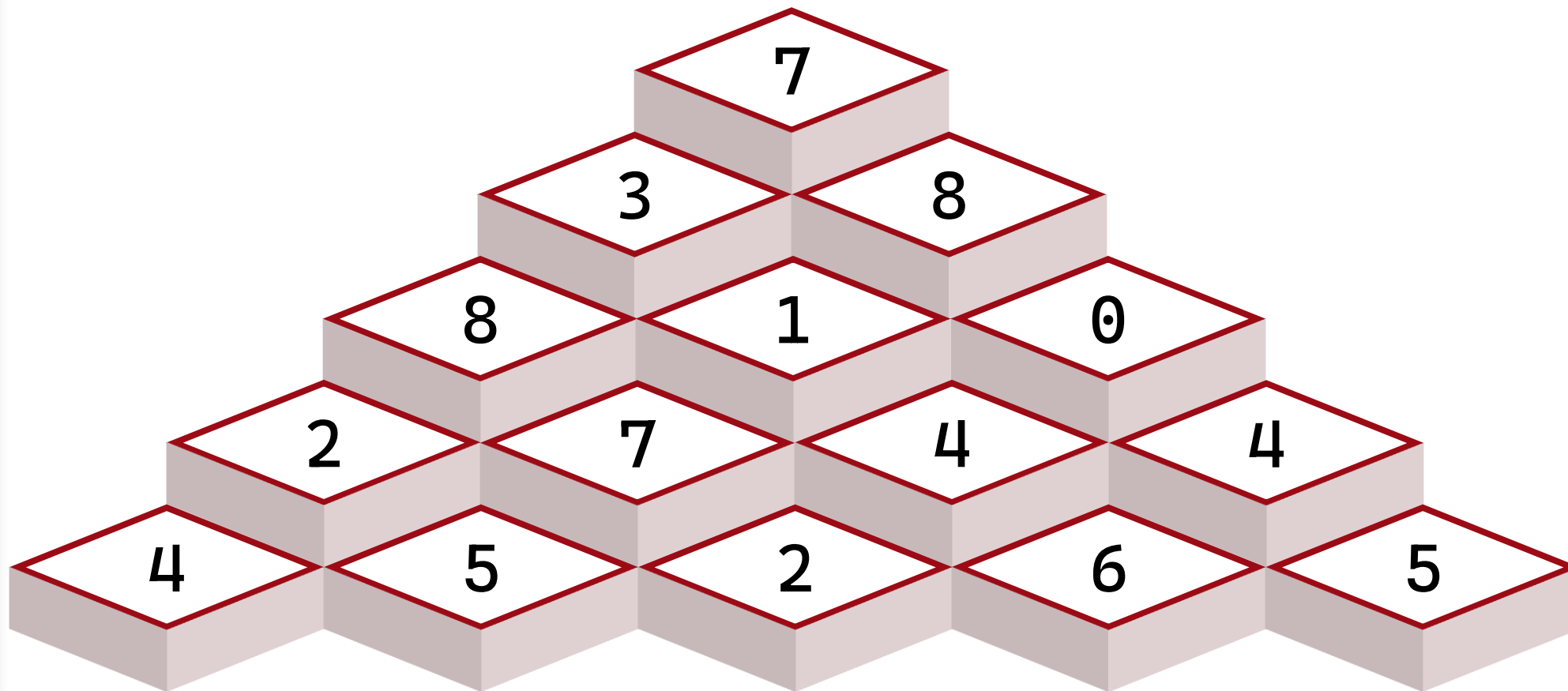
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：整数三角形问题（换一组数字）



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

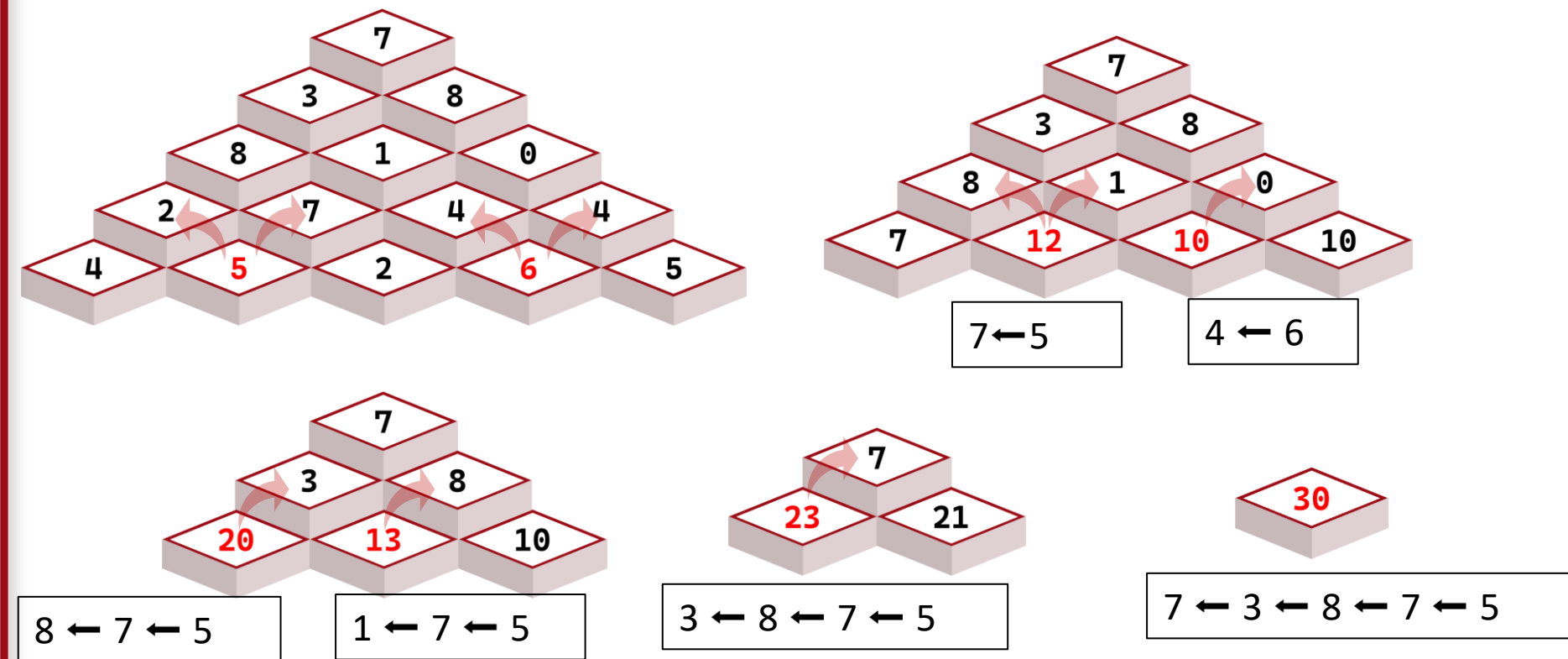
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：整数三角形问题（换一组数字）



【不是我催... 以后大家学动态规划，都得要看这个视频。】

https://www.bilibili.com/video/BV1m9EUzLEB4/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：背包问题-动态规划

- 背包最大承重 $W=10$ ，4个物品重量和价值如表所示

	重量	价值
A	2	3
B	3	4
C	4	5
D	5	7

最优➡

组合	总重量	总价值
{}	0	0
{A}	2	3
{B}	3	4
{C}	4	5
{D}	5	7
{A, B}	5	7
{A, C}	6	8
{A, D}	7	10
{B, C}	7	9
{B, D}	8	11
{C, D}	9	12
{A, B, C}	9	12
{A, B, D}	10	14
{A, C, D}	11	15
{B, C, D}	12	16
{A, B, C, D}	14	19

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：背包问题-动态规划

记录 $dp[i][j]$ ：从 $0 \sim i$ 个物品中任取物品放入容量为 j 的背包，价值总和最大多少？

	重量	价值
1	2	3
2	3	4
3	4	5
4	5	7

如果不选取第 i 个物品： $dp[i][j]=dp[i-1][j]$

$W=10$

如果选取第 i 个物品： $dp[i][j]=dp[i-1][j-\text{weight}[i]]+\text{value}[i]$

在给物品 i 留够容量后，剩余容量能装载的最大价值
物品 i 的价值

综合：

$dp[i][j]=\max(dp[i-1][j], dp[i-1][j - \text{weight}[i]] + \text{value}[i])$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i - 1][j] \\ dp[i - 1][j - weight[i]] + value[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7	7
3	0	0	3	4	5	7	8	9	9	12	12
4	0	0	3	4	5	7	8	10	11	12	14

	重量	价值
1	2	3
2	3	4
3	4	5
4	5	7

W=10

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例1： dp[2,3]表示重量限制为3， 可选物品为1、 2， dp[1][3]=3， dp[1][0]+4=4， 因此dp[2,3]=4

例2： dp[2,5]表示重量限制为5， 可选物品为1、 2， dp[1][5]=3， dp[1][2]+4=7， 因此dp[2,5]=7

例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i - 1][j] \\ dp[i - 1][j - weight[i]] + value[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7	7
3	0	0	3	4	5	7	8	9	9	12	12
4	0	0	3	4	5	7	8	10	11	12	14

	重量	价值
1	2	3
2	3	4
3	4	5
4	5	7

W=10

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例3： dp[3,8]表示重量限制为8，可选物品为1、2、3，
dp[2][8]=7， dp[2][4]+5=9， 因此dp[3,8]=9
例4： dp[4][10]表示重量限制为10，可选物品为1、2、3、4，
dp[3][10]=12， dp[3][5]+7=14， 因此dp[4,10]=14

例：背包问题-动态规划

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	3	3	3	3	3	3	3	3	3
2	0	0	3	4	4	7	7	7	7	7	7
3	0	0	3	4	5	7	8	9	9	12	12
4	0	0	3	4	5	7	8	10	11	12	14

	重量	价值
1	2	3
2	3	4
3	4	5
4	5	7

W=10

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

1. 从dp[4][10]开始，发现dp[4][10]≠dp[3][10]，说明第4个物品被选取。
剩余容量：5。
2. 查看dp[3][5]和dp[2][5]，发现dp[3][5]=dp[2][5]，说明第3个物品未被选取。
3. 继续查看dp[2][5]和dp[1][5]，发现dp[2][5]≠dp[1][5]，说明第2个物品被选取。剩余容量：2。
4. 最后，dp[1][2]≠dp[0][2]，说明第1个物品被选取。
- 由此可知，最优选取：1、2、4，最大价值14。

例：背包问题-动态规划（换一组数据）

- 背包最大承重 $W=10$ ，5个物品重量和价值如表所示

	重量	价值
A	2	6
B	5	3
C	4	5
D	2	4
E	3	6

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i - 1][j] \\ dp[i - 1][j - weight[i]] + value[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1											
2											
3											
4											
5											

	重量	价值
A	2	6
B	5	3
C	4	5
D	2	4
E	3	6

W=10

算法	
一、算法基础	
1	问题与问题求解
2	算法与算法分析
3	算法设计及算法分

类	
二、常见算法	
3.4	搜索问题与查找算 法
3.5	排序问题及排序算 法
3.6	网络搜索问题



例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i-1][j] \\ dp[i-1][j - \text{weight}[i]] + \text{value}[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6	6	6
2											
3											
4											
5											

	重量	价值
A	2	6
B	5	3
C	4	5
D	2	4
E	3	6

W=10

算法

一、算法基础

1 问题与问题求解

2 算法与算法分析

3 算法设计及算法分

类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i-1][j] \\ dp[i-1][j - weight[i]] + value[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	6	6	6	9	9	9	9
3											
4											
5											

	重量	价值
A	2	6
B	5	3
C	4	5
D	2	4
E	3	6

W=10

算法
一、算法基础
1 问题与问题求解
2 算法与算法分析
3 算法设计及算法分
类
二、常见算法
3.4 搜索问题与查找算
法
3.5 排序问题及排序算
法
3.6 网络搜索问题

例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i-1][j] \\ dp[i-1][j - weight[i]] + value[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	6	6	6	9	9	9	9
3	0	0	6	6	6	6	11	11	11	11	11
4											
5											

	重量	价值
A	2	6
B	5	3
C	4	5
D	2	4
E	3	6

W=10

算法
一、算法基础
1 问题与问题求解
2 算法与算法分析
3 算法设计及算法分
类
二、常见算法
3.4 搜索问题与查找算
法
3.5 排序问题及排序算
法
3.6 网络搜索问题



例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i - 1][j] \\ dp[i - 1][j - weight[i]] + value[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	6	6	6	9	9	9	9
3	0	0	6	6	6	6	11	11	11	11	11
4	0	0	6	6	10	10	11	11	15	15	15
5											

	重量	价值
A	2	6
B	5	3
C	4	5
D	2	4
E	3	6

W=10

类

算法

一、算法基础

1 问题与问题求解

2 算法与算法分析

3 算法设计及算法分

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i - 1][j] \\ dp[i - 1][j - weight[i]] + value[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	6	6	6	9	9	9	9
3	0	0	6	6	6	6	11	11	11	11	11
4	0	0	6	6	10	10	11	11	15	15	15
5	0	0	6	6	10	12	12	16	16	17	17

	重量	价值
A	2	6
B	5	3
C	4	5
D	2	4
E	3	6

W=10

算法
一、算法基础
1 问题与问题求解
2 算法与算法分析
3 算法设计及算法分

二、常见算法
3.4 搜索问题与查找算 法
3.5 排序问题及排序算 法
3.6 网络搜索问题

例：背包问题-动态规划

$$dp[i][j] = \max \begin{cases} dp[i-1][j] \\ dp[i-1][j - weight[i]] + value[i] \end{cases}$$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	6	6	6	9	9	9	9
3	0	0	6	6	6	6	11	11	11	11	11
4	0	0	6	6	10	10	11	11	15	15	15
5	0	0	6	6	10	12	12	16	16	17	17

	重量	价值
A	2	6
B	5	3
C	4	5
D	2	4
E	3	6

W=10

类

算法

一、算法基础

1 问题与问题求解

2 算法与算法分析

3 算法设计及算法分

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

dp[5][10]>dp[4][10]说明物品E被选中，去掉物品E的重量后还剩W=7的空间；
dp[4][7]=dp[3][7]说明物品D没被选中，dp[3][7]>dp[2][7]说明物品C被选中，还剩W=3；
dp[2][3]=dp[1][3]说明物品B没被选中，dp[1][3]>dp[0][3]说明物品A被选中。
因此，最优装法选择物品ACE，总价值17。

算法总结

算法名称	核心思想	问题案例
穷举法	逐一列举所有可能的求解方案	百钱买百鸡（多元方程组求解问题）
递推法	根据递进关系求解问题	斐波那契、卡车储油点
递归法	复杂问题递推简单问题，求解后代入回复杂问题	阶乘、汉诺塔问题
回溯法	试探 - 失败返回 - 再试探	走迷宫、八皇后问题
迭代法	不断用变量旧值递推计算新值	辗转相除法
分治法	大问题划分为小问题，再合并求解结果	数组排序
贪心法	始终选择当前最优的行动	背包问题
动态规划	存储子问题最优解	整数三角形问题、背包问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结

思考：预测一段DNA能编码什么样的蛋白质

在生物体内，基因（DNA）通过转录和翻译指导蛋白质的合成。但这个过程并不简单：DNA序列中包含**外显子**（exon）—— 会被保留并翻译成蛋白质的部分。也包含**内含子**（intron）—— 在RNA加工过程中被剪切掉的“非编码”部分。剪接（splicing）是指将外显子连接起来，形成成熟的mRNA，再翻译成蛋白质。

一个典型的基因结构如下：

DNA: ...ATG [外显子1] GT...AG [内含子] GT...AG [外显子2] TAA...

 ↑ ↑ ↑ ↑ ↑

 起始 剪接供体 剪接受体 剪接供体 终止

 密码子 位点 (SD) 位点 (SA) 位点 (SD) 密码子

科学家的任务是：从一段DNA序列中，找出最可能的外显子组合路径，预测它能编码的最长、最合理的蛋白质。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结

思考：预测一段DNA能编码什么样的蛋白质

例：你获得了一段来自某生物的DNA序列：

DNA序列 (30 bp) :

5'-ATG CGTAGCTAG CTGACGTAG TAGCTAGCC TGA-3'



已知：

- 蛋白质翻译从 **起始密码子 ATG** 开始。
- 翻译在遇到 **终止密码子 (TAA、TAG 或 TGA)** 时结束。
- 外显子之间通过剪接连接，剪接通常发生在 GT...AG 序列边界（即剪接供体SD和受体SA）。
- 每个外显子长度一般 ≥ 6 bp。

你的任务是：

预测这段DNA可能编码的最长蛋白质，并解释你是如何找到它的。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结

思考：预测一段DNA能编码什么样的蛋白质

[方法一] 穷举法：尝试所有可能的剪接方式

问题：我们不知道内含子从哪开始、在哪结束。

穷举策略：尝试所有可能的“剪接位点”组合。

比如：剪掉第5-10位？ 5-15？ 6-10？

对每种剪法，把剩下的外显子拼起来，翻译成蛋白质，看哪个最长。

✓ **优点：**一定能找到最优解。

✗ **缺点：**如果有10个可能的剪接位点，组合数爆炸，算到天荒地老。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结

思考：预测一段DNA能编码什么样的蛋白质

[方法二]分治法：把大基因拆成小段分析

问题：我们不知道内含子从哪开始、在哪结束。

策略：

1. 先找“起始密码子” ATG（通常是蛋白质开头）。
2. 再找“终止密码子” TAA/TAG/TGA（蛋白质结尾）。
3. 把DNA按这些关键点**分成若干段**。
4. 分别分析每段是否可能为外显子（比如看GC含量、保守性）。
5. 最后再尝试组合这些“候选外显子”。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结

思考：预测一段DNA能编码什么样的蛋白质

[方法三]回溯法：尝试拼接外显子，错了就退回

场景：我们有候选外显子列表：[Ex1, Ex2, Ex3]

策略：

1. 先尝试 $\text{ATG} + \text{Ex1} + \text{Ex2} + \text{TAA} \rightarrow$ 翻译成蛋白质，长度100。
2. 再尝试 $\text{ATG} + \text{Ex1} + \text{Ex3} + \text{TAA} \rightarrow$ 发现Ex1和Ex3拼接后出现提前终止 $\rightarrow$ **不合理，退回。**
3. 换 $\text{ATG} + \text{Ex2} + \text{Ex3} + \text{TAA} \rightarrow$ 长度120，更好。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结

思考：预测一段DNA能编码什么样的蛋白质

[方法四]贪心法：每次都选得分最高的外显子

场景：我们有候选外显子列表：[Ex1, Ex2, Ex3]

策略：

1. 给每个外显子打分（基于长度、保守性、剪接信号强度）。
2. 从ATG开始，**每次都选当前得分最高的外显子**接上。
3. 直到遇到终止密码子。

✗ 缺点：可能错过全局最优（比如两个中等得分外显子组合更好）。

✓ 优点：快！适合初步筛选。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结

思考：预测一段DNA能编码什么样的蛋白质

[方法五]动态规划：全局最优剪接路径

思想：我们想找到**得分最高**的外显子组合路径。

做法：

把DNA序列看成一串位置。

定义 $dp[i]$ = 到位置 i 为止，能获得的最高分。

遍历所有可能的外显子，更新 dp 表。

最终 $dp[n]$ 就是全局最优解。

代表工具：GENSCAN、Augustus 等基因预测软件的核心算法。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结练习题

1. 在计算两个大整数相乘时，将大整数分成两半分别计算再合并结果，这种方法属于：

- A. 递推法
- B. 分治法
- C. 动态规划
- D. 贪心法

2. 小红在解决迷宫问题时，采用了“走一步看看能否到终点，不能到达就退回来换一条路继续尝试”的方法。这种解决问题的思路最接近：

- A. 递推法
- B. 回溯法
- C. 迭代法
- D. 分治法

3. 小明在计算阶乘时，发现可以把问题分解为： $n! = n * (n-1)!$ ，这种思路属于：

- A. 递推法
- B. 递归法
- C. 迭代法
- D. 穷举法

4. 在登山过程中，为了尽可能快地到达山顶，每次都选择当前位置最陡的路径往上走。这种解决问题的策略最符合：

- A. 贪心法
- B. 动态规划
- C. 回溯法
- D. 递推法



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

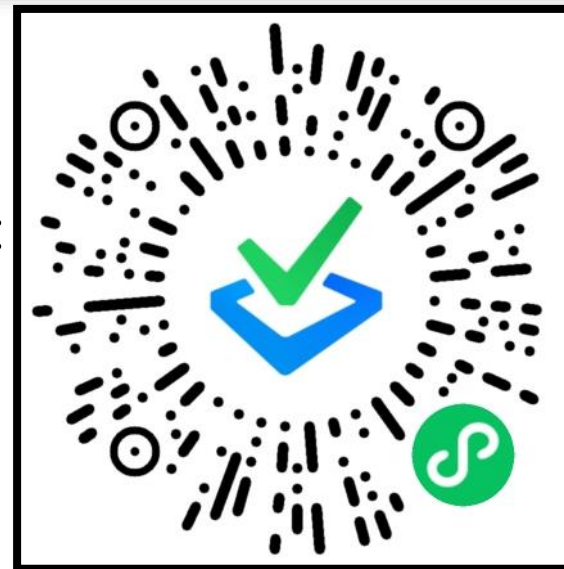
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结练习题



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

5. 一只青蛙一次可以跳上1级台阶，也可以跳上2级台阶。例：

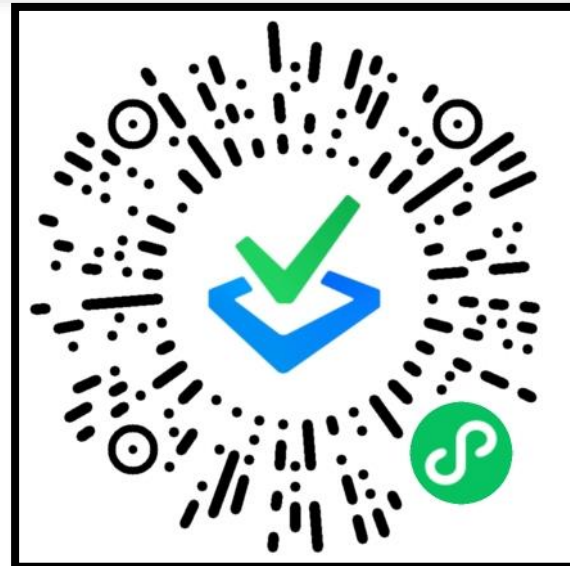
- 当有2级台阶时，青蛙有2种跳法：
 - a. 一次跳1级，再跳1级
 - b. 直接跳2级
- 当有3级台阶时，青蛙有3种跳法：
 - a. 一次跳1级，再跳1级，再跳1级
 - b. 先跳1级，再跳2级
 - c. 先跳2级，再跳1级

要求计算该青蛙跳上一个 n 级的台阶总共有多少种不同的跳法。请理解题目内容，完成以下选择题：

(1) 小华在解决青蛙跳台阶问题时，是这样思考的：“要跳上5级台阶，最后一步可以是跳1级或跳2级。如果最后一步跳1级，那就要先跳上4级；如果最后一步跳2级，那就要先跳上3级……” 以下哪个选项最准确地描述了这种思考方式的特点？

- A. 从后往前思考，把大问题分解成小问题，并利用小问题的解
- B. 不断尝试各种跳法，直到找到所有可能的组合
- C. 每次选择当前最优的跳法，以求得最少的跳跃次数
- D. 用数学公式直接推导出结果

算法总结练习题



5. 一只青蛙一次可以跳上1级台阶，也可以跳上2级台阶。例：

- 当有2级台阶时，青蛙有2种跳法：
 - a. 一次跳1级，再跳1级
 - b. 直接跳2级
- 当有3级台阶时，青蛙有3种跳法：
 - a. 一次跳1级，再跳1级，再跳1级
 - b. 先跳1级，再跳2级
 - c. 先跳2级，再跳1级

要求计算该青蛙跳上一个 n 级的台阶总共有多少种不同的跳法。请理解题目内容，完成以下选择题：

(2) 关于青蛙跳台阶问题，以下说法正确的是：

- A. 跳上 n 级台阶的跳法总数只需要知道跳上 $(n-1)$ 级的跳法总数就够了
- B. 跳上 n 级台阶的跳法总数等于跳上 $(n-1)$ 级和 $(n-2)$ 级台阶的跳法总数之和
- C. 这个问题最适合用贪心算法解决，因为每次跳都要选择最优的跳法
- D. 这个问题最适合用穷举法解决，需要列出所有可能的跳法

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法总结练习题

5. 一只青蛙一次可以跳上1级台阶，也可以跳上2级台阶。例：

- 当有2级台阶时，青蛙有2种跳法：
 - a. 一次跳1级，再跳1级
 - b. 直接跳2级
- 当有3级台阶时，青蛙有3种跳法：
 - a. 一次跳1级，再跳1级，再跳1级
 - b. 先跳1级，再跳2级
 - c. 先跳2级，再跳1级

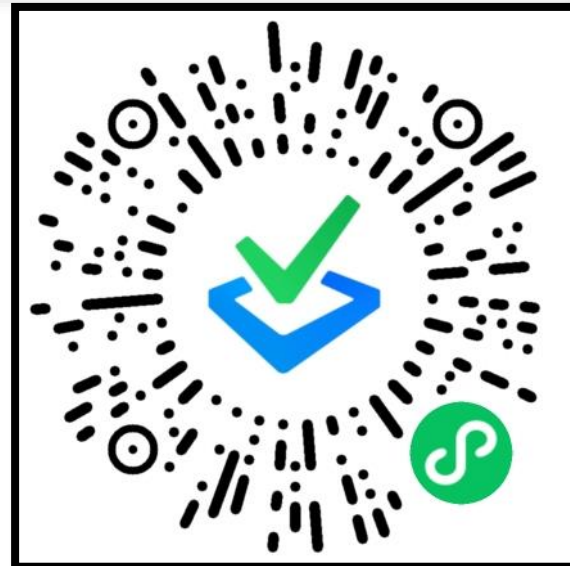
要求计算该青蛙跳上一个 n 级的台阶总共有多少种不同的跳法。
请理解题目内容，完成以下选择题：

(3) 小明在解决青蛙跳台阶问题时发现：要跳上第 n 级台阶的跳法，可以从两种情况得到：

- a. 先跳到第 $(n-1)$ 级，再跳1级
- b. 先跳到第 $(n-2)$ 级，再跳2级

这种思路最符合以下哪种算法思想？

- A. 穷举法 B. 贪心法 C. 动态规划 D. 回溯法



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题





3.4 搜索问题与查找算法

- 搜索问题
- 顺序查找
- 折半查找

知识要点

搜索，
关键字，
主关键字，
次关键字，
顺序查找，
折半查找，
平均查找长度。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



搜索问题

- 在人们的生活和工作中
 - 到教室找一个同学
 - 在商场，找一样商品
 - 在书店、图书馆，找一本书
 - 在网络中，查找信息
- 搜索（Search），就是指仔细查找、搜寻的意思
 - 搜索的对象
 - 搜索的范围

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



基本概念

- 文件（File），若干条结构相同的记录构成，文件一般存储在外存储器中，用于永久的保存信息。
- 记录，是由若干相关字段取值构成的数据整体。
- 字段（Field）是最小的不可分割的数据单位，每个字段都有一个特定的数据类型。
- 关键字（Keyword），就是记录中的一个字段，用它来标志和控制一条记录。
 - 主关键字，能够唯一的标志与控制一个记录的关键字称为主关键字
 - 其它的称为辅关键字（次关键字）

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



举例

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

字段

次关键字

主关键字

序号	书名	主编	其他作者	出版社	出版时间		ISBN
1	大学计算机--计算思维的视角 (第3版)	郝兴伟		高等教育出版社	2014-04		978-7-04-034833-0
2	计算机网络技术及应用(第三版)	郝兴伟		高等教育出版社	2013-07		978-7-04-037826-9
3	Web 技术导论 (第3版)	郝兴伟		清华大学出版社	2012-09		978-7-302-29629-4
4	C/C++程序设计	郝兴伟	贾小珠, 周转, 刘明军, 王璐	高等教育出版社	2010-08		978-7-04-030288-2
5	数据库技术与程序设计	郝兴伟	郭爱章, 韩振, 李蕴, 贾晓毅	山东大学出版社	2010-01		978-7-5607-3067-7
⋮							

记录 →

搜索问题的抽象

- 给定一个文件（或表） $F=\{R_1, R_2, \dots, R_n\}$ ，其中 R_i 是以 K_i （ $i=1, 2, \dots, n$ ）为**主关键字**的记录。现给定一个主关键字 K ，在文件 F 中确定具有主关键字 K 的记录 R 的地址或序号 i ，使得 $K_i=K$ 。
- 结果有两种可能：一种是找到了主关键字值为 K 的记录 R_i ，称为查找成功；第二种情况是，没有找到主关键字值为 K 的记录，称为“查找失败”。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



搜索算法

- 生活中的查找
 - 如何查找的?
 - 查找的方法已经随着生活经验存在于我们的大脑中
- 顺序查找
- 折半查找
- 查找类算法性能评价
 - 时间复杂性
 - 查全率
 - 查准率

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



顺序查找

- 顺序查找就是在要查找范围内，对一个个对象进行比较，看是否需要查找的对象
- 顺序查找算法
 - 从最后一条记录开始，按照记录的逻辑次序，将给定的关键字 k 和当前记录的关键字逐个比较，若某个记录的关键字和给定值相等，则查找成功，返回成功记录的序号；
 - 反之，直到比较完第一条记录，找不到与给定值相等的记录，则查找失败，返回查找失败标记。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



算法分析--1

- 查找类算法元操作——比较运算。
- 平均查找长度（Average Search Length, ASL）是指为为查找关键字，确定记录在查找表中的位置，关键字需和表中元素比较次数的期望值。

对于具有 n 条记录的表，查找成功时的平均检索长度为：

$$ASL = \sum_{i=1}^n p_i c_i$$

P_i 为查找表中第 i 个元素的概率，若查找每个元素的概率相等，有 $P_i=1/n$ 。

C_i 是找到表中关键字与给定值相等时比较的次数，有 $C_i = n-i+1$ 。

对于长度为 n 的线性表，若查找每个元素的概率相等，

查找成功的平均检索长度为：

$$ASL_{ss} = \sum_{i=1}^n P_i C_i = \sum_{i=1}^n \frac{1}{n} (n-i+1) = \frac{1}{n} \sum_{i=1}^n (n-i+1) = \frac{n+1}{2}$$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法分析--2

- 查找结果
 - 成功
 - 不成功
- 假设查找成功和不成功的概率相等，应有 $P_i=1/2n$ 。对于不成功的比较次数均为 $n+1$ 。
- 因此，有：

$$ASL = \frac{1}{2n} \sum_{i=1}^n (n-i+1) + \frac{1}{2} (n+1) = \frac{3}{4} (n+1)$$

↑ ↑
查找成功 查找失败

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

折半查找

- 若文件（或表）中的记录是按照关键字有序的，假定关键字值满足 $K_1 \leq K_2 \leq \dots \leq K_n$
- 折半查找的基本思想
 - 首先选取表的中间元素，设序号为 $m = \lfloor (1+n)/2 \rfloor$ ，元素 R_m 将数据表分成大致相等的两部分 $\{R_1, R_2, \dots, R_{m-1}\}$ 和 $\{R_{m+1}, R_{m+2}, \dots, R_n\}$ 。
先对 K_m 和 K 作比较，比较结果有三种情况：
 - $K = K_m$ ：查找成功，返回元素序号 m 。
 - $K < K_m$ ：折半查找子表 $\{R_1, R_2, \dots, R_{m-1}\}$ 。
 - $K > K_m$ ：折半查找子表 $\{R_{m+1}, R_{m+2}, \dots, R_n\}$ 。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例

• 查找的关键字为 $K=6$ ，折半查找过程

表 3-5 折半查找查找过程

比较第次	数 据 表										比较结果
	K ₁	K ₂	K ₃	K ₄	K ₅	K ₆	K ₇	K ₈	K ₉	K ₁₀	
1	{ 2 , 5 , 6 , 10 , ⑤ , 21 , 26 , 30 , 56 , 78 }										K<K _m
	↑ low		↑ m				↑ high				
2	{ 2 , ⑤ , 6 , 10 , } 15 , 21 , 26 , 30 , 56 , 78 }										K>K _m
	↑ low	↑ m	↑ high								
3	2 , 5 , { ⑥ , 10 , } 15 , 21 , 26 , 30 , 56 , 78 }										K=K _m
	↑↑ low/m		↑ high								

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法分析 (续)

- 查找成功的平均检索长度

假设查找每个元素的概率相等, 则 $P_i=1/n$,

因此折半检索的查找成功的平均检索长度为:

$$\begin{aligned} ASL_{bs} &= \sum_{i=1}^n p_i C_i \\ &= \frac{1}{n} \sum_{j=1}^h j \cdot 2^{j-1} = \frac{n+1}{n} \log_2(n+1) - 1 \\ &\approx \log_2(n+1) - 1 \end{aligned}$$

- 例如: 对于 $n=2^7-1=127$, 顺序查找查找成功的平均检索长度为64, 而折半查找查找成功的平均检索长度为6。

【折半查找 动画演示 完整代码 二分查找 binary search 数据结构与算法】

https://www.bilibili.com/video/BV1Vw4m1Y7NQ/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



3.5 排序问题及排序算法

- 排序问题
- 选择排序
- 交换排序
- 插入排序
- 归并排序
- 基数排序

知识要点

排序，
稳定排序，
内部排序，
选择排序，
交换排序，
插入排序，
归并排序，
基数排序，
外部排序。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



排序问题

- 排序 (Sorting) , 就是把任意文件 (或表) 按照指定的关键字排列成一个有序文件 (或表) 的过程, 有时又称作 “分类” 。
- 例如:
 - 将一堆数由小到大或由大到小排列;
 - 在站队时, 按身高由高到低或由低到高排列等。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

排序问题的抽象

- 假如给定一个记录序列（或表） $F = \{R_1, R_2, \dots, R_n\}$, 其中 R_i ($i = 1, 2, \dots, n$) 为序列中的第 i 个记录, 关键字为 K_i 。所谓排序就是对记录序列中的记录重新排序成一个新的序列 $F' = \{R_1', R_2', \dots, R_n'\}$, 使得它们的关键字值满足非减（或非增）的顺序。即对任意的 $i < j$ ($i, j = 1, 2, \dots, n$), 有 $K_i' \leq K_j'$ （或 $K_i' \geq K_j'$ ）。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

基本概念

- 稳定排序：当初始时序列中 $K_i=K_j$ ，且 $i<j$ 时，排序后记录 R_i 仍然在记录 R_j 前，这样的排序称为稳定排序。
- 内部排序：是指被排序的记录较少，在整个排序过程中，所有的记录和中间结果都放在内存中。
- 外部排序：当被排序的记录数量较大时，记录不能一次全部放在内存中，在整个排序过程中，必须对记录在内存和外存之间来回传递和交换。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



内部排序的分类

- 排序算法的基本操作
 - 比较
 - 移动
- 按照排序效率分类
 - 简单排序, 时间复杂度为 $O(n^2)$
 - 高效排序, 时间复杂度为 $O(n\log_2 n)$
 - 基数排序, 时间复杂度为 $O(d \cdot n)$
- 按照排序过程划分
 - 选择排序
 - 交换排序
 - 插入排序
 - 归并排序
 - 基数排序

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



选择排序

- 基本思想：从被排序的文件（或表）中依次选出关键字最小、次小、...的记录，从而实现排序。
- 根据算法效率
 - 简单选择排序，时间复杂性为 $O(n^2)$
 - 树形选择排序（锦标赛排序）， $O(n\log_2 n)$
 - 堆排序， $O(n\log_2 n)$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



简单选择排序

- 从 $1..n$ 个记录中选出关键字最小的记录，和 R_1 交换，最小的记录放到第1个单元。
- 从 $2..n$ 个记录中选出关键字最小的记录，和 R_2 交换，次小的记录放到第2个单元。
- 依次进行，共需要 $n-1$ 遍，最大的记录留在第 n 个单元。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例

- $F = \{21, 30, 19, 20, 35, 11, 75\}$, $n=7$

- 简单选择排序过程

- 第1次循环 $i=1$, $j=6$, 则 R_1 和 R_6 交换

- 第2次循环 $i=2$, $j=3$, 则 R_2 和 R_3 交换

.....

- 第6次循环 $i=6$,

- 排序结果

$F' = \{11, 19, 20, 21, 30, 35, 75\}$

$i \longrightarrow$	R_1	21
	R_2	30
	R_3	19
	R_4	20
	R_5	35
	R_6	11
	R_7	75

算法分析

- 简单选择排序，共进行 $n-1$ 遍
- 第 i 遍，从 $n-i+1$ 个数中选择一个最小的数。从 $n-i+1$ 个数中选择最小的数需要 $n-i$ 次比较运算
- 因此简单选择排序总的比较次数为 $n(n-1)/2$ 。时间复杂度为 $O(n^2)$ 。

重复（元素个数-1）次

把第一个没有排序过的元素设置为最小值

遍历每个没有排序过的元素

如果元素 $<$ 现在的最小值

将此元素设置成为新的最小值

将最小值和第一个没有排序过的位置交换



交换排序

- 交换类排序（Exchange Sorting）就是将两两元素进行比较，如果发生逆序，即 $R_i > R_j$ ($i < j$)，则将两个元素交换，最后得到一个**非递减**的序列（正序）。
- 交换分类有标准交换、成对交换和穿梭交换三种，比较著名的交换类排序有冒泡排序和快速排序。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

冒泡排序

- 冒泡排序 (Bubbles Sorting) 属于标准交换分类
- 基本思想
 - 第1遍: 首先将 R_n 和 R_{n-1} 进行比较, 若发生逆序, 则交换; 否则, 比较 R_{n-1} 和 R_{n-2} , 直到 R_2 和 R_1 比较。这样, 第一遍结束后, 将把关键值最小的元素移到了第一个单元。最小的元素就像“气泡”一样冒到了顶上, 共比较 $n-1$ 次 (R_n 与前 $n-1$ 个数完成了比较)。
 - 第2遍: 和第1遍一样, 依次将 R_n 和 R_{n-1} 进行比较、 R_{n-1} 和 R_{n-2} , 直到 R_3 和 R_2 比较 (不需要 R_2 和 R_1 比较, 因为上一轮已经确定了最小值)。这样, 第2遍结束后, 将把关键值次小的元素移到了第2个单元。共比较 $n-2$ 次
 - 继续上述过程, 逐遍进行, 在进行 i 遍时, 在前 $i-1$ 遍得到的结果中, $R_n, R_{n-1}, R_{n-2}, \dots, R_{i+1}$ 和 R_i 依次两两比较, 如发生逆序, 则交换位置。

也可以反过来比较, 先将最大的数“冒泡”到顶上。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例

• $F = \{21, 30, 19, 20, 35, 11, 75\}$, $n=7$

• 冒泡排序过程

- 第1遍: $\{11, 21, 30, 19, 20, 35, 75\}$
- 第2遍: $\{11, 19, 21, 30, 20, 35, 75\}$
- 第3遍: $\{11, 19, 20, 21, 30, 35, 75\}$
- 第4遍: 未交换, 退出循环

$i \longrightarrow$

R_1	21
R_2	30
R_3	19
R_4	20
R_5	35
R_6	11
R_7	75

• 排序结果

$F' = \{11, 19, 20, 21, 30, 35, 75\}$

算法分析

- 最好情况下（原始序列为正序，即：

非递减, $R_1 \leq R_2 \leq \dots \leq R_n$)

- 则整个排序共进行一遍，比较 $n-1$ 次。

- 最坏情况下（原始序列为逆序，即：

非递增, $R_1 \geq R_2 \geq \dots \geq R_n$)

- 需要比较 $n-1$ 遍，比较的次数分别是 $n-1, n-2, \dots, 3, 2, 1$,
- 总的比较次数 C 为：

$$\sum_{i=1}^{n-1} (i-1) = n(n-1)/2$$

做

```
swapped = false
```

从 $i = 1$ 到 最后一个没有排序过元素的索引-1

如果 左边元素 > 右边元素

交换（左边元素，右边元素）

```
swapped = true; ++swapCounter (交换计数器)
```

```
while swapped
```

插入排序

- 插入排序 (Insert Sorting) , 是指将一个记录插入到一个已经排序好的有序序列中, 从而得到一个新的、记录个数加1的有序序列。
- 基本思想
 - (1) 初始序列为 $\{R_1\}$, 只有一个元素的序列一定是有序的。
 - (2) 依次将 $R_2, R_3, \dots, R_n$ 插入到上次的有序序列中, 分别得到长度为 $2, 3, \dots, n$ 的有序序列, 从而实现长度为 n 的记录序列的排序。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例

- 输入顺序: 21, 30, 19, 20, 35, 11, 75

- 插入排序过程

- 第1遍: {21, 30, 19, 20, 35, 11, 75}

- 第2遍: {21, 30, 19, 20, 35, 11, 75}

- 第3遍: {19, 21, 30, 20, 35, 11, 75}

- 第4遍: {19, 20, 21, 30, 35, 11, 75}

- 第5遍: {19, 20, 21, 30, 35, 11, 75}

- 第6遍: {11, 19, 20, 21, 30, 35, 75}

- 第7遍: {11, 19, 20, 21, 30, 35, 75}

- 排序结果

$F' = \{11, 19, 20, 21, 30, 35, 75\}$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例

- 输入顺序: 21, 30, 19, 20, 35, 11, 75

- 插入排序过程

- 第1遍: {21}
- 第2遍: {21, 30}
- 第3遍: {19, 21, 30}
- 第4遍: {19, 20, 21, 30}
- 第5遍: {19, 20, 21, 30, 35}
- 第6遍: {11, 19, 20, 21, 30, 35}
- 第7遍: {11, 19, 20, 21, 30, 35, 75}

- 排序结果

$F' = \{11, 19, 20, 21, 30, 35, 75\}$

算法分析

- 最好情况下（原始序列为正序，即：

非递减, $R_1 \leq R_2 \leq \dots \leq R_n$)

- 则整个排序共进行一遍，比较 $n-1$ 次。

- 最坏情况下（原始序列为逆序，即：

非递增, $R_1 \geq R_2 \geq \dots \geq R_n$)

- 需要比较 $n-1$ 遍，比较的次数分别是 $n-1, n-2, \dots, 3, 2, 1$,
- 总的比较次数 C 为: $\frac{n(n-1)}{2}$
- 需要进行 $n-1$ 次插入操作

将第一个元素标记为已排序

对于每一个未排序的元素 x

“提取” 元素 x

i = 最后排序过元素的索引 到 0 的遍历

如果当前元素 $j > x$

将排序过的元素向右移一格

跳出循环并在此插入 x



快速排序

- 快速排序（Quick Sort）方法是由英国计算机科学家霍尔（Charles Antony Richard Hoare，1934年1月11日～）于1962年提出的，后来又有许多人提出了修正方案，这类方法统称为快速排序（分类）。
- 基本思想：分治法
 - 按照一定原则选择某个元素作为控制记录（轴元素），首先把控制元素移动到其正确的位置，使得元素序列中所有比它小的元素都在它的前面，所有比它大的元素都在它的后面。
 - 按照同样的原则处理左右两个子序列，控制记录不再移动位置。
- 时间复杂性
 - 最坏情况： $O(n^2)$
 - 最好情况： $O(n\log_2 n)$

【【TED-ed】快速排序是什么!】

https://www.bilibili.com/video/BV1irPMMeQEcq/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

快速排序

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

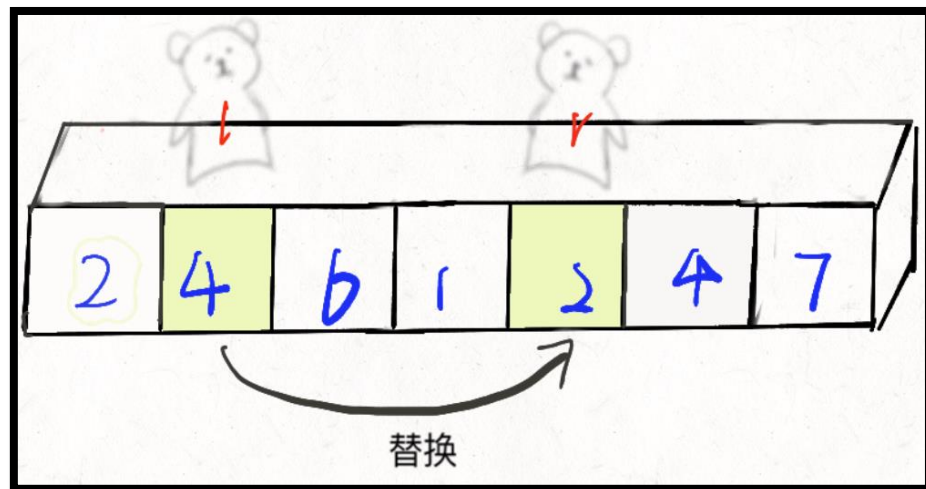
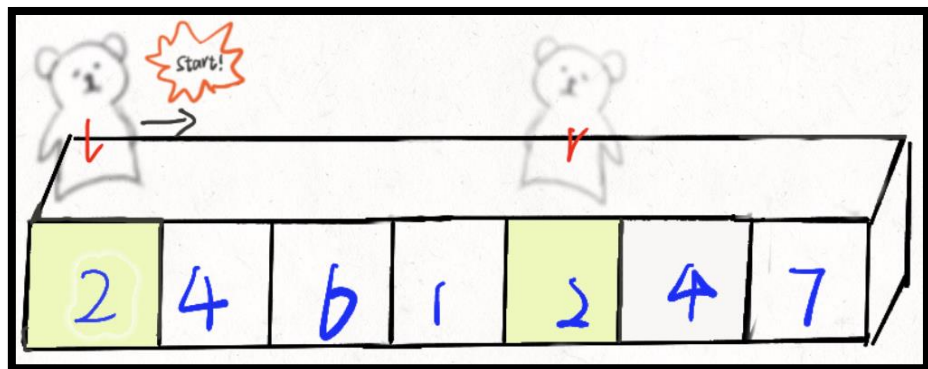
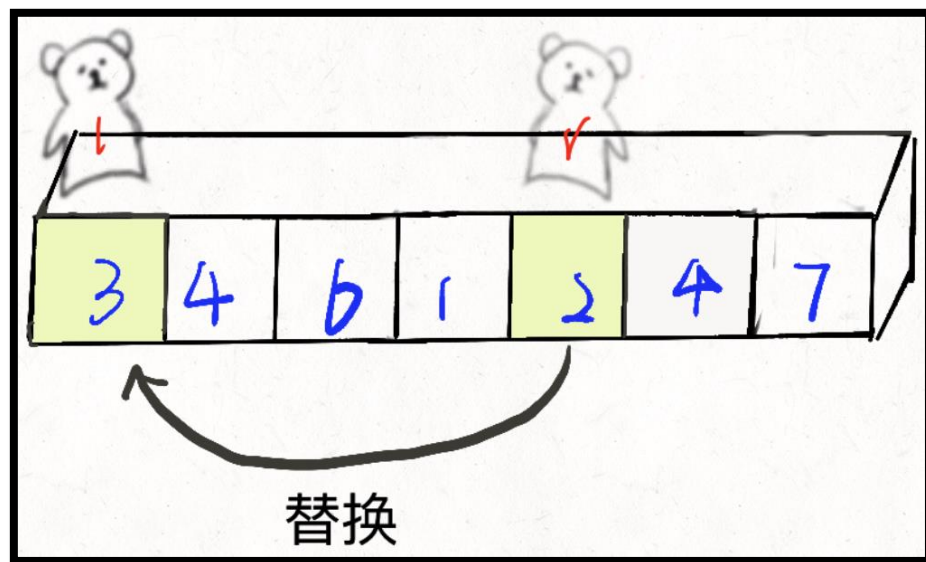
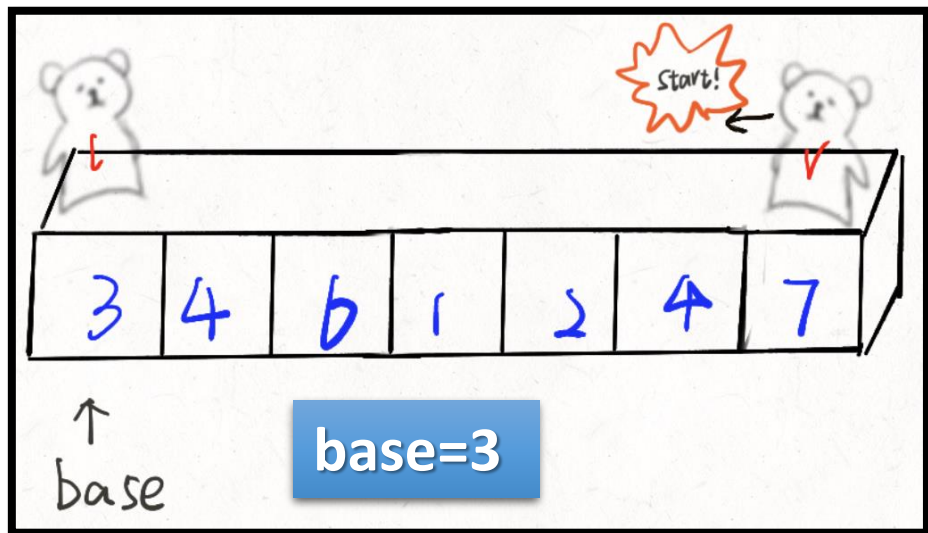
3.3 算法设计及算法分类

二、常见算法

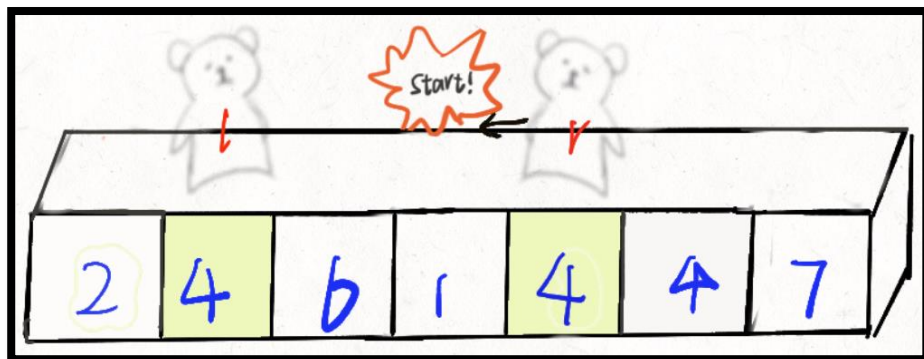
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

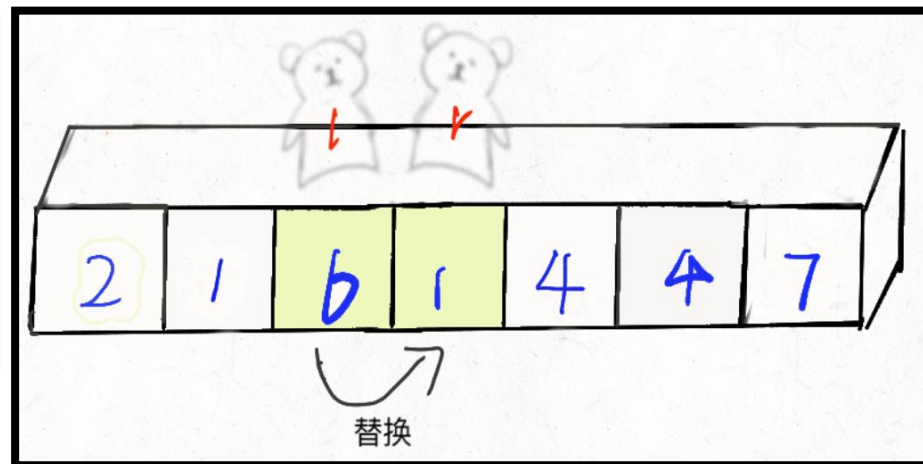
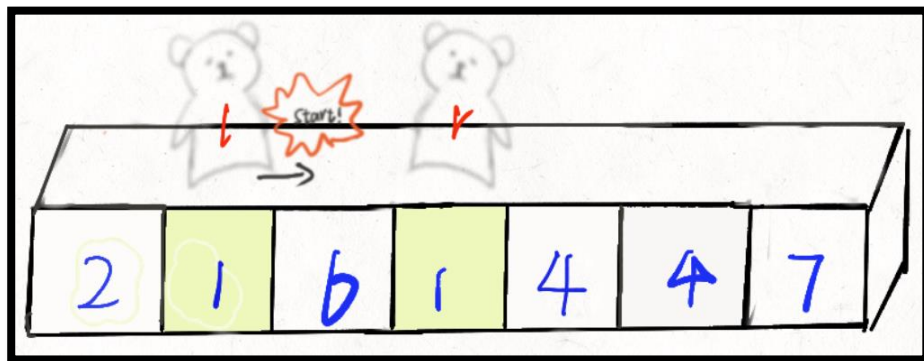
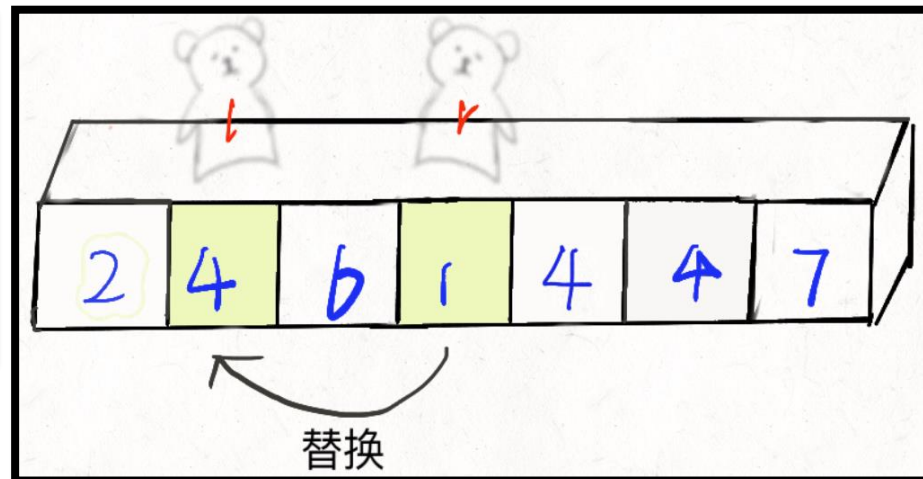
3.6 网络搜索问题



快速排序



base=3



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

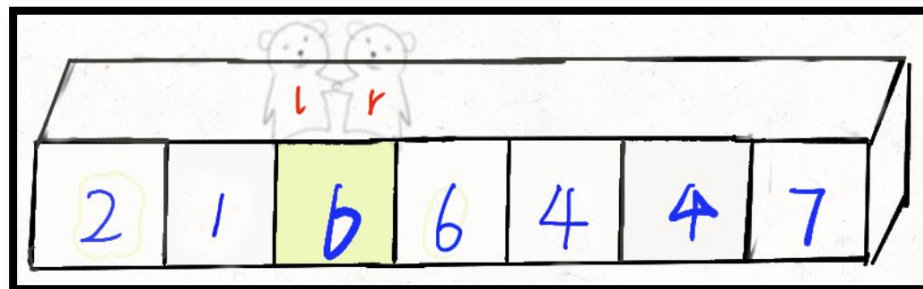
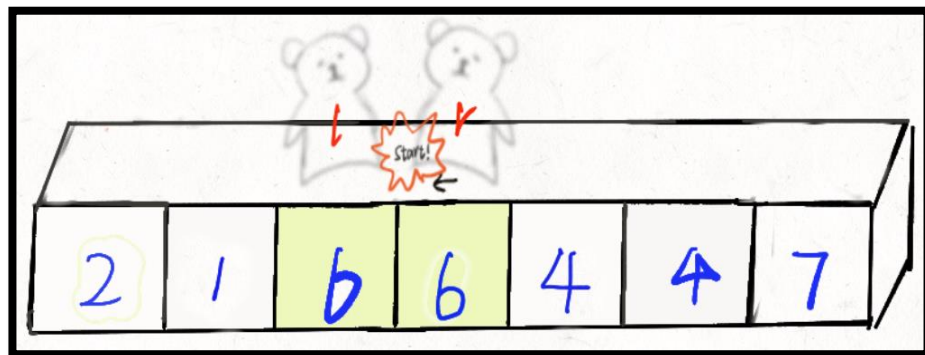
二、常见算法

3.4 搜索问题与查找算法

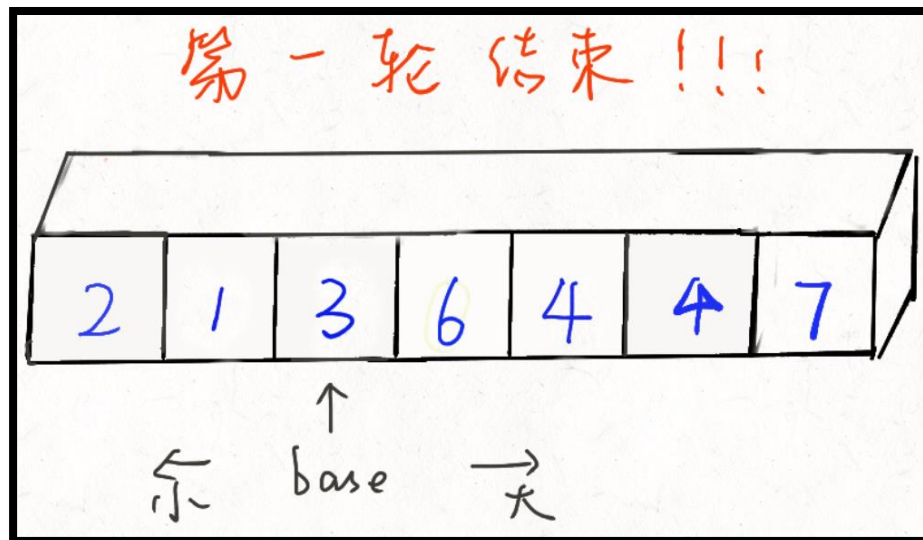
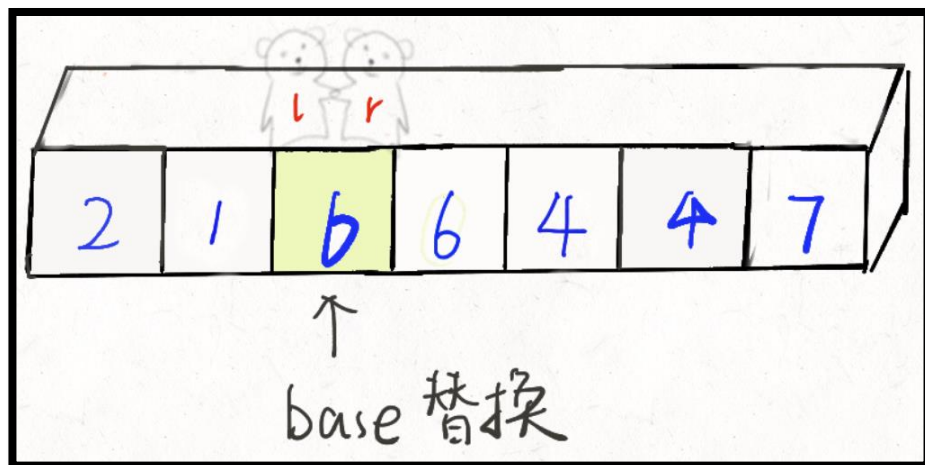
3.5 排序问题及排序算法

3.6 网络搜索问题

快速排序



base=3



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

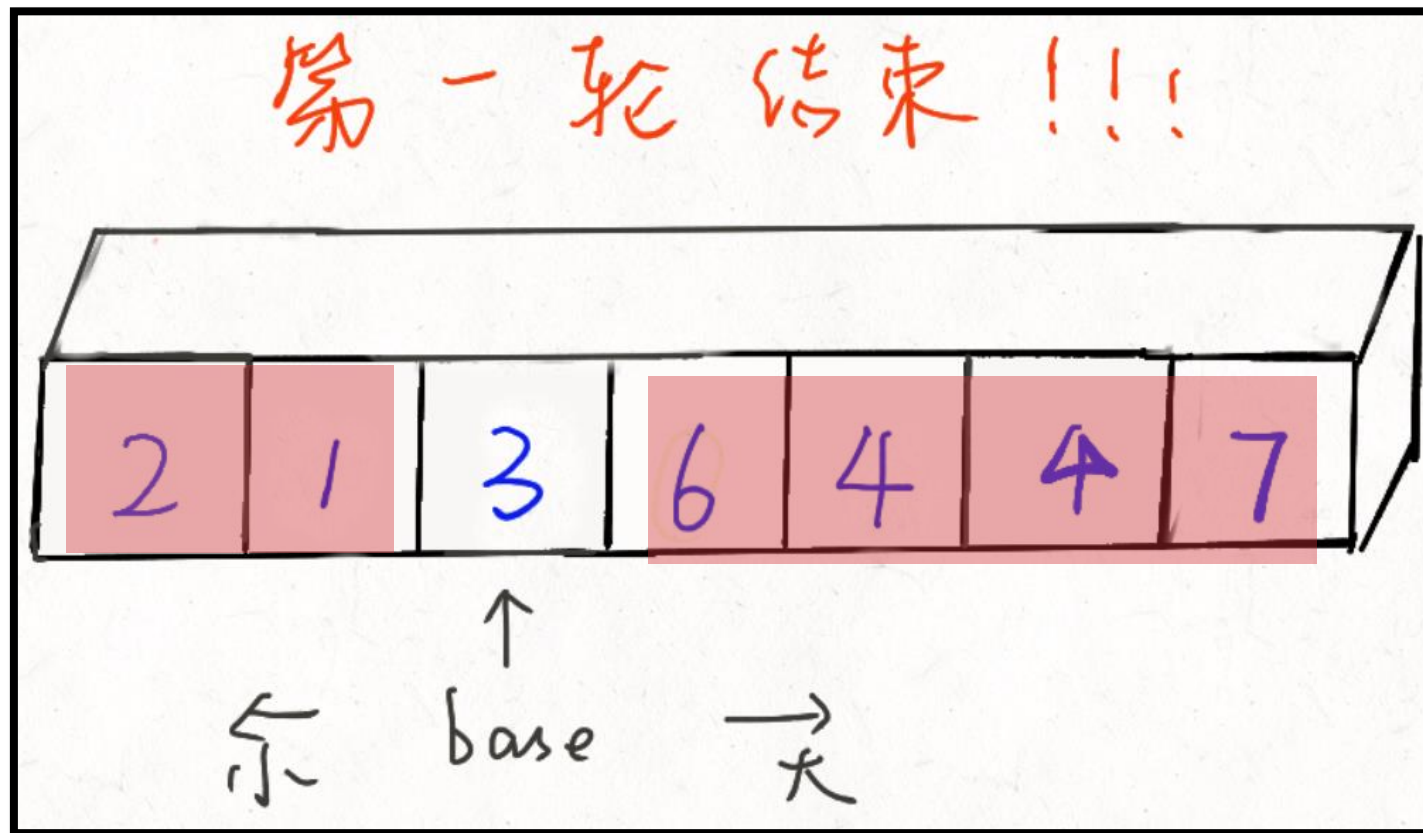
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

快速排序



分别对序列[2,1]和序列[6,4,4,7]进行新一轮排序

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

快速排序 (另一个例子)

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

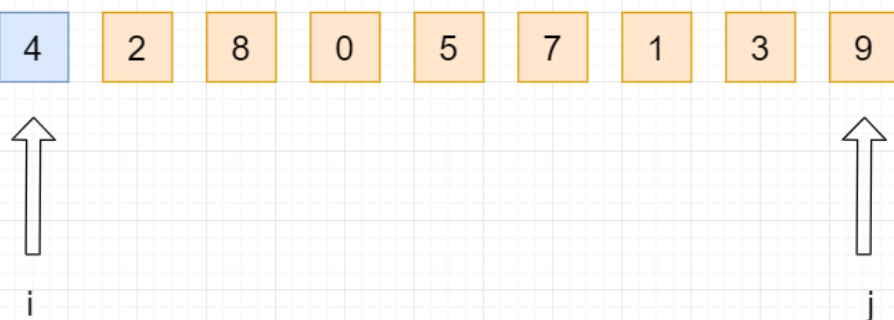
二、常见算法

3.4 搜索问题与查找算法

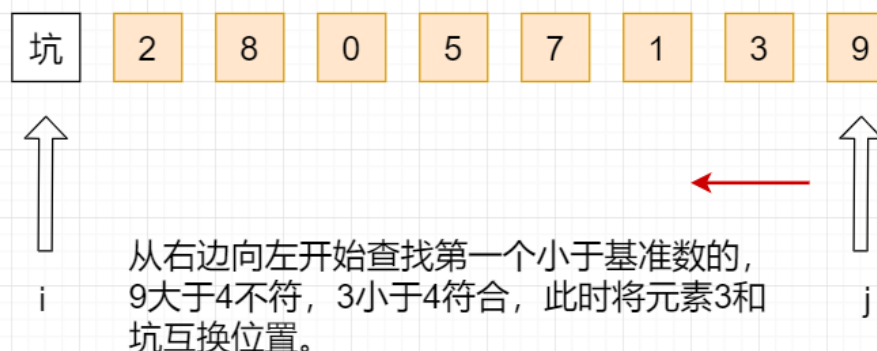
3.5 排序问题及排序算法

3.6 网络搜索问题

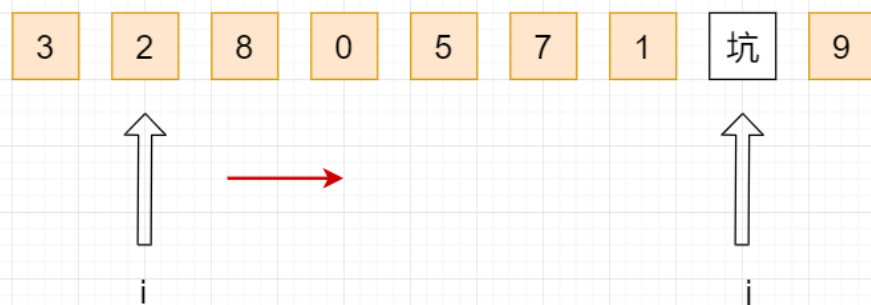
基准数: $arr[0]=4$



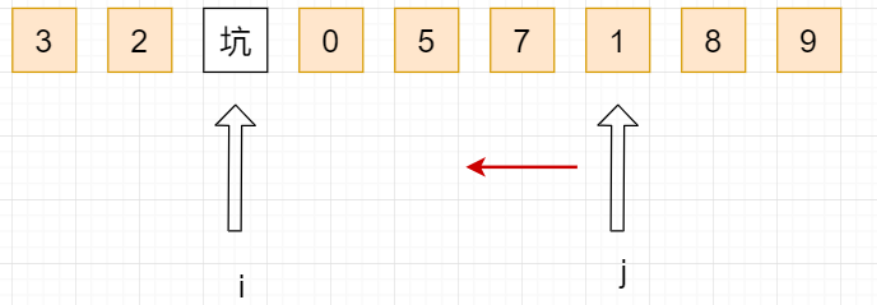
基准数: $arr[0]=4$



基准数: $arr[0]=4$



基准数: $arr[0]=4$



此时元素3与坑互换，这时候我们要从数组的左边向右边开始查找比基准数4大的元素，2比4小不符，8比4大符合，此时再将元素8与坑互换位置。

此时元素8与坑互换，重新从数组的右边向左开始查找比基准数4小的元素，1比4小符合，此时将元素1与坑互换位置，不断重复此操作。

快速排序 (另一个例子)

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

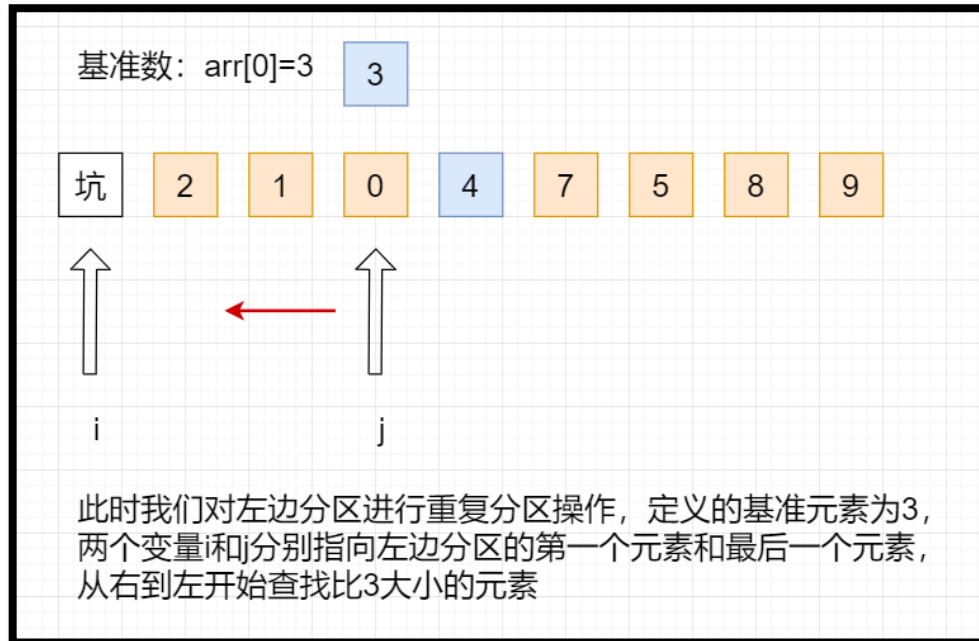
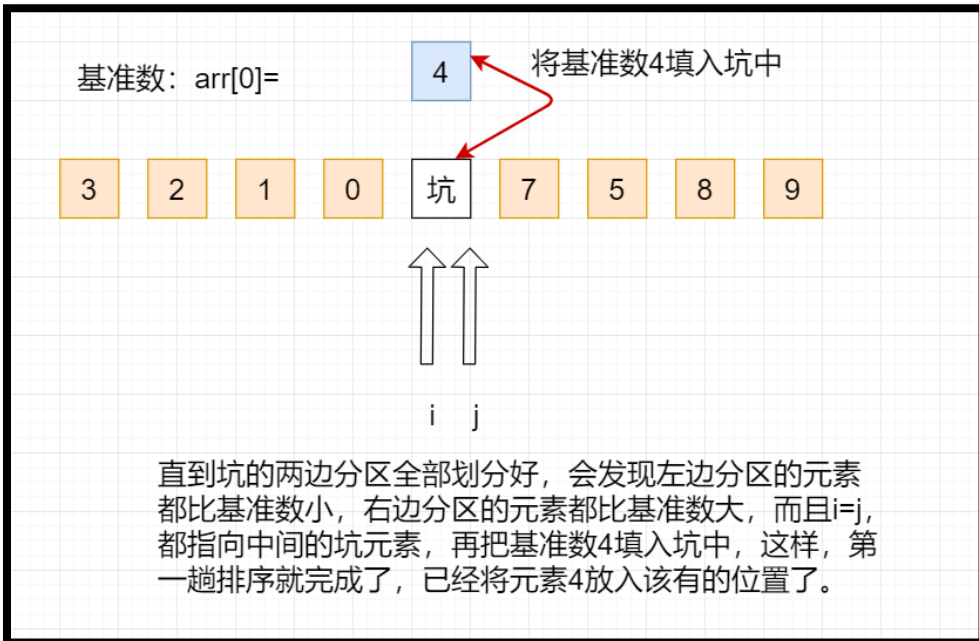
3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



重复执行以上操作, 直到左右两边都是有序列表, 就完成了排序。

快速排序（选最右边作为base）

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

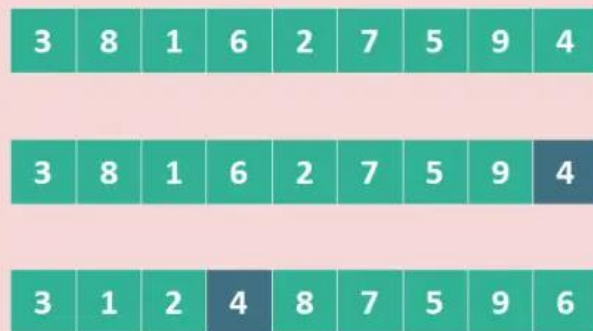
二、常见算法

3.4 搜索问题与查找算法

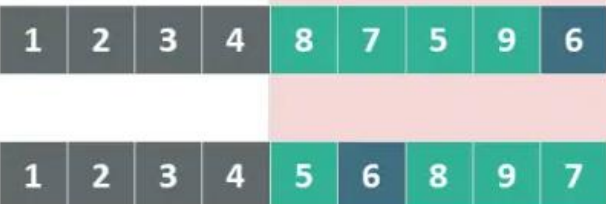
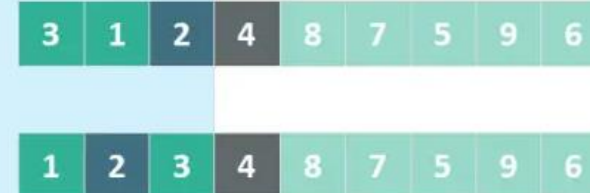
3.5 排序问题及排序算法

3.6 网络搜索问题

开始递归
执行排序划分



递归
第二次排序划分



递归
第三次排序划分



递归
第四次排序划分



递归
第五次排序划分



递归结束

快速排序算法分析

每个（未排序）的部分

将第一个元素设为 pivot

存储索引 = pivot索引 + 1

从 i=pivot索引 + 1 到 最右索引 的遍历

如果 $a[i] < a[\text{pivot}]$

交换 (i, 存储索引); 存储索引++;

交换(pivot, 存储索引 - 1)

最好情况 (Best Case): 比较次数最少

• **发生条件:** 每次分区时, 选定的基准值恰好是当前子数组的**中位数**。这意味着每次都能将数组**完美地一分为二**, 左右两部分的大小几乎相等 (最多相差1个元素)。

• 比较次数分析:

- **每层的比较次数:** 在递归的每一层, 所有子问题的总和就是对整个数组进行一次分区。要将 n 个元素划分为两部分, 基准值需要与**其他 $n-1$ 个元素**各比较一次。因此, 每一层的总比较次数是 $n-1$ 。
- **递归深度:** 由于每次都将问题规模减半, 递归的深度 (树的高度) 是 $\log_2 n$ 。
- **总比较次数:** 总比较次数 $\approx$ 每层比较次数 $\times$ 层数 $= (n-1) \times \log_2 n$ 。
 - 当 n 很大时, $n-1 \approx n$, 所以总比较次数约为 $n \log_2 n$ 。

3.3 异/法设计以及异/法刀
类

二、常见算法

3.4 搜索问题与查找算
法

3.5 排序问题及排序算
法

3.6 网络搜索问题

快速排序算法分析

最好情况 (Best Case): 比较次数最少

[例]

假设数组为 [4,1,3,2,6,5,7]，每次都选到中位数作为基准。

- 第一次选 4 为基准，与其余6个元素比较，划分为 [1,3,2] 和 [6,5,7]。**比较6次。**
- 第二次，对 [1,3,2] 选 2 为基准，比较2次；对 [6,5,7] 选 6 为基准，比较2次。**共比较4次。**
- 第三次，处理的子数组长度都 ≤ 2 ，很快完成。**比较次数很少。** 总比较次数远小于 $7^2=49$ ，接近 $7 * \log_2 7 \approx 7 * 2.8 = 19.6$ 。

每个（未排序）的部分

将第一个元素设为 pivot

存储索引 = pivot索引 +1

从 i=pivot指数 +1 到 最右索引 的遍历

如果 $a[i] < a[\text{pivot}]$

交换 (i, 存储索引); 存储索引++;

交换(pivot, 存储索引 - 1)

3.3 异/法设计以及异/法刀
类

二、常见算法

3.4 搜索问题与查找算
法

3.5 排序问题及排序算
法

3.6 网络搜索问题

快速排序算法分析

每个（未排序）的部分

将第一个元素设为 pivot

存储索引 = pivot索引 + 1

从 i=pivot索引 + 1 到 最右索引 的遍历

如果 $a[i] < a[\text{pivot}]$

交换 (i, 存储索引); 存储索引++;

交换(pivot, 存储索引 - 1)

最坏情况 (Worst Case): 比较次数最多

- **发生条件:** 每次分区时, 选定的基准值都是当前子数组的**最大值或最小值**。这通常发生在**数组已经完全有序或逆序**, 且我们总是选择第一个或最后一个元素作为基准时。
- **比较次数分析:**
 - **第一层:** 对 n 个元素进行分区, 基准值需要与 $n-1$ 个元素比较。比较 $n-1$ 次。
 - **第二层:** 一个子数组有 $n-1$ 个元素 (另一个为空), 需要比较 $n-2$ 次。
 - ...
 - **最后一层:** 比较 1 次。
- **总比较次数:** 这是一个等差数列求和: $(n-1) + (n-2) + (n-3) + \dots + 1 = (n-1) * n / 2$ 这等于 $n^2/2 - n/2$ 。

3.3 异/法设计以及异/法万类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

快速排序算法分析

最坏情况 (Worst Case): 比较次数最多

[例]

假设待排序序列为 [1,2,3,4,5], 选首元素为基准:

- 基准 1, 与 2,3,4,5 比较4次 $\rightarrow$ [1], [2,3,4,5]
- 基准 2, 与 3,4,5 比较3次 $\rightarrow$ [2], [3,4,5]
- 基准 3, 与 4,5 比较2次 $\rightarrow$ [3], [4,5]
- 基准 4, 与 5 比较1次 $\rightarrow$ [4], [5] **总比较次数** $= 4+3+2+1 = 10$, 而 $(5*4)/2 = 10$ 。

每个 (未排序) 的部分

将第一个元素设为 pivot

存储索引 = pivot索引 +1

从 i=pivot指数 +1 到 最右索引 的遍历

如果 $a[i] < a[pivot]$

交换 (i, 存储索引); 存储索引++;

交换 (pivot, 存储索引 - 1)

3.5 异/去设计以及异/去刀
类

二、常见算法

3.4 搜索问题与查找算
法

3.5 排序问题及排序算
法

3.6 网络搜索问题



快速排序与冒泡排序对比

冒泡、选择、插入排序：解决“局部冲突”

- 它们的比较大多是为了调整两个元素的相对顺序。
- 信息价值低：一次比较只能告诉你“A和B谁大谁小”，无法直接推断出A或B与其他未参与比较的元素之间的关系。
- 例子：在 $[5, 4, 3, 2, 1]$ 中，比较 $5 > 4$ 后交换，得到 $[4, 5, 3, 2, 1]$ 。这个结果对3、2、1的位置没有提供任何新信息。5还需要一步步“冒泡”到末尾。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

快速排序与冒泡排序对比

快速排序：实现“全局划分”

- 它通过一个基准值（pivot），一次性与数组中的多个（通常是所有）其他元素进行比较。
- 信息价值高：每次与基准的比较，都能确定该元素是属于“小集合”还是“大集合”。这相当于用一次操作，就把整个数组按大小关系划分为两个独立区域。
- 例子：同样是 $[5, 4, 3, 2, 1]$ ，选 3 为基准：

$5 > 3 \rightarrow$ 知道 5 属于“大集合”

$4 > 3 \rightarrow$ 知道 4 属于“大集合”

$2 < 3 \rightarrow$ 知道 2 属于“小集合”

$1 < 3 \rightarrow$ 知道 1 属于“小集合”

经过 4 次比较，我们不仅把 3 放到了正确位置，还知道了 $\{2, 1\}$ 都小于 3， $\{4, 5\}$ 都大于 3。后续只需分别处理这两个子集，它们之间再也不需要互相比较，减少了比较次数。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

快速排序与冒泡排序对比

问题规模的缩减速度：指数级 vs 线性级

冒泡排序：

- 每完成一轮外层循环，通常只能确保**一个**元素到达最终位置。
- 问题规模从 n 减到 $n-1$ ，再减到 $n-2$ 是**线性缩减**。
- 因此，总比较次数是 $n + (n-1) + (n-2) + \dots + 1 \approx n^2/2$ 。

快速排序：

- 在最好/平均情况下，每次分区都将问题**一分为二**。
- 问题规模从 $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots$ 是**指数级缩减**。
- 只需要 $\log n$ 层递归就能将问题分解完毕。
- 每层总共需要 $O(n)$ 次比较（所有子问题加起来扫描一遍当前层的所有元素）。
- 总比较次数 = $O(n) \times O(\log n) = O(n \log n)$ 。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



归并排序

- 把两个或多个有序文件（或表）合并成一个有序文件（或表）的过程称归并（Merge）。当归并文件有 k 个时，称为 k 路归并
- 归并排序（Merge Sorting）——重复利用归并思想对任意文件（或表）进行排序的过程

【【排序算法精华2】归并排序】

https://www.bilibili.com/video/BV1Ma4y1n7E2/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

二路归并排序

- (1) 将文件（或表） $F=\{R_1, R_2, \dots, R_n\}$ 中的每一个记录看作一个文件（或表），它们都是有序的（仅含一个记录）。
- (2) 把子文件（或子表）按照相邻的位置分成若干对（如果文件或子表个数是奇数，最后一个单独一组）。
- (3) 对每对中的子文件（或子表）进行二路归并。归并后，每个子文件都是有序的，且子文件的个数减少一半。
- (4) 重复步骤 (2) ~ (4)，直到归并成一个有序文件为止。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例

• 2路归并过程

初始序列

(25) (30) (80) (11) (76) (58) (65) (98)

一趟归并后

(25,30) (11,80) (58,76) (65,98)

二趟归并后

(11,25,30,80) (58,65,76,98)

三趟归并后

(11,25,30,58,76,80,98)

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

基数排序

- 基数排序是一种借助于**多关键字**排序的思想实现的
对单个逻辑关键字进行排序的方法。
- 多关键字，例如

花 色：梅花 (♣) < 方块 (♦) < 红桃 (♥) < 黑桃 (♠)

牌面值：2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A

因此按照从小到大的顺序，扑克牌的排列次序为：

2	3		A	2	3		A	2	3		A	2	3		A
♣	♣	...	♣	♦	♦	...	♦	♥	♥	...	♥	♠	♠	...	♠

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



十进制基数分类

- 假定被分类的关键字值是十进制整数，将每位数字视为一个关键字，个位为最低关键字，十位为次低关键字，以此类推。
- 进行十进位基数分类的过程是：
 - 把输出分成10个桶（0桶，1桶，...，9桶），
 - 整个分类过程分成 d 遍（ d 为被分类数字的最多位数）。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



分类过程

- 第一遍：首先对最低关键字（个位）、进行桶分类，对序列中的每一个数由前向后将最低位数字相同的数字依次放在对应的上述10个桶中（如个位为6的放在6桶中），直到所有数字都分完。
- 第二遍：把上一遍各桶内的数字，按照从0桶,1桶, ..., 9桶的编号次序，同一桶内按由上倒下的次序收集起来，作为第二遍的输入，再按次关键字（十位）的状态把各个数分别放入0,1,2,...,9对应的桶内。
- 然后再把各个桶内的数字收集起来作为第三遍的输入，继续上述过程，直到按照最高关键字的状态把所有数再分到对应的桶内。
- 最后，按照0桶, 1桶, 2桶, ..., 9桶的顺序把各桶内的数据收集起来（同一桶内按由上倒下的次序收集）就得到所要的结果。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例（续）

- 收集第一遍桶分类结果{80, 11, 52, 34, 26, 56, 86, 68}

表 3-8 十进制基数分类示例（第二遍结果）

桶	0	1	2	3	4	5	6	7	8	9
输入表										
80		11	26	34		52	68		80	
11						56			86	
52										
34										
26										
56										
86										
68										

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

排序算法总结

算法名称	算法思想	平均时间复杂度	最好时间复杂度	最差时间复杂度	稳定性
选择排序	从待排序列表中选最小的元素与第一个未排序元素交换。	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
冒泡排序	从左到右两两比较和交换，从而将最大的元素放到最右边，在剩下的元素中重复执行上述操作，直到都排好序。	$O(n^2)$	$O(n)$	$O(n^2)$	Yes
插入排序	将无序列表中的元素依次插入到有序列表中。	$O(n^2)$	$O(n)$	$O(n^2)$	Yes
快速排序	选择一个基准元素，然后重新排列数组，使得所有小于基准的元素都移到基准的左边，所有大于基准的元素都移到基准的右边。	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
归并排序	对半拆分，分别排序，然后合并。	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
基数排序	借助于多关键字排序的思想实现的对单个逻辑关键字进行排序。	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	Yes

<https://visualgo.net/zh/sorting>

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



趣味排序算法

意大利面条排序 (Spaghetti sort)

把输入的 n 个数对应成 n 根不同长度的面条，其中每根面条的长度对应于每个数的大小。例如对于 $[4, 2, 3, 5, 9]$ 这样一组数，就做出长度分别为4cm, 2cm, 3cm, 5cm, 9cm的意大利面条。然后用手把这些长度不一的面条一头抓住，悬空在桌子上方。然后把手慢慢垂直下降，第一个触碰到桌面的面条对应的数，就是最大的数；第二个触碰到的，就是第二大的数。

睡眠排序 (Sleep sort)

睡眠排序可以认为是上面意大利面条排序的计算机实现版。对于数组中的每个数，都对应一个会睡眠这么长时间的线程。例如对于 $[4, 2, 3, 5, 9]$ 这样一组数，就创建5个线程，每个线程睡眠4s, 2s, 3s, 5s, 9s。这些线程睡醒之后，就把自己对应的数报出来即可。有的人觉得睡眠排序可能效率不高，其实可以将每个线程睡眠的时间单位设的尽可能小一些来加速。但是真正实现起来，多线程的诸多问题让这个算法基本上只是一个概念。

【人类奇葩排序算法鉴赏】

https://www.bilibili.com/video/BV1nJBTYoEQm/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



3.6 网络搜索问题

- 搜索引擎及其工作原理
 - 搜索引擎的概念及分类
 - 全文搜索引擎工作原理
 - 网页索引数据库
- PageRank排序算法
- 搜索引擎的启示

知识要点

搜索引擎，
PageRank算法，
网页重要性度量，
正向链接，
反向链接。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

搜索引擎的概念

- 搜索引擎产生的背景
 - 20世纪90年代，互联网发展开始进入民用阶段，Internet上信息资源及服务器数量急剧增加
 - 普通网络用户想找到自己所需的信息简直如同大海捞针
- 搜索引擎（Search Engine）是指根据一定的策略、运用特定的计算机程序从互联网上搜集网页、图片、ftp文件等各类信息，在对信息进行组织和处理后，为用户提供检索服务的系统。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

搜索引擎的产生

- 1990年，McGill University学生Alan Emtage等人发明Archie（档案检索系统），被称为搜索引擎的鼻祖。
 - WorldWideWeb尚未出现，但网络中文件传输还是相当频繁的，由于大量的文件散布在各个分散的FTP主机中，查询起来非常不便，因此Alan Emtage等想到了开发一个可以用文件名查找文件的系统，于是便有了Archie。
 - Archie是第一个自动索引互联网上匿名FTP网站文件的程序，用户输入精确的文件名，Archie会告诉用户哪一个FTP地址可以下载该文件。它只是一个可搜索的FTP文件名列表，还不是真正的搜索引擎。
- 1993年，在万维网出现后，第一个“蜘蛛”程序被麻省理工学院学生Matthew Gray开发出来，现代意义上的搜索引擎开始初露端倪
 - 开始它只用来统计互联网上的服务器数量，
 - 后来发展为能够捕获网址（URL）

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

Archie Query Form

Search for:

Database: ☐ Worldwide Anonymous FTP ☒ Polish Web Index
Search Type: ☒ Sub String ☐ Exact ☐ Regular Expression
Case: ☒ Insensitive ☐ Sensitive

Do you want to look up strings only (no sites returned):
☒ NO ☐ YES

Output Format For Web Index Search: ☐ Keywords Only
☒ Excerpts Only
☐ Links Only

搜索引擎的发展--1第一代搜索引擎

- 1994年4月，美国斯坦福大学电机工程系博士生大卫·费罗（David Filo）和杨致远（Jerry Yang），共同创办超级目录索引Yahoo!，建立了自己的网络指南信息库。分类目录包括：政治、经济、文化、科技、房地产、教育、艺术、娱乐、体育等各方面
- 目录（Directory）搜索引擎，或主题查询型搜索引擎，按照主题建立分类索引，形成一个树形分类体系结构
- 2002年10月9日，雅虎放弃了网站目录搜索，改为默认Google的搜索结果，标志着第一代搜索引擎的寿终正寝

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

YAHOO! DIRECTORY

☐ Web | ☒ Directory | ☐ Category

Search



Love the DIRECTORY?
Add it to Search!

GO!

Internet Marketing and Advertising Services

[Email this page](#)

[Directory](#) > [Business and Economy](#) > [Business to Business](#) > [Marketing and Advertising](#) > [Internet](#)

CATEGORIES [\(What's This?\)](#)

- [Advertising](#) (141)
- [Direct Email@](#)
- [Internet Marketing Books@](#)
- [Market Research](#) (57)
- [Promotion](#) (850) **NEW!**
- [Software](#) (28)
- [Web and Internet Marketing and Advertising Information@](#)
- [Web Site Affiliate Programs@](#)



搜索引擎的发展--2第二代搜索引擎

- 第一代搜索引擎的问题
 - 人工分类整理搜索目录，网页数量剧增，人工根本无法处理。
 - 只能针对工作人员输入的关键词和描述进行搜索，无法针对网页内容进行搜索
- 使用蜘蛛程序取代人工分类和输入关键词、描述的方式，自动在网络上寻找资料，并自动将取得的结果存入资料库中

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

第二代搜索引擎的代表

- 谷歌（Google）搜索引擎
 - 1998年9月7日，美国斯坦福大学博士生拉里·佩奇（LarryPage）和赛吉·布林（SergeyBrin）创立谷歌（Google）。
 - 1999年6月，通过站点www.google.com开始向互联网用户提供搜索服务，为信息内容供应商（综合性或专业性网站）提供联合品牌的网络搜索解决方案。
 - Google推出PageRank、动态摘要、网页快照等功能革新，改变了搜索引擎的定义。
- 2000年1月，百度（Baidu）在北京中关村成立
- 2004年，雅虎结束了与Google的合作，推出了自己的第二代搜索引擎，重新进入搜索市场。在国内，百度逐渐取代新浪、搜狐，成为最主要的中文第二代搜索引擎。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



搜索引擎的分类

- 分类目录
- 全文索引
- 元搜索引擎
- 垂直搜索引擎
- 集合式搜索引擎
- 门户搜索引擎
- 免费链接列表

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



搜索引擎--1分类目录

- 分类检索、目录索引等，是因特网上最早提供WWW资源查询的服务
- 通过搜集和整理因特网的资源，根据搜索到网页的内容，将其网址分配到相关分类主题目录的不同层次的类目之下，形成像图书馆目录一样的分类树形结构索引。
- 目录索引无需输入检索信息，只要根据网站提供的主题分类目录，层层点击进入，便可查到所需的网络信息资源。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

搜索引擎--2全文搜索引擎

- 拥有自己的检索程序 (Indexer) , 俗称 “蜘蛛” (Spider) 程序或 “机器人” (Robot) 程序, 自建网页数据库, 搜索结果直接从自身的数据库中调用。如: Google和百度
- 租用其他搜索引擎的数据库, 并按自定的格式排列搜索结果, 如Lycos搜索引擎
- 自动信息搜集
 - 定期搜索, Google一般是28天, 以更新自建的网页数据库
 - 提交网站搜索, 即网站拥有者主动向搜索引擎提交网址

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

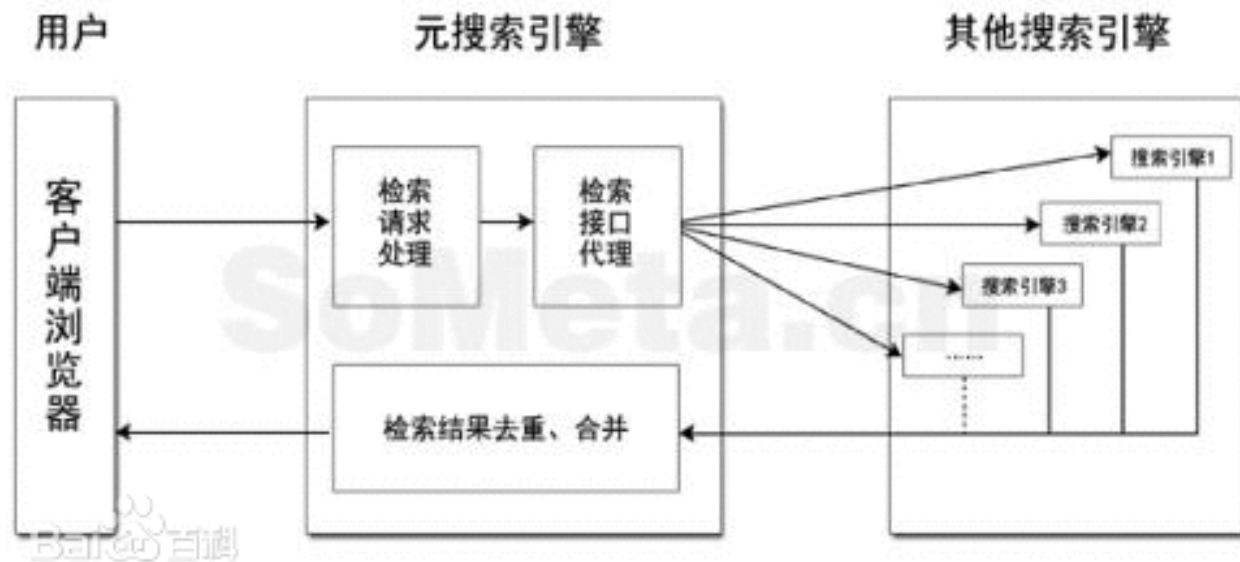
3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

搜索引擎--3元搜索引擎

- 1995年出现元搜索引擎（Meta Search Engine），本身并不直接面向用户提供服务，而是向其他搜索引擎提供全文网页搜索服务
- 元搜索引擎就是通过一个统一的用户界面帮助用户在多个搜索引擎中选择和利用合适的（甚至是同时利用若干个）搜索引擎来实现检索操作，将结果返回给用户。



图片来源：百度百科

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

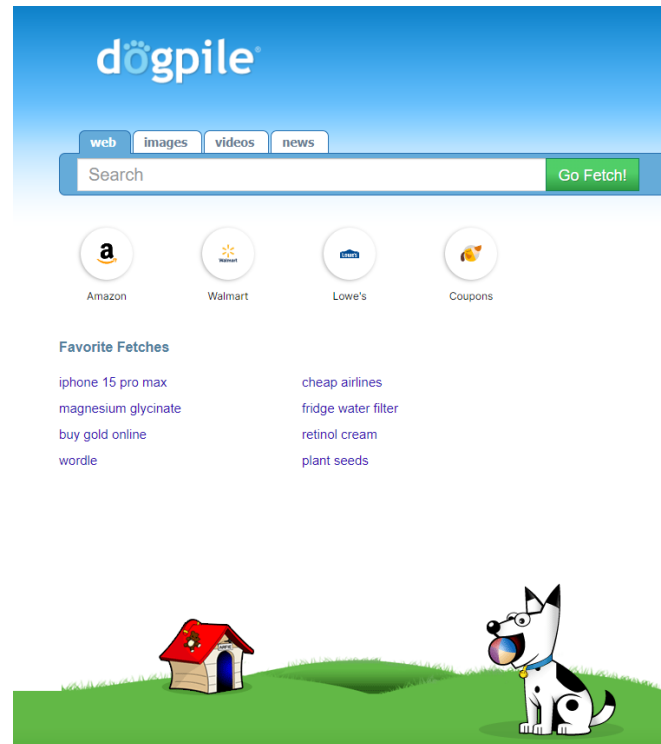
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

- Dogpile是一个聚合谷歌、雅虎、Yandex和其他流行的搜索引擎结果的元搜索引擎。它于1996年11月开始运作。





搜索引擎—4其它搜索引擎

- 垂直搜索引擎
 - 专注于特定的搜索领域和搜索需求（例如：房源搜索，机票搜索、旅游搜索、生活搜索、小说搜索、视频搜索等）
 - 垂直搜索需要的硬件成本低
- 集合式搜索引擎
 - 类似元搜索引擎，区别在于它并非同时调用多个搜索引擎进行搜索，而是由用户从提供的若干搜索引擎中选择
- 门户搜索引擎
 - 自身既没有分类目录也没有网页数据库，其搜索结果完全来自其他搜索引擎的一类搜索。这些搜索引擎通常提供分类目录、链接集合、新闻、社区论坛等功能，以帮助用户在互联网上找到他们感兴趣的内容。
- 免费链接列表（Free For All Links, FFA），指一般只简单地滚动链接条目，部分有简单的分类目录，规模要比Yahoo! 等目录索引小很多。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

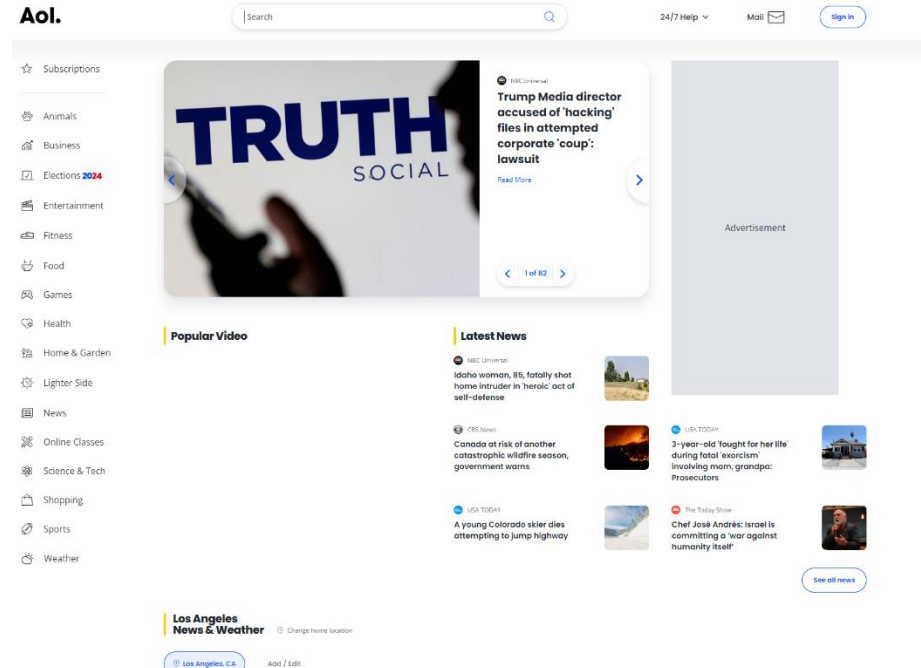
3.5 排序问题及排序算法

3.6 网络搜索问题

搜索引擎—4其它搜索引擎



垂直搜索引擎：携程网



门户搜索引擎：AOL美国在线

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

全文搜索引擎工作原理

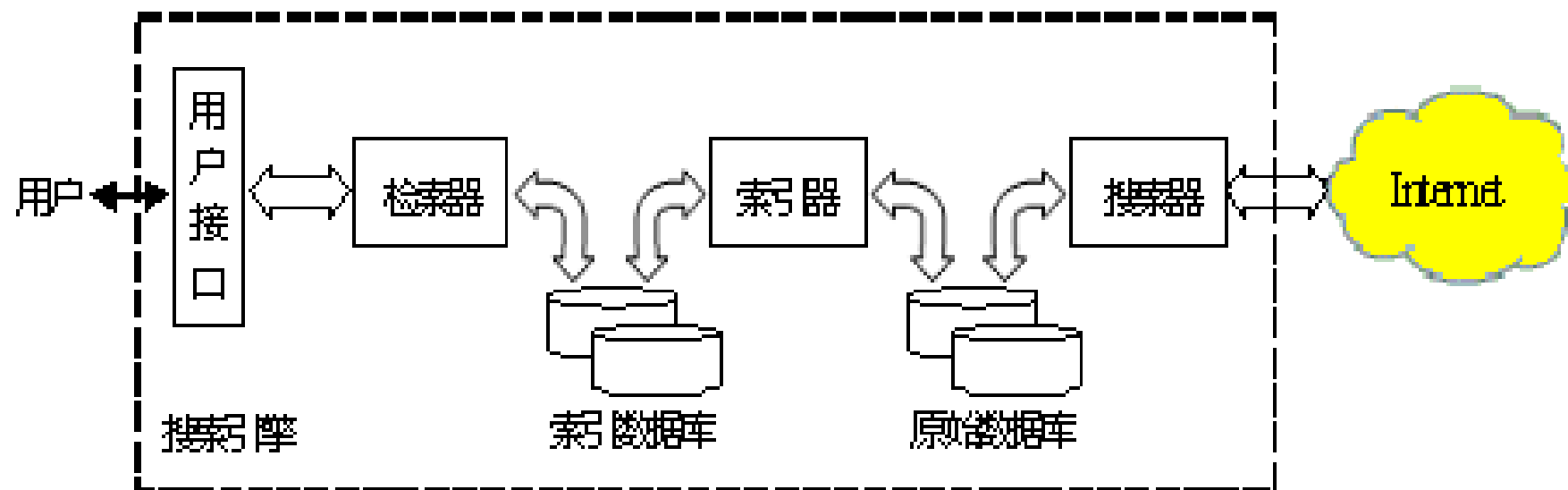


图 3-17 搜索引擎体系结构

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

全文搜索引擎的构成

- (1) 搜索器，负责从 Web 上抓取网页，为提高网页搜集速度，通常可以启动上百个搜集器同时工作。搜集器同时对搜集回来的网页内容进行分析处理，包括调用**切词**软件以提取关键词和摘要、提取网页URL、记录网页元信息（如作者、修改日期、长度等），并将这些内容存入原始数据库。
- (2) 索引器，将原始数据库的内容重新组织，建立索引数据库，以提高检索效率。
- (3) 检索器，根据用户输入的查询项和索引数据库的内容，找到匹配的网页后，进行相关度计算并排序，然后通过用户接口返回给用户。
- (4) 用户接口，接受用户的查询请求，将它转发给检索器。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

网页索引数据库

- 搜索器（蜘蛛程序） 蜘蛛程序获得网页等资源信息保存到原始数据库
- 根据特定的权值计算方法进行计算，得到网页中每一个关键词的权值，存入网页索引数据库，并建立索引数据表，以实现快速查询。

表 3-10 索引数据表结构

序号	记录项	说明
1	关键词列表	网页包含的关键词
2	关键词权重	关键字在网页中的比重
3	快照	页面内容摘要
4	网页重要性度量	标识网页的重要性，用于结果页面的显示排名
5	URL-ID	网址（域名或 IP 地址）

算法

一、算法基础

- 3.1 问题与问题求解
- 3.2 算法与算法分析
- 3.3 算法设计及算法分类

二、常见算法

- 3.4 搜索问题与查找算法
- 3.5 排序问题及排序算法
- 3.6 网络搜索问题

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例—搜索引擎的使用

• 用户接口—搜索条件输入页面





算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

举例（续）

• 用户接口—搜索结果页面





网页搜索中的查找和排序

- 网页搜索的问题
 - 搜索结果页面多。不同于传统的查找类算法
 - 结果如何排序?
- 基于链接的评价
 - 一个网页的重要性取决于它被其它网页链接的数量，特别是一些已经被认定是“重要”的网页的链接数量。
 - 互联网商业化因素，这种评价体制在某种程度上缺乏客观性，因为链接可以人为故意操作。
- 基于访问大众性的评价
 - 多数人选择访问的网站就是最重要的网站
 - 趋众性特点或水军，这种评价体制与基于链接的评价有着同样的缺点

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



网页排名的PageRank排序算法

- 拉里·佩奇 (Larry Page) 于1997年在斯坦福大学读研究生时开发
- PageRank思想
 - 基于接入链接的数量和重要性 (被知名网站链接则重要性提高) 对网页进行评级, 也就是通过网络的集体智慧确定哪些网站最有用
 - PageRank根据网站的外部链接和内部链接的数量和质量来衡量网站的价值, 每个到页面的链接都是对该页面的一次投票 (Vote), 被链接的越多, 就意味着被其他网站投票越多。

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

网页链接关系

- 正向链接，一种是网页到其它网页的超链接
- 反向链接，其他网页到该网页的超链接

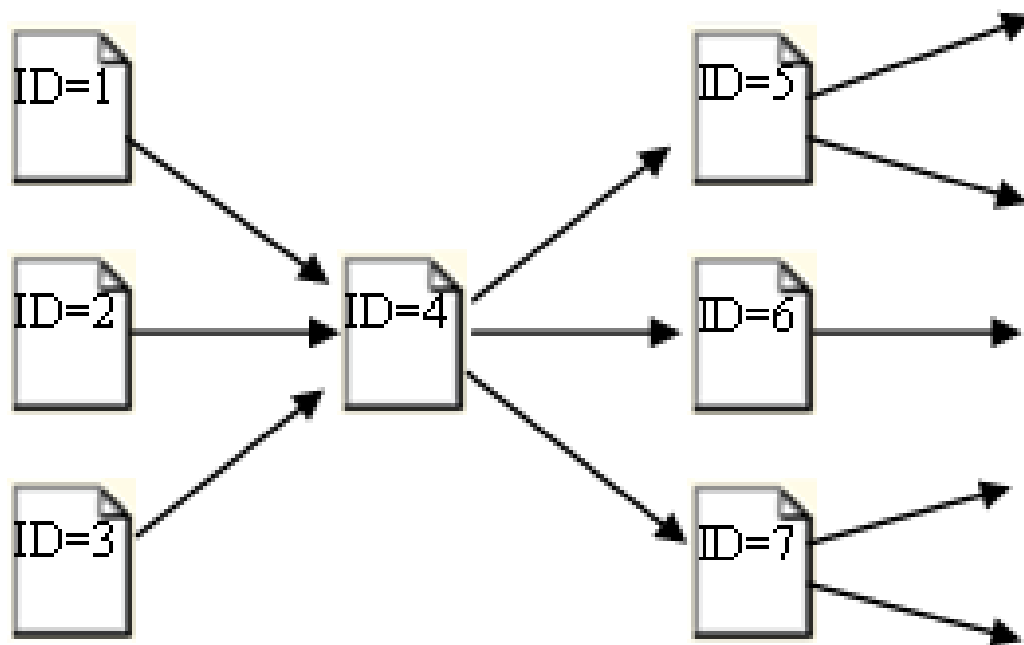


图 3-18 网页链接关系图示

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

PageRank值及其计算

• 核心思想

- 一个网页如果被很多其他重要的网页链接(引用), 那这个网页就更重要;
- 重要性是会传递的: 如果A网页很重要, 而且A链接了B, 那么B也会因此变得更重要;
- 每个网页都有一定的“重要性分数”, 这个分数会通过链接关系传递给其他网页, 如果一个网页链接了10个其他网页, 那么它的分数会平均分配给这10个网页;
- 这个过程会不断重复进行,直到每个网页的分数趋于稳定;
- 在现实中, 用户不一定会顺着链接点下去, 因此引入 “阻尼系数 d ”, 通常为0.85, 表示用户有0.85的概率会按照链接继续浏览 (即重要性分数由此传递), 0.15的概率直接输入网址打开某个网页。由此, 最终得到的重要性分数 = $d \times (\text{从其他页面获得的分数}) + (1-d)/N$, 其中 N 表示总网页数。

【谷歌PageRank算法的工作原理: 理解搜索引擎排名的关键 - Spanning Tree】

https://www.bilibili.com/video/BV1FZ421M7dm/?share_source=copy_web&vd_source=f29334cd47f8c16971c64516ce205b6e

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

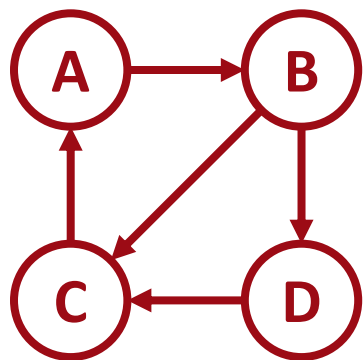
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

PageRank值及其计算



四个网页的初始分值都是0.25;
d=0.85

第一轮计算:

$$PR(A) = \frac{0.15}{4} + 0.85 \times PR(C) = 0.0375 + 0.85 \times 0.25 = 0.25$$

$$PR(B) = \frac{0.15}{4} + 0.85 \times PR(A) = 0.0375 + 0.85 \times 0.25 = 0.25$$

$$PR(C) = \frac{0.15}{4} + 0.85 \times \left(\frac{PR(B)}{2} + PR(D) \right) = 0.0375 + 0.85 \times (0.125 + 0.25) = 0.357$$

$$PR(D) = \frac{0.15}{4} + 0.85 \times \frac{PR(B)}{2} = 0.0375 + 0.85 \times 0.125 = 0.144$$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

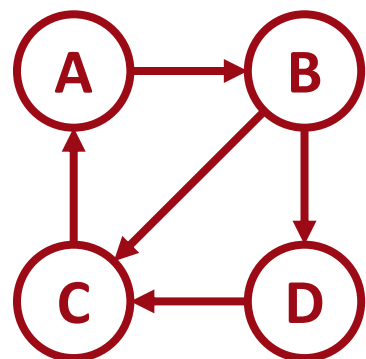
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

PageRank值及其计算



网页	第一轮
A	0.25
B	0.25
C	0.357
D	0.144

第二轮计算:

$$PR(A) = \frac{0.15}{4} + 0.85 \times PR(C) = 0.0375 + 0.85 \times 0.357 = 0.341$$

$$PR(B) = \frac{0.15}{4} + 0.85 \times PR(A) = 0.0375 + 0.85 \times 0.341 = 0.327$$

$$PR(C) = \frac{0.15}{4} + 0.85 \times \left(\frac{PR(B)}{2} + PR(D) \right) = 0.0375 + 0.85 \times \left(\frac{0.327}{2} + 0.144 \right) = 0.299$$

$$PR(D) = \frac{0.15}{4} + 0.85 \times \frac{PR(B)}{2} = 0.0375 + 0.85 \times \frac{0.327}{2} = 0.176$$

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

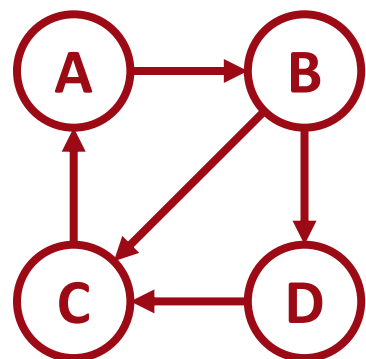
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

PageRank值及其计算



第三轮计算:

$$PR(A) = 0.0375 + 0.85 \times 0.299 = 0.292$$

$$PR(B) = 0.0375 + 0.85 \times 0.292 = 0.285$$

$$PR(C) = 0.0375 + 0.85 \times \left(\frac{0.285}{2} + 0.176 \right) = 0.308$$

$$PR(D) = 0.0375 + 0.85 \times \frac{0.285}{2} = 0.159$$

网页	第一轮	第二轮
A	0.25	0.341
B	0.25	0.327
C	0.357	0.299
D	0.144	0.176

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

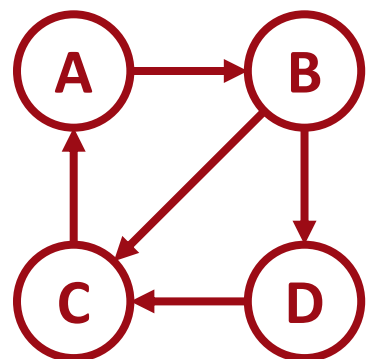
二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

PageRank值及其计算



第四轮计算:

$$PR(A) = 0.0375 + 0.85 \times 0.308 = 0.299$$

$$PR(B) = 0.0375 + 0.85 \times 0.299 = 0.292$$

$$PR(C) = 0.0375 + 0.85 \times \left(\frac{0.292}{2} + 0.159 \right) = 0.297$$

$$PR(D) = 0.0375 + 0.85 \times \frac{0.292}{2} = 0.162$$

网页	第一轮	第二轮	第三轮
A	0.25	0.341	0.292
B	0.25	0.327	0.285
C	0.357	0.299	0.308
D	0.144	0.176	0.159

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

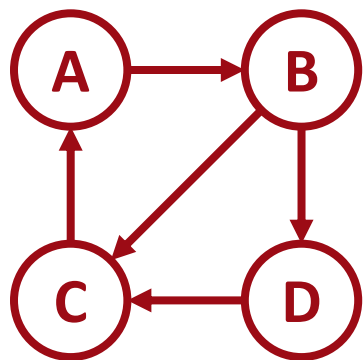
二、常见算法

3.4 搜索问题与查找算法

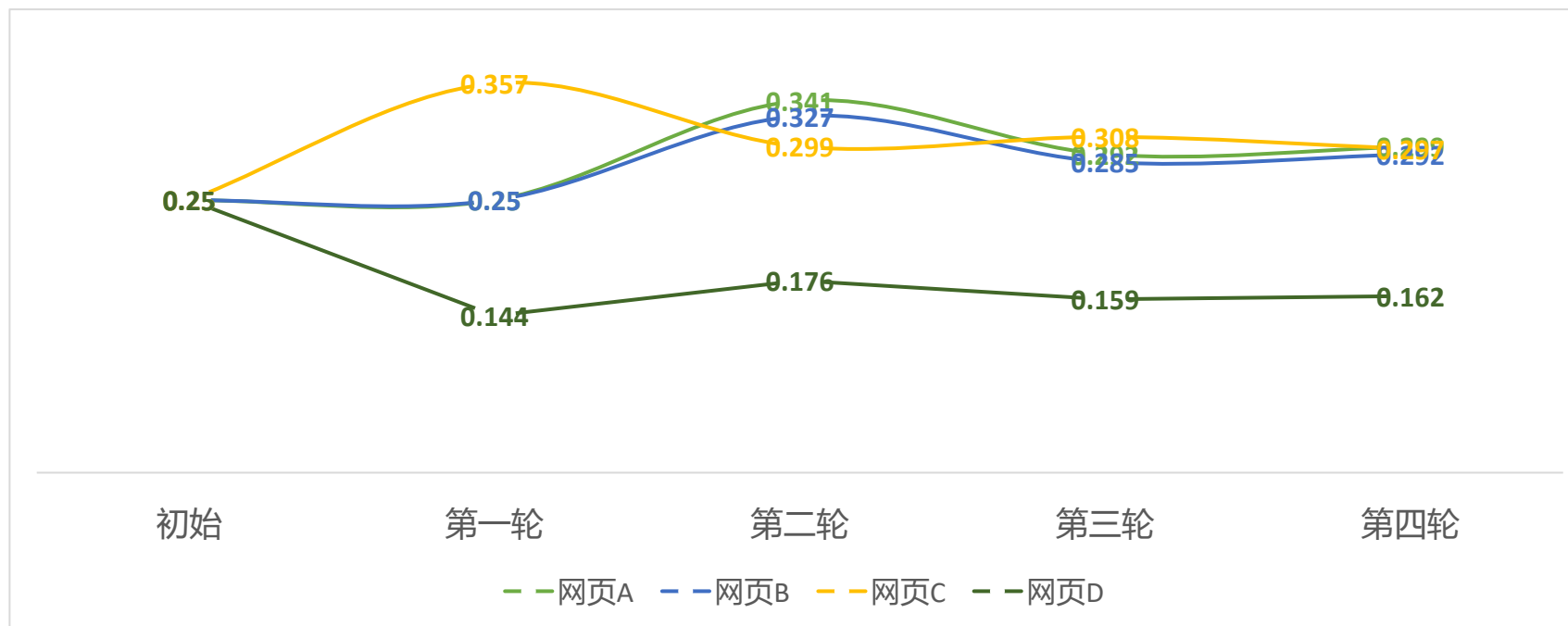
3.5 排序问题及排序算法

3.6 网络搜索问题

PageRank值及其计算



网页	初始	第一轮	第二轮	第三轮	第四轮
A	0.25	0.25	0.341	0.292	0.299
B	0.25	0.25	0.327	0.285	0.292
C	0.25	0.357	0.299	0.308	0.297
D	0.25	0.144	0.176	0.159	0.162



算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题



搜索引擎的启示

- 互联网正在成为人们创新的重要源泉
- PR值的计算法则导致了人们对链接的着魔。人们忙于争夺、交换甚至销售链接
- Google修改PR值计算规则
 - 来自缺乏内容的“link farm”（链接工厂）网站的链接将不会提供页面的PageRank
 - 从PageRank较高的页面得到链接但是内容毫不相关，也不会提供页面的PageRank
 - 降低对PageRank的更新频率

算法

一、算法基础

3.1 问题与问题求解

3.2 算法与算法分析

3.3 算法设计及算法分类

二、常见算法

3.4 搜索问题与查找算法

3.5 排序问题及排序算法

3.6 网络搜索问题

算法

一、算法基础

- 3.1 问题与问题求解
- 3.2 算法与算法分析
- 3.3 算法设计及算法分类

二、常见算法

- 3.4 搜索问题与查找算法
- 3.5 排序问题及排序算法
- 3.6 网络搜索问题

本章小结

程序与算法基础

